BUKU AJAR
ALGORITMA DAN STRUKTUR DATA

Tim Penulis :
Moh. Erkamim, S.Kom., M.Kom
Iim Abdurrohim, S.T., M.T
Siti Yuliyanti, S.T., M.Kom
Rahmat Karim, S.Si., M.Si
Alkautsar Rahman, S.T., M.Kom
Tb. M. Adrie Admira, S.T., M.Kom
Achmad Ridwan, S.Kom., M.Kom

SONPEDIA.COM
PT. Sonpedia Publishing Indonesia

BUKU AJAR
ALGORITMA DAN STRUKTUR DATA

Tim Penulis :
Moh. Erkamim, S.Kom., M.Kom
Iim Abdurrohim, S.T., M.T
Siti Yuliyanti, S.T., M.Kom
Rahmat Karim, S.Si., M.Si
Alkautsar Rahman, S.T., M.Kom
Tb. M. Adrie Admira, S.T., M.Kom
Achmad Ridwan, S.Kom., M.Kom

Penerbit

SONPEDIA.COM
PT. Sonpedia Publishing Indonesia

BUKU AJAR
ALGORITMA DAN STRUKTUR DATA

Tim Penulis :
Moh. Erkamim, S.Kom., M.Kom
Iim Abdurrohim, S.T., M.T
Siti Yuliyanti, S.T., M.Kom
Rahmat Karim, S.Si., M.Si
Alkautsar Rahman, S.T., M.Kom
Tb. M. Adrie Admira, S.T., M.Kom
Achmad Ridwan, S.Kom., M.Kom

ISBN : 978-623-8483-85-3

Editor :
Efitra
Penyunting :
Nur Safitri
Desain sampul dan Tata Letak :
Yayan Agusdi
Penerbit :
PT. Sonpedia Publishing Indonesia
Redaksi :
Jl. Kenali Jaya No 166 Kota Jambi 36129 Telp. +6282177858344
Email : sonpediapublishing@gmail.com
Website : www.buku.sonpedia.com

Anggota IKAPI : 006/JBI/2023

Cetakan Pertama, Januari 2024

Hak cipta dilindungi undang-undang
Dilarang memperbanyak karya tulis ini dalam bentuk dan dengan cara Apapun tanpa ijin dari penerbit

KATA PENGANTAR

Puji syukur kepada Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga penulis dapat menyelesaikan buku ini dengan baik. Buku ini berjudul **"*BUKU AJAR ALGORITMA DAN STRUKTUR DATA*".** Tidak lupa kami ucapkan terima kasih bagi semua pihak yang telah membantu dalam penulisan dan penerbitan buku ini.

Dalam era di mana teknologi terus berkembang dengan cepat, pemahaman yang kuat tentang algoritma dan struktur data menjadi semakin penting. Buku ini disusun dengan penuh dedikasi untuk memberikan pemahaman yang kokoh tentang prinsip-prinsip dasar tersebut, mulai dari konsep dasar hingga aplikasi praktisnya dalam pemrograman.

Buku Ajar ini disusun sebagai buku panduan komprehensif yang menjelajahi kompleksitas dan mendalamnya tentang ilmu sistem informasi. Buku ini dapat digunakan oleh pendidik dalam melaksanakan kegiatan pembelajaran di bidang ilmu sistem informasi dan diberbagai bidang Ilmu terkait lainnya. Buku ini dapat digunakan sebagai panduan dan referensi mengajar mata kuliah algoritman dan struktur data dan menyesuaikan dengan Rencana Pembelajaran Semester tingkat Perguruan Tinggi masing-masing.

Secara garis besar, buku ajar ini pembahasannya mulai dari pengenalan pemrograman berorientasi objek, struktur data linked list, stack dan queue, struktur data tree and binary tree, pengantar graf (Representasi Graf dan Penelusuran Graf). Selain itu materi mengenai Minimum Spanning Tree, dan Geometri Algorithm (Convex Hull) dibahas secara mendalam. Buku ajar ini disusun secara sistematis, ditulis dengan bahasa yang jelas dan mudah dipahami, dan dapat digunakan dalam kegiatan pembelajaran.

Buku ini mungkin masih terdapat kekurangan dan kelemahan. Oleh karena itu, saran dan kritik para pemerhati sungguh penulis harapkan. Semoga buku ajar ini memberikan manfaat dan menambah khasanah ilmu pengetahuan dalam pembelajaran

Jambi, Januari 2024
Tim Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iv
KEGIATAN BELAJAR 1 PENGENALAN PEMROGRAMAN BERORIENTASI OBJEK 1
DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN
A. PENGERTIAN PEMROGRAMAN BERORIENTASI OBJEK 2
B. SEJARAH DAN EVOLUSI PEMROGRAMAN BERORIENTASI OBJEK. 3
C. KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK 5
D. PRINSIP-PRINSIP PEMROGRAMAN BERORIENTASI OBJEK 6
E. KEUNTUNGAN PEMROGRAMAN BERORIENTASI OBJEK 8
F. PEMROGRAMAN BERORIENTASI OBJEK DALAM ALGORITMA DAN STRUKTUR DATA..... 10
G. RANGKUMAN 11
H. TES FORMATIF 12
I. LATIHAN..... 13
KEGIATAN BELAJAR 2 STRUKTUR DATA LINKED LIST 14
DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN
A. DASAR-DASAR LINKED LIST..... 15
B. OPERASI DASAR LINKED LIST 17
C. PENCARIAN DAN AKSES ELEMEN 20
D. MANIPULASI LINKED LIST 22
E. APLIKASI LINKED LIST DALAM PEMROGRAMAN 23
F. ANALISIS KINERJA LINKED LIST 27
G. RANGKUMAN 28
H. TES FORMATIF 29

I. LATIHAN 31

KEGIATAN BELAJAR 3 STACK (TUMPUKAN) & QUEUE (ANTRIAN) 32

DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN

A. KONSEP DASAR STACK 33

B. KARAKTERISTIK STACK 36

C. APLIKASI STACK 37

D. KONSEP DASAR QUEUE 41

F. CIRCULAR QUEUE DENGAN ARRAY 46

G. RANGKUMAN 50

H. TES FORMATIF 51

I. LATIHAN 51

KEGIATAN BELAJAR 4 STRUKTUR DATA *TREE* DAN BINARY *TREE* 52

DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN

A. PENGANTAR *TREE* 53

B. BINARY TREE 57

C. RANGKUMAN 64

D. TES FORMATIF 65

E. LATIHAN 67

KEGIATAN BELAJAR 5 PENGANTAR GRAF: REPRESENTASI GRAF DAN PENELUSURAN GRAF (DEPTH FIRST BREATH FIRST SEARCH) 68

DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN

A. PENGERTIAN DAN SEJARAH 69

B. DEFINISI GRAF 71

C. JENIS JENIS GRAF 73

D. DEPTH-FIRST SEARCH (DFS) 78

E. BREADTH FIRST SEARCH (BFS) 80

F. RANGKUMAN 81

G. TES FORMATIF 82
H. LATIHAN 82
KEGIATAN BELAJAR 6 MINIMUM SPANNING TREE (MST) 83
DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN
A. PENGERTIAN MINIMUM SPANNIG TREE (MST) 84
B. FUNGSI MINIMUM SPANNING TREE DALAM TEORI GRAF 85
C. MANFAAT DAN FUNGSI PENERAPAN SPANNING TREE DALAM BERBAGAI BIDANG 86
D. ALGORITMA PRIM 88
E. ALGORITMA SOLIN 90
F. ALGORITMA KRUSKAL 93
G. RANGKUMAN 95
H. TES FORMATIF 96
I. LATIHAN 97
KEGIATAN BELAJAR 7 GEOMETRI ALGORITHM 98
DESKRIPSI, KOMPETENSI DAN PETA KONSEP PEMBELAJARAN
A. PENGENALAN GEOMETRI KOMPUTASIONAL 99
B. PENGERTIAN DAN KONSEP ALGORITMA GEOMETRI 100
C. REPRESENTASI TITIK, GARIS, POLIGON 102
D. CONVEX HULL 103
E. CONVEX HULL DENGAN DIVIDE AND CONQUER 107
F. IDE DIVIDE AND CONQUER 108
G. RANGKUMAN 111
H. TES FORMATIF 112
I. LATIHAN 113

DAFTAR PUSTAKA 114
TENTANG PENULIS 121

KEGIATAN BELAJAR 1

PENGENALAN PEMROGRAMAN BERORIENTASI OBJEK

DESKRIPSI PEMBELAJARAN

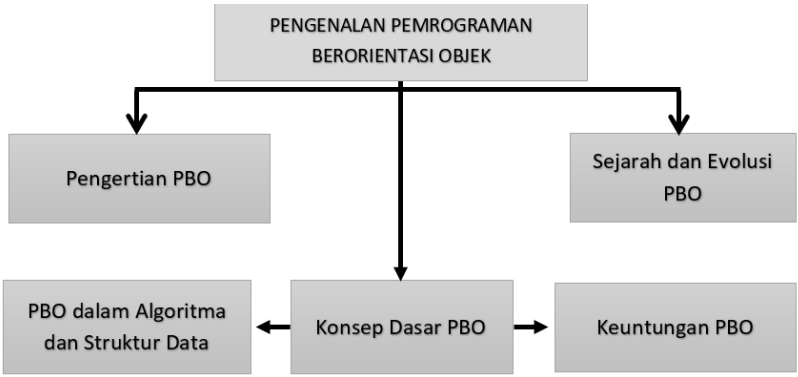
Pada bab ini mahasiswa mempelajari pengenalan dan konsep dasar teoritis tentang Pemrograman Berorientasi Objek (PBO). Diharapkan mahasiswa memahami konsep dasar PBO, termasuk perbedaan antara pemrograman prosedural dan berorientasi objek, serta mempelajari tentang kelas, objek, enkapsulasi, pewarisan, dan polimorfisme.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu menguraikan definisi paradigma pemrograman yang berfokus pada penggunaan objek.
2. Mempu menjelaskan fungsi dan manfaat Pemrograman Berorientasi Objek
3. Mampu menjelaskan konsep dasar kelas, objek, enkapsulasi, pewarisan, dan polimorfisme.

PETA KONSEP PEMBELAJARAN



A. PENGERTIAN PEMROGRAMAN BERORIENTASI OBJEK

Pemrograman Berorientasi Objek (PBO) adalah paradigma pemrograman yang berfokus pada penggunaan objek, yang merupakan entitas yang mengkombinasikan data dan metode untuk berinteraksi dengan data tersebut. Konsep ini dirancang untuk mempermudah pemeliharaan kode dan meningkatkan modularitas serta fleksibilitas. Berbeda dengan pendekatan pemrograman prosedural yang berfokus pada fungsi atau prosedur, PBO mengutamakan objek sebagai inti dari proses pemrograman. Konsep ini dirancang untuk membuat kode lebih modular, fleksibel, dan mudah untuk di-maintain. Dengan PBO, pemrogram dapat memodelkan objek dunia nyata atau konsep abstrak dalam kode, memberikan cara intuitif untuk berinteraksi dengan komponen-komponen dalam program.

Dua konsep inti dalam PBO adalah kelas dan objek. Kelas dapat dianggap sebagai cetakan atau blueprint yang mendefinisikan karakteristik dan perilaku (yaitu, atribut dan metode) dari objek. Sebagai contoh, dalam konteks pengembangan perangkat lunak untuk manajemen perpustakaan, kelas bisa menjadi "Buku",

dengan atribut seperti judul, pengarang, dan ISBN, serta metode seperti meminjam atau mengembalikan buku. Objek adalah instansi dari kelas, yang berarti setiap "Buku" yang spesifik dengan judul dan pengarang tertentu merupakan objek dari kelas "Buku".

Enkapsulasi adalah konsep di mana data dan metode yang berkaitan dikumpulkan menjadi sebuah unit tunggal atau objek. Ini membantu dalam menyembunyikan kompleksitas internal dari penggunaan objek dan memungkinkan pengembang untuk mengubah bagian internal dari objek tanpa mempengaruhi bagian lain dari program. Pewarisan adalah fitur PBO yang memungkinkan kelas baru untuk mewarisi sifat dan kemampuan dari kelas yang sudah ada. Ini memfasilitasi penggunaan kembali kode dan mengurangi redundansi. Polimorfisme adalah kemampuan untuk memproses objek secara berbeda berdasarkan kelas atau tipe data mereka. Ini memungkinkan satu antarmuka untuk digunakan dengan berbagai bentuk data. Dalam PBO, polimorfisme memungkinkan metode dengan nama yang sama untuk memiliki perilaku yang berbeda untuk kelas yang berbeda. Melalui konsep-konsep seperti kelas dan objek, enkapsulasi, abstraksi, pewarisan, dan polimorfisme, PBO menyediakan kerangka kerja yang efisien untuk membangun aplikasi yang kompleks dan mudah di-maintain, dengan menekankan pada modularitas, fleksibilitas, dan penggunaan kembali kode.

B. SEJARAH DAN EVOLUSI PEMROGRAMAN BERORIENTASI OBJEK

Pemrograman Berorientasi Objek (PBO) bermula dari konsep yang lebih tua dalam desain dan arsitektur perangkat lunak. Awalnya dikembangkan pada tahun 1960-an, PBO dipengaruhi oleh bahasa pemrograman seperti Simula, yang diperkenalkan pada tahun 1967 oleh Ole-Johan Dahl dan Kristen Nygaard dari Norwegia. Simula adalah bahasa pertama yang memperkenalkan konsep kelas dan

objek, yang sekarang menjadi pondasi utama PBO. Konsep ini kemudian dikembangkan lebih lanjut dalam bahasa pemrograman Smalltalk pada tahun 1970-an, yang dikenal sebagai bahasa pertama yang sepenuhnya berorientasi objek.

Seiring waktu, PBO mendapatkan popularitas karena kemampuannya untuk memodelkan sistem perangkat lunak yang kompleks secara intuitif. Pada tahun 1980-an, bahasa pemrograman seperti C++ mengadopsi paradigma PBO, menggabungkan konsep objek dengan kekuatan dan fleksibilitas bahasa C, yang sudah luas digunakan. C++ membawa PBO ke dalam pengembangan perangkat lunak mainstream, diikuti oleh bahasa lain seperti Python, Java, dan Ruby. Java, khususnya, dirilis pada pertengahan 1990-an dengan slogan "*Write Once, Run Anywhere*", menekankan portabilitas dan efisiensi dalam pengembangan perangkat lunak.

Dengan berkembangnya PBO, pendidikan dalam pemrograman juga mengalami perubahan. Universitas dan lembaga pendidikan mulai mengadopsi PBO sebagai bagian penting dari kurikulum ilmu komputer, mengajarkan konsep seperti kelas, objek, pewarisan, dan polimorfisme. Dalam industri, PBO digunakan dalam berbagai aplikasi, dari pengembangan perangkat lunak sistem hingga aplikasi web dan mobile. Keberhasilannya dalam mengatasi masalah desain perangkat lunak yang kompleks menjadikannya pilihan populer di antara pengembang dan perusahaan.

Evolusi PBO juga mempengaruhi cara perangkat lunak dirancang dan dikembangkan. Metodologi seperti Agile dan Scrum, yang menekankan iterasi, modularitas, dan pengujian yang sering, cocok dengan pendekatan berorientasi objek. PBO memungkinkan pengembang untuk membangun aplikasi dalam komponen yang lebih kecil dan dapat diuji, yang dapat dikembangkan secara independen dan kemudian diintegrasikan. Ini juga mendorong penggunaan kembali kode, yang berkontribusi pada efisiensi dan pengurangan redundansi dalam pengembangan perangkat lunak.

Meskipun PBO telah menjadi salah satu paradigma pemrograman yang paling berpengaruh, ia tidak tanpa tantangan. Kritik terhadap PBO mencakup kompleksitas yang bisa timbul dari pewarisan berlebihan dan kesulitan dalam memahami arsitektur yang besar dan terdistribusi. Namun, evolusi berkelanjutan dalam desain bahasa pemrograman dan praktek pengembangan perangkat lunak terus mengatasi tantangan ini. Dengan berkembangnya teknologi baru seperti pemrograman fungsional dan reaktif, PBO terus beradaptasi, menunjukkan fleksibilitas dan ketahanannya sebagai paradigma pemrograman utama dalam industri perangkat lunak.

C. KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK

Pemrograman Berorientasi Objek (PBO) berupaya untuk menerapkan prinsip dan konsep yang ada dalam dunia nyata ke dalam pemrograman, dengan tujuan untuk membuat kode yang lebih bersih, mudah dipahami, dan dapat digunakan kembali. Berikut adalah beberapa konsep dasar PBO yang penting untuk dipahami:

1. **Kelas dan Objek**

   Kelas adalah cetakan atau blueprint dari mana objek dibuat. Kelas mendefinisikan atribut (sifat-sifat) dan metode (fungsi-fungsi) yang akan dimiliki oleh objek yang dihasilkan. Objek adalah instansi atau contoh dari kelas. Jika kelas adalah rancangan, objek adalah realisasi nyata dari rancangan tersebut. Setiap objek memiliki identitas, perilaku, dan status yang diwarisi dari kelasnya.

2. **Enkapsulasi**

   Enkapsulasi adalah mekanisme untuk menyembunyikan detail implementasi dari pengguna dan hanya mengekspos fungsionalitas yang relevan. Dalam prakteknya, ini berarti bahwa data internal dari objek dilindungi dari akses langsung dan modifikasi oleh kode luar. Enkapsulasi juga memungkinkan

pemrogram untuk mengubah bagian internal objek tanpa mempengaruhi bagian lain dari program.

3. **Abstraksi**
   Abstraksi memungkinkan pemrogram untuk menyederhanakan kompleksitas dengan mengabaikan detail yang kurang penting dan hanya fokus pada aspek yang relevan. Dalam PBO, ini berarti menciptakan model yang menangkap aspek esensial dari entitas dunia nyata atau konsep dalam konteks program, tanpa terlibat dalam kompleksitas yang tidak perlu.
4. **Pewarisan**
   Pewarisan adalah mekanisme di mana kelas baru dapat dibuat berdasarkan kelas yang sudah ada. Kelas baru ini, yang dikenal sebagai kelas turunan, mewarisi atribut dan metode dari kelas induknya, sambil menambahkan atau memodifikasi fitur-fitur khusus. Pewarisan memfasilitasi penggunaan kembali kode dan hierarki kelas yang logis.
5. **Polimorfisme**
   Polimorfisme memungkinkan objek untuk diolah sebagai instansi dari kelas induknya atau antarmuka. Ini berarti bahwa metode yang sama dapat memiliki perilaku yang berbeda-beda pada objek yang berbeda, bahkan jika objek tersebut berasal dari kelas yang sama. Polimorfisme meningkatkan fleksibilitas dan kemampuan untuk menangani kasus-kasus khusus.

D. PRINSIP-PRINSIP PEMROGRAMAN BERORIENTASI OBJEK

Pemrograman Berorientasi Objek (PBO) didasarkan pada empat prinsip utama yang membentuk fondasi dari pendekatan ini terhadap pengembangan perangkat lunak. Prinsip-prinsip ini adalah Enkapsulasi, Abstraksi, Pewarisan, dan Polimorfisme. Masing-masing prinsip ini memainkan peran krusial dalam membantu

pemrogram untuk menciptakan kode yang lebih bersih, lebih modular, dan lebih mudah untuk dipelihara.

1. **Enkapsulasi**

   Enkapsulasi adalah tentang menggabungkan data (atribut) dan metode (fungsi atau prosedur) yang beroperasi pada data tersebut menjadi satu unit, atau objek. Tujuannya adalah untuk menyembunyikan detail internal objek dari dunia luar.

   Penerapan dalam praktiknya, enkapsulasi dapat dicapai melalui penggunaan modifier akses (seperti private, public, dan protected) yang mengontrol akses ke komponen objek. Ini memungkinkan objek untuk menyembunyikan data internalnya dan mengekspos hanya apa yang diperlukan.

2. **Abstraksi**

   Abstraksi memungkinkan pemrogram untuk menyederhanakan kompleksitas dengan fokus pada apa yang dilakukan objek, bukan bagaimana objek melakukan tugas-tugas tersebut. Ini memudahkan pemahaman dan penggunaan objek. Penerapan dalam PBO, abstraksi sering dicapai melalui definisi interface atau kelas abstrak yang menentukan 'apa' yang harus dilakukan tanpa mengimplementasikan 'bagaimana' melakukannya, meninggalkan detail implementasi kepada kelas turunan.

3. **Pewarisan**

   Pewarisan memungkinkan kelas baru untuk mewarisi sifat dan perilaku dari kelas yang sudah ada. Ini mempromosikan penggunaan kembali kode dan membuat kode lebih terorganisir. Penerapan dalam pewarisan, kelas yang ada (disebut kelas induk atau *superclass*) memberikan fondasi bagi kelas baru (kelas anak atau *subclass*). Kelas anak mewarisi atribut dan metode dari kelas induk, dan dapat menambah atau mengubah beberapa aspeknya.

4. **Polimorfisme**

   Polimorfisme adalah kemampuan untuk mengolah objek yang berbeda dengan cara yang sama. Ini memberikan fleksibilitas dalam menggunakan objek dari kelas yang berbeda. Penerapan dalam PBO, polimorfisme biasanya diimplementasikan melalui

metode yang di *override* dalam kelas turunan atau melalui *interface*. Ini memungkinkan metode dengan nama yang sama untuk memiliki perilaku yang berbeda pada objek dari kelas yang berbeda.

Prinsip-prinsip PBO Enkapsulasi, Abstraksi, Pewarisan, dan Polimorfisme bersama-sama membentuk sebuah paradigma yang kuat untuk pemrograman perangkat lunak. Dengan memahami dan menerapkan prinsip-prinsip ini, pemrogram dapat mengembangkan aplikasi yang lebih fleksibel, efisien, dan mudah dipelihara, serta lebih mudah untuk diadaptasi dan dikembangkan di masa depan. Pendekatan ini sangat cocok untuk mengatasi kompleksitas dalam proyek perangkat lunak skala besar, memungkinkan pembangunan sistem yang lebih terstruktur dan terorganisir.

E. KEUNTUNGAN PEMROGRAMAN BERORIENTASI OBJEK

Pemrograman Berorientasi Objek (PBO) adalah paradigma yang memiliki banyak keuntungan, terutama dalam pengembangan perangkat lunak yang kompleks dan skala besar. Berikut adalah beberapa keuntungan utama dari penggunaan PBO:

1. **Modularitas**
   Definisi dan manfaat modularitas dalam PBO merujuk pada konsep pembagian program menjadi unit-unit kecil dan terpisah, yang dikenal sebagai objek. Hal ini memudahkan pemeliharaan dan pengelolaan kode. Implementasi dengan menggunakan kelas dan objek, pemrogram dapat membangun komponen-komponen yang dapat digunakan kembali di berbagai bagian program atau bahkan di proyek yang berbeda, yang meningkatkan efisiensi dan konsistensi.
2. **Penggunaan Kembali Kode (*Reusability*)**
   Definisi dan manfaat PBO memungkinkan pemrogram untuk menggunakan kembali kode yang telah ada, yang mengurangi duplikasi kerja dan mempercepat proses pengembangan.

Implementasi Melalui konsep pewarisan, kelas-kelas baru dapat dibuat dengan menggunakan sifat dan perilaku dari kelas yang sudah ada. Hal ini meminimalkan kebutuhan untuk menulis ulang kode yang serupa.

3. **Pemeliharaan dan Skalabilitas**
   Definisi dan manfaat PBO mempermudah pemeliharaan dan modifikasi program karena perubahan pada bagian tertentu dari kode tidak mempengaruhi bagian lain. Implementasi ketika perubahan diperlukan, pemrogram hanya perlu memodifikasi kelas tertentu tanpa harus mengubah kode di tempat lain. Ini menjadikan aplikasi lebih mudah untuk ditingkatkan dan dikembangkan.
4. **Abstraksi dan Enkapsulasi**
   Definisi dan manfaat Abstraksi memungkinkan pemrogram untuk fokus pada apa yang dilakukan objek, bukan bagaimana hal itu dilakukan. Enkapsulasi melindungi data dan fungsi dari interferensi atau penyalahgunaan eksternal. Implementasi objek menyediakan antarmuka yang jelas untuk interaksi, sementara detail internalnya tersembunyi, memudahkan pemahaman dan penggunaan kode.
5. **Polimorfisme**
   Definisi dan manfaat polimorfisme di PBO memberikan fleksibilitas dalam penggunaan metode dan objek, memungkinkan satu antarmuka untuk digunakan dengan berbagai bentuk data. Implementasi hal ini memungkinkan kelas-kelas yang berbeda untuk didefinisikan dengan metode yang sama, tetapi dengan implementasi yang berbeda, meningkatkan fleksibilitas dan kesederhanaan dalam pengelolaan kode.

Keuntungan-keuntungan PBO seperti modularitas, penggunaan kembali kode, kemudahan pemeliharaan dan skalabilitas, serta abstraksi dan polimorfisme, menjadikannya paradigma yang sangat cocok untuk pengembangan perangkat lunak yang kompleks dan dinamis. PBO tidak hanya memudahkan proses pengembangan tetapi juga memperkuat kualitas dan keandalan dari perangkat

lunak yang dihasilkan. Pendekatan ini sangat membantu dalam memastikan bahwa proyek-proyek perangkat lunak dapat berkembang dan beradaptasi dengan kebutuhan yang berubah seiring waktu.

F. PEMROGRAMAN BERORIENTASI OBJEK DALAM ALGORITMA DAN STRUKTUR DATA

Pemrograman Berorientasi Objek (PBO) menawarkan pendekatan yang unik dan efektif dalam implementasi algoritma dan struktur data. Dalam konteks ini, PBO tidak hanya memfasilitasi organisasi kode yang lebih baik tetapi juga menyediakan kerangka kerja yang memudahkan pemahaman konsep yang mendasarinya.

1. **Objektifikasi Struktur Data**
   PBO memungkinkan struktur data tradisional seperti array, list, stack, queue, trees, dan graphs untuk diimplementasikan sebagai objek. Hal ini memberikan struktur yang lebih jelas dan memudahkan manajemen data. Manfaat dengan mendefinisikan struktur data sebagai kelas, pemrogram dapat menambahkan metode yang relevan untuk manipulasi data, seperti penambahan, penghapusan, atau pencarian, yang membuat kode lebih intuitif dan mudah dipelihara.
2. **Implementasi Algoritma dengan PBO**
   Terdapat banyak algoritma dari pencarian dan pengurutan hingga algoritma yang lebih kompleks, dapat diimplementasikan melalui pendekatan PBO. Ini memungkinkan untuk mengabstraksi proses algoritma dan fokus pada logika inti.
   Manfaat PBO membantu dalam mendefinisikan algoritma sebagai set metode dalam kelas, memungkinkan pemrogram untuk mengisolasi fungsionalitas algoritma dari struktur data yang mereka operasikan, sehingga meningkatkan keterbacaan dan penggunaan kembali kode.
3. **Abstraksi dan Encapsulation dalam Algoritma**

PBO menekankan pada abstraksi dan enkapsulasi yang memungkinkan pemrogram untuk menyembunyikan detail kompleks dari implementasi algoritma, mengekspos hanya fungsi atau metode yang diperlukan untuk penggunaan. Hal ini memudahkan pemahaman dan penggunaan algoritma, serta memungkinkan pemrogram untuk mengubah implementasi internal tanpa mempengaruhi kode lain yang menggunakan algoritma tersebut.

4. **Pewarisan dan Polimorfisme dalam Struktur Data**
   Konsep pewarisan memungkinkan struktur data untuk diturunkan dari struktur data yang lebih umum, sementara polimorfisme memungkinkan untuk menggunakan struktur data ini dalam berbagai konteks. Hal ini memudahkan pembuatan struktur data yang lebih spesifik tanpa harus menulis ulang kode yang ada, dan memungkinkan struktur data yang berbeda untuk digunakan secara interchangeably dimana mereka dibutuhkan.

Penerapan PBO dalam algoritma dan struktur data tidak hanya meningkatkan keterbacaan dan organisasi kode, tetapi juga memfasilitasi pengembangan yang lebih modular dan fleksibel. Dengan memisahkan kekhawatiran antara logika algoritma dan manajemen data, PBO memperkuat fondasi untuk aplikasi yang kompleks dan dinamis, membantu pemrogram untuk menangani berbagai tantangan dalam pengembangan perangkat lunak.

G. RANGKUMAN

Pemrograman Berorientasi Objek (PBO) adalah paradigma yang memfokuskan pada penggunaan objek entitas yang menggabungkan data dan metode, untuk mempermudah pemeliharaan kode dan meningkatkan modularitas serta fleksibilitas. Konsep ini, bermula pada tahun 1960-an dan berevolusi melalui bahasa seperti **Simula** dan **C++**, menyederhanakan pemodelan sistem perangkat lunak yang

kompleks dan telah menjadi bagian penting dari pendidikan pemrograman. Dalam PBO, konsep **kelas** dan **objek**, **enkapsulasi** (menyembunyikan detail internal), **pewarisan** (kelas baru mewarisi dari kelas yang ada), dan **polimorfisme** (metode yang berbeda untuk kelas yang berbeda) memungkinkan pembangunan aplikasi yang lebih efisien, modular, dan mudah dipelihara. PBO juga memainkan peran penting dalam implementasi algoritma dan struktur data, meningkatkan keterbacaan dan organisasi kode, serta memfasilitasi pengembangan yang lebih modular dan fleksibel. Pendekatan ini membantu mengatasi tantangan dalam pengembangan perangkat lunak, menjadikannya paradigma yang cocok untuk proyek skala besar dan dinamis.

H. TES FORMATIF

1. Apa yang dimaksud dengan 'Objek' dalam Pemrograman Berorientasi Objek?
   a. Sebuah fungsi atau prosedur.
   b. Entitas yang menggabungkan data dan metode.
   c. Sebuah tipe data primitif.
   d. Sebuah bahasa pemrograman.

2. Manakah dari berikut ini yang merupakan contoh 'Kelas' dalam PBO?
   a. Jumlah pelanggan dalam sebuah database.
   b. Sebuah aplikasi mobile.
   c. Template untuk objek 'Mobil' dengan atribut dan metode.
   d. Fungsi untuk menghitung luas lingkaran.

3. Apa tujuan utama dari enkapsulasi dalam PBO?
   a. Meningkatkan kecepatan program.
   b. Menyembunyikan detail internal objek.
   c. Mempermudah akses ke semua metode dan data.

d. Mengurangi ukuran kode sumber.

4. Pilih pernyataan yang benar tentang pewarisan dalam PBO.
   a. Pewarisan memungkinkan kelas baru untuk mewarisi sifat dari lebih dari satu kelas induk.
   b. Pewarisan menghilangkan kemungkinan penggunaan kembali kode.
   c. Kelas anak tidak dapat memiliki metode atau atribut tambahan selain yang diwarisi.
   d. Pewarisan hanya digunakan untuk kelas abstrak.

5. Dalam konteks PBO, apa itu Polimorfisme?
   a. Kemampuan untuk mengubah kode sumber tanpa mempengaruhi kinerja.
   b. Kemampuan objek untuk memproses metode dengan cara yang berbeda berdasarkan tipe atau kelasnya.
   c. Konsep dimana objek dari kelas yang berbeda dapat diakses melalui interface yang sama.
   d. Proses menggabungkan dua atau lebih kelas menjadi satu.

I. LATIHAN

1. Jelaskan perbedaan antara 'kelas' dan 'objek' dalam PBO.
2. Berikan contoh sederhana pewarisan dalam PBO.
3. Apa tujuan dari enkapsulasi dalam PBO dan berikan contoh penggunaannya?
4. Bagaimana polimorfisme mempengaruhi desain dan implementasi program dalam PBO?
5. Mengapa modularitas penting dalam PBO?

KEGIATAN BELAJAR 2

STRUKTUR DATA LINKED LIST

DESKRIPSI PEMBELAJARAN

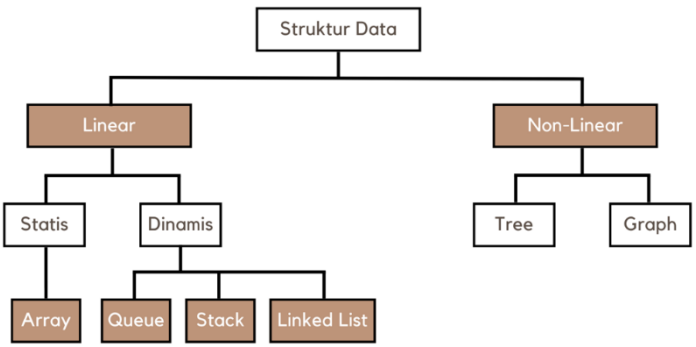
Pada bab ini mahasiswa mempelajari tentang konsep dasar linked list, operasi dasar linked list, dan penerapan linked list dalam berbagai aplikasi. Diharapkan mahasiswa dapat memahami pentingnya struktur data dalam konteks pemrograman dan mendapatkan gambaran awal tentang apa yang akan dipelajari, khususnya terkait linked list.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti matakuliah ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar linked list, termasuk node, pointer, dan operasi dasar linked list..
2. Menerapkan linked list dalam berbagai aplikasi, seperti daftar, stack, dan queue.
3. Menganalisis kinerja linked list untuk berbagai operasi.

PETA KONSEP PEMBELAJARAN



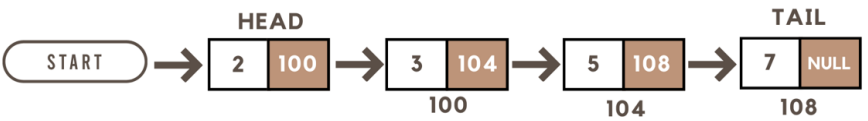
Gambar 2.1: Peta Konsep Pembelajaran

A. DASAR-DASAR LINKED LIST

Linked list adalah struktur data linier yang terdiri dari simpul-simpul yang saling terhubung melalui pointer. Setiap simpul menyimpan data dan alamat (pointer) ke simpul berikutnya dalam urutan. Terdapat beberapa jenis linked list, antara lain: *single linked list*, *double linked list*, dan *circular linked list* (Trivusi, 2022).

1. **Singly linked list**

   Singly linked list adalah linked list unidirectional. Jadi, kita hanya dapat melintasinya dalam satu arah, yaitu dari simpul kepala ke simpul ekor.



Gambar 2.2: Singly Linked List

2. **Doubly linked list**

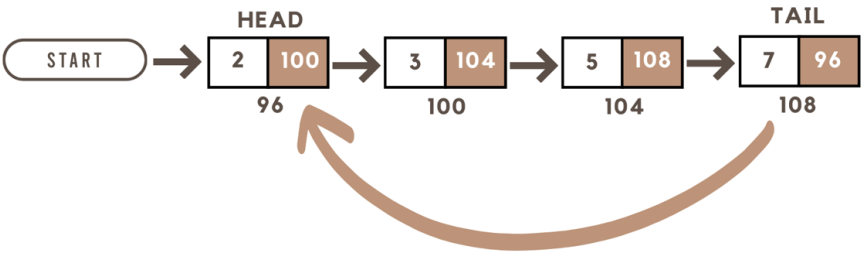
   Doubly linked list adalah linked list bidirectional. Jadi, kita bisa melintasinya secara dua arah. Tidak seperti singly linked list, simpul doubly linked list berisi satu pointer tambahan yang disebut previous pointer. Pointer ini menunjuk ke simpul sebelumnya.



Gambar 2.3: Doubly Linked List

3. **Circular linked list**

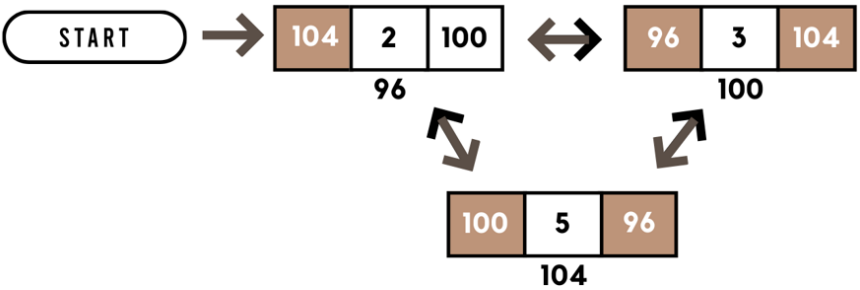
   Circular linked list adalah linked list unidirectional. Kita hanya dapat melintasinya dalam satu arah. Tetapi jenis linked list ini memiliki simpul terakhir yang menunjuk ke simpul kepala. Jadi saat melintas, kita harus berhati-hati dan berhenti saat mengunjungi kembali simpul kepala.



Gambar 2.4: Circular Linked List

4. Circular doubly linked list

Circular doubly linked list adalah gabungan dari Doubly linked list dan Circular linked list. Seperti Doubly linked list, linked list ini memiliki pointer tambahan yang disebut previous pointer, dan mirip dengan Circular linked list, simpul terakhirnya menunjuk pada simpul kepala. Jenis linked list ini adalah bidirectional. Jadi, kita bisa melintasinya dua arah.



Gambar 2.5: Circular Doubly Linked List

Setiap jenis memiliki karakteristik dan kegunaan masing-masing. Misalnya, single linked list hanya memiliki satu variabel pointer dan digunakan untuk menyimpan data secara dinamis (Ulum, 2018). Linked list umumnya dapat digunakan untuk mengimplementasikan struktur data lain seperti stack, queue, ataupun graf. Kelebihan dari tipe linked list antara lain ukuran yang lebih dinamis dan penambahan atau pengurangan data yang lebih cepat, namun memiliki kekurangan seperti menghabiskan memori yang lebih

besar dan tidak dapat langsung mengakses data dengan indeks (Penulis idwebhost, 2022).

Linked list memiliki kelebihan dan kekurangan sebagai berikut:

Kelebihan:
1. Penambahan elemen tidak terbatas: Linked list memungkinkan penambahan elemen tanpa perlu mengalokasikan memori secara berurutan, sehingga memungkinkan untuk diubah secara dinamis.
2. Penghapusan elemen lebih cepat: Operasi penghapusan elemen pada linked list dapat dilakukan dengan mudah dan relatif cepat.

Kekurangan:
1. Hanya bisa diakses secara sekuensial: Linked list memerlukan traversal dari awal hingga elemen yang diinginkan, sehingga akses data secara acak tidak dapat dilakukan secara efisien.
2. Memerlukan memori dalam jumlah yang besar: Linked list memerlukan lebih banyak memori dibandingkan dengan array karena setiap elemen harus menyimpan alamat/referensi dari elemen berikutnya.

Dengan demikian, meskipun linked list memiliki kelebihan dalam penambahan dan penghapusan elemen, namun memiliki keterbatasan dalam akses data secara sekuensial dan penggunaan memori yang lebih besar dibandingkan dengan array (Sigit, 2021).

B. OPERASI DASAR LINKED LIST

Operasi dasar pada linked list meliputi:
- Penciptaan & Penghancuran simpul: Membuat dan menghapus elemen-elemen linked list.
- Inisialisasi linked list: Menyiapkan linked list untuk digunakan.
- Pemeriksaan list kosong: Memeriksa apakah linked list kosong.

Selain itu, terdapat operasi-operasi lain seperti traversal (mengakses setiap elemen), insertion (menambahkan elemen baru), deletion (menghapus elemen), dan searching (mencari elemen tertentu). Operasi dasar ini memungkinkan manipulasi data dalam linked list, seperti penambahan, penghapusan, dan akses terhadap elemen-elemen di dalamnya.

1. **Operasi Penciptaan Simpul pada Linked List**
   Operasi penciptaan simpul pada linked list melibatkan pembuatan simpul pertama dari linked list, yang disebut sebagai \textit{head} atau \textit{node} pertama. Langkah-langkahnya meliputi memberikan nilai awal terhadap variabel pointer dan inisialisasi linked list. Jika kondisi linked list masih kosong, maka simpul baru akan diciptakan. Langkah-langkah ini merupakan tahap awal dalam membangun linked list sebelum melakukan operasi-operasi lain seperti penyisipan, penghapusan, dan penelusuran.

2. **Operasi Penghancuran Simpul pada Linked List**
   Untuk menghancurkan simpul pada linked list, langkah-langkahnya meliputi:
   - Mencari simpul yang akan dihapus.
   - Mengubah pointer simpul sebelum simpul yang akan dihapus agar menunjuk ke simpul setelah simpul yang akan dihapus.
   - Menghapus simpul yang ditunjuk.

   Dalam melakukan penghapusan simpul, perlu diperhatikan bahwa simpul yang dihapus harus dihapus dari memori agar tidak menimbulkan memory leak. Oleh karena itu, setelah simpul dihapus, memori yang digunakan oleh simpul tersebut harus dikembalikan ke sistem operasi.

3. **Inisialisasi linked list**
   Untuk melakukan inisialisasi linked list, langkah-langkahnya meliputi:
   - Membuat variabel pointer yang menunjuk ke simpul pertama (head) dengan nilai null.

- Menentukan tipe data yang akan disimpan dalam linked list.
- Menentukan jumlah elemen awal dalam linked list.
- Mengisi linked list dengan elemen-elemen awal jika ada.

Dalam melakukan inisialisasi linked list, perlu diperhatikan bahwa variabel pointer harus diatur dengan nilai null agar tidak menunjuk ke alamat memori yang tidak valid. Selain itu, tipe data dan jumlah elemen awal dalam linked list juga harus ditentukan terlebih dahulu sebelum melakukan inisialisasi.

Setelah melakukan inisialisasi linked list, beberapa operasi yang harus dilakukan meliputi:

- Pemeriksaan list kosong: Memeriksa apakah linked list kosong atau tidak. Jika list kosong, maka dapat melakukan inisialisasi dengan menambahkan elemen pertama ke dalam linked list.
- Penyisipan simpul sebagai simpul pertama setelah simpul tertentu sebagai simpul terakhir: Setelah inisialisasi, perlu menyisipkan simpul pertama setelah simpul tertentu sebagai simpul terakhir.
- Menghapus simpul: Jika ada kondisi di mana linked list memiliki elemen, maka dapat menghapus simpul dengan mengubah pointer sebelum simpul yang akan dihapus agar menunjuk ke simpul setelah simpul yang akan dihapus, dan kemudian menghapus simpul yang ditunjuk.
- Penelusuran linked list: Melakukan traversal (mengakses setiap elemen) pada linked list untuk memastikan bahwa semua elemen telah di-insert dengan benar.

Dengan melakukan operasi-operasi ini setelah inisialisasi linked list, Anda dapat memastikan bahwa linked list telah di-inisialisasi dengan benar dan siap untuk melakukan operasi-operasi lain seperti penyisipan, penghapusan, dan penelusuran.

4. Pemeriksaan list kosong

Pemeriksaan list kosong pada linked list melibatkan memeriksa variabel pointer head, yang menunjuk ke simpul pertama dari linked list. Jika variabel head bernilai null, maka linked list dikatakan sebagai list kosong. Berikut adalah langkah-langkah untuk melakukan pemeriksaan list kosong:

- Inisialisasi linked list.
- Memeriksa nilai variabel head.
- Jika variabel head bernilai null, maka linked list dikatakan sebagai list kosong.

Setelah melakukan pemeriksaan list kosong, Anda dapat mengevaluasi apakah linked list telah di-inisialisasi dengan benar dan siap untuk melakukan operasi-operasi lain seperti penyisipan, penghapusan, dan penelusuran.

C. PENCARIAN DAN AKSES ELEMEN

Untuk melakukan pencarian dan akses elemen dalam linked list, Anda dapat mengikuti langkah-langkah berikut:

1. Pemeriksaan list kosong: Cocokkan variabel head dengan nilai null, jika list kosong.
2. Mengakses elemen: Traversal melalui linked list untuk mencari elemen yang ingin diakses. Traversal melibatkan penggunaan pointer untuk mengakses elemen berikutnya.
3. Akses elemen: Setelah menemukan elemen yang ingin diakses, Anda dapat mengakses nilai data dan pointer dari elemen tersebut.

Dalam melakukan pencarian dan akses elemen dalam linked list, perlu diperhatikan bahwa akses langsung ke elemen dan nilai data elemen memerlukan traversal melalui linked list, yang melibatkan penggunaan pointer untuk mengakses elemen berikutnya.

Beberapa algoritma yang umumnya digunakan untuk pencarian elemen pada linked list antara lain:

- ***Sequential Search***: Pencarian elemen dilakukan secara sekuensial dari awal hingga akhir linked list. Algoritma ini cocok digunakan untuk linked list yang tidak terurut.
- ***Binary Search***: Pencarian elemen dilakukan dengan membagi linked list menjadi dua bagian dan mencari elemen pada bagian yang sesuai. Algoritma ini cocok digunakan untuk linked list yang terurut.
- ***Hashing***: Pencarian elemen dilakukan dengan menggunakan fungsi hash untuk menghasilkan indeks yang menunjukkan posisi elemen dalam linked list. Algoritma ini cocok digunakan untuk linked list yang besar.

Dalam melakukan pencarian elemen pada linked list, perlu dipertimbangkan jenis linked list yang digunakan dan apakah linked list tersebut terurut atau tidak. Algoritma pencarian yang tepat dapat membantu meningkatkan efisiensi dan kecepatan pencarian elemen pada linked list.

Untuk mengakses elemen pada linked list berdasarkan indeks, Anda dapat mengikuti langkah-langkah berikut:

- Pemeriksaan list kosong: Cocokkan variabel head dengan nilai null, jika list kosong.
- Menghitung panjang linked list: Menghitung jumlah elemen dalam linked list dengan menggunakan variabel tail atau dengan traversal melalui linked list.
- Menghitung indeks yang diinginkan: Mengalikan panjang linked list dengan indeks yang diinginkan untuk menghitung indeks yang akurat.
- Mengakses elemen berdasarkan indeks: Traversal melalui linked list dengan menggunakan indeks yang diinginkan untuk menemukan elemen yang sesuai dengan indeks yang diinginkan.

Namun, perlu diperhatikan bahwa akses elemen berdasarkan indeks tidak dapat dilakukan secara langsung seperti pada array, karena memerlukan traversal melalui linked list dari awal hingga elemen yang diinginkan. Oleh karena itu, untuk mengakses elemen berdasarkan indeks, perlu dilakukan traversal dan penelusuran melalui linked list untuk menemukan elemen yang sesuai dengan indeks yang diinginkan.

Untuk mengakses elemen pada linked list secara sequential, Anda perlu melakukan traversal melalui linked list dari awal hingga elemen yang diinginkan. Hal ini karena linked list tidak dapat diakses secara langsung seperti pada array, sehingga memerlukan penelusuran satu per satu untuk mencapai elemen yang diinginkan. Anda dapat menggunakan pointer untuk mengakses elemen berikutnya dalam linked list. Namun, perlu diperhatikan bahwa akses elemen pada linked list secara sequential memerlukan waktu yang lebih lama dibandingkan dengan akses elemen pada array.

D. MANIPULASI LINKED LIST

Untuk melakukan manipulasi pada linked list, Anda dapat mengakses elemen secara sequential dengan melakukan traversal melalui linked list dari awal hingga elemen yang diinginkan. Selain itu, Anda juga dapat menggunakan List Iterator (ListItr) untuk mengakses elemen secara lintas-lintas dalam linked list. Untuk melakukan manipulasi pada linked list, Anda dapat menggunakan beberapa operasi seperti penyisipan, penghapusan, dan penelusuran. Selain itu, perlu diperhatikan bahwa manipulasi pada linked list tidak dapat dilakukan langsung ke node yang dituju, melainkan harus melalui node pertama dalam linked list.

E. APLIKASI LINKED LIST DALAM PEMROGRAMAN

Linked list adalah struktur data yang digunakan dalam pemrograman untuk menyimpan data dan algoritma dalam bentuk yang dinamis dan efisien. Berikut adalah beberapa aplikasi linked list dalam pemrograman:

1. Implementasi Stack dengan Linked List

Anda dapat mengimplementasikan stack dengan menggunakan linked list. Berikut adalah cara mengimplementasikan stack dengan linked list:

1) Kelas Node: Buat kelas Node untuk menciptakan node linked list. Kelas ini akan berisi data dan tautan ke node berikutnya dalam list.

```
python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

2) Kelas Stack: Buat kelas Stack yang akan mengatur operasi stack. Kelas ini akan memiliki atribut head yang menunjukkan node pertama dalam stack.

```
python
class Stack:
    def __init__(self):
        self.head = None
    def is_empty(self):
        if self.head is None:
            return True
        else:
            return False
    def push(self, data):
        if self.head is None:
            self.head = Node(data)
        else:
            current = self.head
            while current.next is not None:
                current = current.next
            new_node = Node(data)
            new_node.next = current.next
            current.next = new_node
    def pop(self):
        if self.head is None:
            print("Stack tidak tidak kosong, tidak ada yang dapat dihapus")
        else:
            data = self.head.data
            self.head = self.head.next
            return data
    def display(self):
        current = self.head
        while current is not None:
            print(current.data, end=" -> ")
```

3) Operasi Stack: Dengan kelas Stack yang telah dibuat, Anda dapat menambahkan operasi stack seperti push, pop, dan

display.

- “push”: Menambahkan data ke stack dengan menciptakan baru node dan menambahkannya ke bagian awal stack.
- “pop”: Mengeluarkan data dari stack dengan mengaitalah node pertama dan mengembalikan data ke dalam stack.
- “display”: Menampilkan semua data dalam stack.

Dengan menggunakan kelas “Stack” yang dijelaskan di atas, Anda dapat mengimplementasikan stack dengan linked list.

2. Implementasi Queue dengan Linked List

Berikut adalah langkah-langkah untuk mengimplementasikan queue dengan linked list:

1) Deklarasi Struktur Node: Buat struktur node yang berisi data dan pointer ke node berikutnya.

```c
struct Node {
   int data;
   struct Node* next;
};
```

2) Deklarasi Queue: Buat struktur untuk merepresentasikan queue yang menggunakan linked list. Queue ini harus memiliki pointer ke node pertama (front) dan terakhir (rear) dalam linked list.

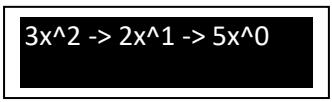
```c
struct Queue {
   struct Node *front, *rear;
};
```

3) Inisialisasi Queue: Inisialisasikan front dan rear queue menjadi NULL saat queue kosong.
4) Operasi Enqueue (Penambahan): Untuk menambahkan elemen ke queue, buat node baru dan atur rear untuk menunjuk ke node baru. Jika queue kosong, atur front juga menunjuk ke node baru.
5) Operasi Dequeue (Penghapusan): Untuk menghapus elemen dari queue, atur front untuk menunjuk ke node berikutnya dari node yang akan dihapus. Jika setelah penghapusan tidak ada elemen dalam queue, atur rear menjadi NULL.
6) Operasi Display: Buat fungsi untuk menampilkan elemen-elemen dalam queue.

Dengan mengikuti langkah-langkah di atas, Anda dapat mengimplementasikan queue menggunakan linked list dalam bahasa pemrograman C.

3. Implementasi Queue dengan Linked List

Representasi polinom dengan menggunakan linked list dapat dilakukan dengan menyimpan koefisien dan pangkat setiap term polinom sebagai data dalam simpul-simpul linked list. Setiap simpul akan merepresentasikan sebuah term dalam polinom. Sebagai contoh, untuk polinom (3x^2 + 2x + 5), kita dapat menggunakan linked list dengan struktur sebagai berikut:



Setiap simpul akan menyimpan koefisien dan pangkat dari term polinom, serta merujuk ke simpul berikutnya dalam linked list.

Dengan representasi ini, operasi-operasi seperti penambahan, pengurangan, dan perkalian polinom dapat diimplementasikan dengan menggunakan linked list. Selain itu, representasi polinom dengan linked list juga memungkinkan untuk menyimpan polinom dengan jumlah term yang dinamis tanpa perlu alokasi memori statis.

Sumber yang disediakan tidak secara khusus menjelaskan implementasi representasi polinom dengan linked list. Namun, dengan menggunakan pendekatan yang disebutkan di atas, representasi polinom dengan linked list dapat diimplementasikan dalam berbagai bahasa pemrograman.

F. ANALISIS KINERJA LINKED LIST

Analisis kinerja linked list melibatkan beberapa aspek penting, seperti waktu, memori, dan efisiensi. Berikut adalah beberapa poin penting yang perlu diperhatikan dalam analisis kinerja linked list:

1. **Waktu**: Waktu yang diperlukan untuk melakukan berbagai operasi pada linked list, seperti penambahan dan penghapusan elemen, pengecekan elemen, dan lainnya. Waktu yang diperlukan untuk melakukan operasi ini dipengaruhi oleh jumlah elemen dalam linked list dan struktur yang digunakan.
2. **Memori**: Memori yang diperlukan untuk menyimpan data dan alamat elemen dalam linked list. Memori yang diperlukan juga untuk menyimpan alamat elemen yang berbeda pada setiap elemen.
3. **Efisiensi**: Efisiensi linked list diperlukan untuk mengakses elemen di berbagai posisi dalam list. Dalam kasus ketiga, linked list memerlukan lebih banyak memori dan waktu daripada data yang terurut, tetapi lebih efisien daripada data yang acak.

Beberapa alat dan metode yang dapat digunakan untuk menganalisis kinerja linked list meliputi:

- **Benchmarking**: Metode ini melibatkan pengukuran waktu yang diperlukan untuk melakukan berbagai operasi pada linked list berberapa kali dan menyimpannya. Kemudian, waktu rata-rata dan waktu deviasi dapat digunakan untuk menganalisis kinerja linked list.
- **Pemantauan Memori**: Pemantauan memori dapat digunakan untuk menganalisis kinerja linked list dengan mengevaluasi

perubahan memori yang terjadi selama operasi pada linked list, seperti penambahan dan penghapusan elemen.

- **Analisis Waktu**: Analisis waktu dapat dilakukan dengan menggunakan alat seperti Python profiler untuk mengukur waktu yang diperlukan untuk melakukan berbagai operasi pada linked list.

Dengan memahami cara kerja dan analisis kinerja linked list, Anda dapat membuat desain yang efisien dan memilih metode yang tepat untuk mengimplementasikan dan menggunakan linked list dalam berbagai aplikasi pemrograman.

G. RANGKUMAN

Berdasarkan uraian tersebut diatas struktur data linked list terdiri dari sejumlah simpul yang saling terhubung melalui pointer. Setiap simpul menyimpan data dan alamat dari simpul berikutnya. Linked list memiliki beberapa jenis, seperti single linked list, double linked list, dan circular linked list. Operasi dasar linked list yang umum dilakukan penciptaan dan penghancuran simpul berdasarkan data yang diberikan, membuat linked list baru dengan hanya satu elemen kunci, pemeriksaan list kosong dan melakukan operasi lainnya seperti membangun linked list, penelusuran linked list, penyisipan simpul sebagai simpul pertama setelah simpul tertentu sebagai simpul terakhir, dan menghapus simpul. Linked list dapat diimplementasikan pada berbagai aplikasi pemrograman, terutama ketika jumlah data tidak pasti, sebagai contoh mengelola data pelanggan, transaksi, dan produk dalam aplikasi berbasis platform. Dengan memahami struktur data linked list, Anda dapat menggunakan linked list dengan efisien dan efektif dalam berbagai proyek pemrograman.

H. TES FORMATIF

1. Apa yang menjadi elemen dasar dalam struktur data linked list?
   a. Pointer
   b. Array
   c. String
   d. Integer

2. Jenis linked list yang memiliki dua pointer, satu ke node sebelumnya dan satu ke node berikutnya, disebut:
   a. Single Linked List
   b. Double Linked List
   c. Circular Linked List
   d. Linear Linked List

3. Apa keuntungan utama penggunaan linked list dalam pemrograman?
   a. Pengelolaan memori yang efisien
   b. Pencarian cepat
   c. Kecepatan akses yang tinggi
   d. Struktur yang sederhana

4. Operasi apa yang digunakan untuk menambahkan node di tengah-tengah linked list?
   a. addFirst()
   b. addLast()
   c. addMiddle()
   d. insertAfter()

5. Apa kekurangan utama dari penggunaan linked list?
   a. Kecepatan akses tinggi
   b. Pemakaian memori yang lebih rendah
   c. Kesulitan dalam pengurutan
   d. Overhead memori

6. Manakah pernyataan berikut yang benar tentang Circular Linked List?
   a. Tidak memiliki awal atau akhir yang jelas
   b. Setiap node memiliki dua pointer
   c. Hanya dapat diakses secara sequential
   d. Tidak dapat digunakan untuk mewakili grafik

7. Apa yang dilakukan operasi reverse pada linked list?
   a. Mengurutkan elemen secara ascending
   b. Membalik urutan node
   c. Menghapus elemen di awal linked list
   d. Menyisipkan node baru di akhir linked list

8. Bagaimana cara mengakses elemen pada linked list berdasarkan indeks?
   a. Menggunakan pointer
   b. Secara sequential
   c. Dengan mencari nilai elemen
   d. Tidak dapat diakses berdasarkan indeks

9. Apa fungsi dari pointer NULL pada linked list?
   a. Menandakan akhir linked list
   b. Menandakan awal linked list
   c. Menunjuk ke node pertama
   d. Menunjuk ke node terakhir

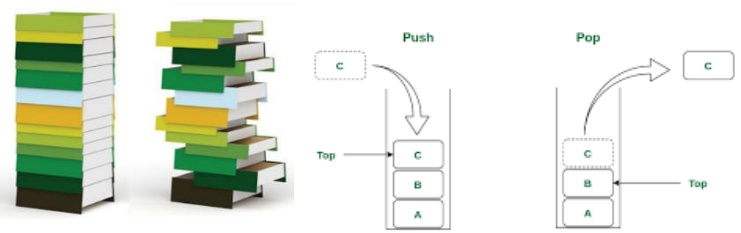
10. Apa kegunaan linked list dalam implementasi struktur data stack dan queue?
    a. Menyimpan data dengan urutan tertentu
    b. Memudahkan operasi pencarian
    c. Mewakili data dalam format tabel
    d. Mendukung operasi push dan pop.

I. LATIHAN

1. Buatlah sebuah program dalam bahasa pemrograman pilihan Anda untuk menginisialisasi linked list kosong.
2. Implementasikan fungsi untuk menambahkan elemen baru di akhir linked list.
3. Buatlah fungsi untuk menghapus elemen pertama dari linked list.

KEGIATAN BELAJAR 3

STACK (TUMPUKAN) & QUEUE (ANTRIAN)



DESKRIPSI

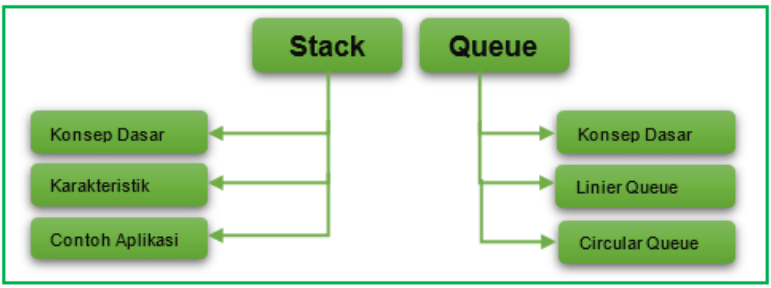
Pada bab ini mahasiswa mempelajari konsep dasar algoritma *stack* dan *queue* sehingga mampu implementasi studi kasus pada salah satu bahasa pemrograman menggunakan operasinya.

KOMPETENSI PEMBELAJARAN

Mahasiswi memiliki pengetahuan dan kemampuan dalam

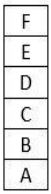
1. Menjelaskan konsep dasar *stack dan queue* seperti pendeklarasian dan berbagai jenis operasinya
2. Mampu mengimplementasikan *stack dan queue* pada berbagai kasus dengan salah satu bahasa pemrograman.

PETA KONSEP PEMBELAJARAN



A. KONSEP DASAR STACK

Stack adalah kumpulan elemen yang hanya dapat ditambah dan atau dihapus dari satu ujung (gerbang) yang sama, seolah-olah bertumpuk. *Stack* mempunyai sifat LIFO (*Last In First Out*) yang berarti elemen terakhir masuk akan pertama keluar yang direpresentasikan menggunakan Linked List dan array yang disajikan Single Stack atau Double Stack (Suyanto, 2021).
Misalkan sebuah kotak ditumpukkan, seperti pada Gambar 1. Kotak A diletakkan pertama sekali kemudian diikuti secara berturut-turut kotak B, kotak C, kotak D, kotak E, dan terakhir kotak F. Maka untuk mengambil tiap kotak dari tumpukan harus dilakukan secara berturut-turut mulai dari kotak F, kotak E, kotak D, kotak C, kotak B, kemudian kotak A. Karena jika kita membutuhkan kotak C misalnya dan langsung mengambil kotak C tanpa terlebih dahulu mengambil kotak di atasnya maka tumpukan akan tumbang.



Gambar 3.1 Tumpukan 6 kotak

1. Deklarasi Stack

Stack dideklarasikan dengan dua cara, bentuk deklarasi pertama:

```
#define MaxS n
TypeData Isi(MaxS);
TypeData Top
```

Bentuk deklarasi kedua:

```
#define MaxS n
struct Stack
{
    TypeData Isi(MaxS);
    TypeData Top;
```

```
};
```

2. Operasi pada Stack

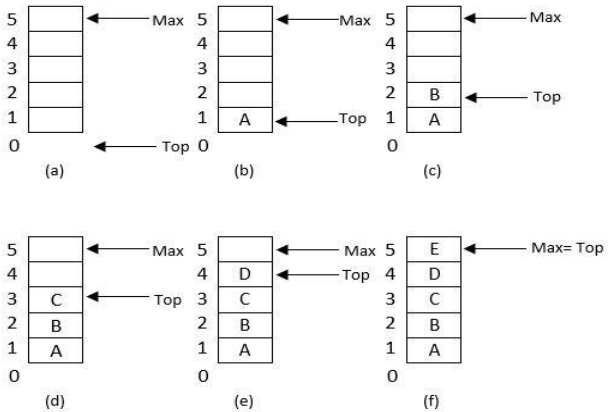
Operasi dasar yang dapat dilakukan terhadap sebuah Stack, yaitu operasi Insert atau penyisipan (PUSH) dan operasi Delete atau penghapusan elemen (POP) (Yuliyanti, 2023).

1. Inisialisasi Stack

   Sebelum Stack dapat dioperasikan terlebih dahulu diinisialisasikan dengan memberi harga S.Top = 0 (Adib, 2021).

2. Operasi PUSH

   Operasi PUSH memasukkan elemen ke dalam suatu Stack, jika Stack belum mencapai Max. Misalkan diketahui suatu Stack S dengan maksimum 5 elemen. Andaikan elemen-elemen A, B, C, D, dan E dimasukkan (PUSH) berurutan maka elemen A akan menempati posisi pertama (1), elemen B pada posisi ke-2, elemen C pada posisi ke-3, elemen D pada posisi ke-4, dan elemen E pada posisi ke-5, dapat dilihat pada Gambar 3.2.

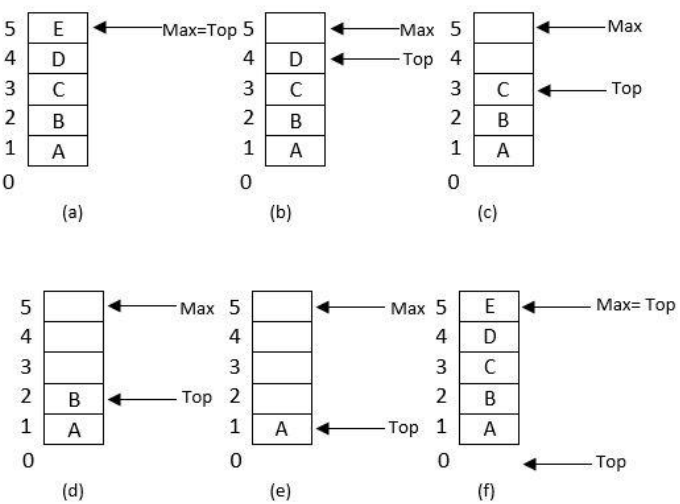


Gambar 3.2. Operasi penyisipan elemen ke dalam stack

Perhatikan petunjuk ***Max*** tidak pernah bergerak, yang bergerak ***Top***, ***Top*** bertambah satu jika ada elemen masuk (***PUSH***) ke dalam Stack hingga Top = Max. Gambar a merupakan suatu Stack ***kosong***, dengan Top=0 dan Max=5. Gambar b penyisipan elemen A sehingga Top=1 dan Max=5. Gambar c penyisipan elemen B sehingga Top=2 dan Max=5. Gambar d penyisipan elemen C sehingga Top=3 dan Max=5. Gambar e penyisipan elemen D sehingga Top=4 dan Max=5. Gambar f penyisipan elemen E sehingga Top=5 dan Max=5 yang menyatakan Stack **Penuh**.

3. Operasi POP

   Operasi mengambil atau menghapus elemen paling puncak dari Stack. Setiak kali operasi POP dilakukan maka petunjuk ***Top*** akan berkurang satu. Operasi POP dilakukan jika Stack tidak kosong. Misalkan kita punya Stack seperti pada Gambar 2. (f), Ilustrsi jika penghapusan elemen dapat dilihat pada Gambar 3.



Gambar 3.3. Operasi Penghapusan Elemen dari Stack

Gambar a merupakan suatu Stack **Penuh**, dengan Top=Max. Gambar b penghapusan elemen E sehingga Top=4 dan Max=5. Gambar c penghapusan elemen D sehingga Top=3 dan Max=5. Gambar d penghapusan elemen C sehingga Top=2 dan Max=5. Gambar e penghapusan elemen B sehingga Top=1 dan Max=5. Gambar f penghapusan elemen A sehingga Top=0 dan Max=5 menyatakan Stack **Kosong**.

4. Operasi FULL
   Digunakan mengecek Stack penuh atau tidak, jika penuh maka S.Top bernilai MaxS (S.Top=MaxS). Jika S.Top=MaxS maka fungsi mengembalikan nilai ***true***, jika tidak maka fungsi mengembalikan nilai ***false***.
5. Operasi Empty
   Operasi digunakan untuk mengecek apakah Stack kosong atau tidak. Suatu Stack dikatakan kosong jika S.Top bernilai nol (S.Top=0). Maka jika S.Top=0 maka fungsi akan mengembalikan nilai ***true***, jika tidak fungsi mengembalikan nilai ***false***.
6. Operasi Clear
   Operasi Clear adalah operasi yang akan digunakan untuk mengosongkan Stack. Suatu Stack dikatakan kosong jika Top=0.

B. KARAKTERISTIK STACK

Dari operasi yang dilakukan diperoleh bahwa karakteristik dari suatu Stack adalah (S, 2021):

1. **Elemen Stack** yaitu item-item data yang terdapat dalam Stack.
2. **Top** menunjukkan posisi puncak pada Stack.
3. **Max** menunjukkan banyaknya maksimum item dari Stack.
4. **Stack Kosong** tidak mungkin dilakukan POP karena akan menyebabkan
5. **Stack Penuh** tidak mungkin dilakukan PUSH karena akan menyebabkan

C. APLIKASI STACK

Stack dapat diaplikasikan untuk simulasi stack dalam dunia nyata, pemanggilan fungsi/procedure, rekursif, penanganan interupsi, evaluasi ekspresi, konversi notasi infiks ke notasi postfiks dan konversi bilangan basis 10 (desimal) ke basis 2 (biner) (Cecep Kurnia Sastradipraja, 2022).

Contoh Syntax program stack berserta penjelasnya:

```
//header file
#include <iostream>
#include <conio.h>
#define maxstack 5
using namespace std; //untuk melegalkan header
iostream

//pendeklarasian struct
struct STACK
{
      int top;
      float data[4];
};
float dta;

/*struct yang telah dibuat (STACK) dijadikan
suatu Tipe data, dimana disebut tipe data
abstrak*/
struct STACK stackbaru;

//fungsi boolean untuk mengetahui apakah stack
penuh
bool isfull()
{
      if(stackbaru.top == maxstack)
            return true;
      else
            return false;
}
```

```
//fungsi boolean untuk mengetahui apakah stack
kosong
bool isempty()
{
      if(stackbaru.top == -1)
            return true;
      else
            return false;
}

//fungsi untuk menambahkan data pada stack
void push(float dta)
{
     if(isfull() == true) /*panggil fungsi
isempty(), jika kondisi benar pernyataan
dijalankan*/
      {
            puts("Maaf, stack penuh");
      }
      else{
            stackbaru.top++;
            stackbaru.data[stackbaru.top]=dta;
      }
}

//fungsi untuk mengambil data pada stack
void pop()
{
      if(isempty() == true) //panggil fungsi
isempty(), jika kondisi benar pernyataan
dijalankan
      {
            cout<<"Data telah kosong!";
      }
      else
      {
            cout<<"Data yang terambil : "
<<stackbaru.data[stackbaru.top]<<endl;
            stackbaru.top--;
      }
```

```
}

//fungsi untuk mencetak data pada stack
void print()
{
      printf("\nData yang terdapat dalam stack :
\n");
      printf("--------------------------------
\n");
      for(int i=0; i<=stackbaru.top; i++)
      {
            cout<<stackbaru.data[i]<<"    ";
      }
}

//fungsi untuk membersihkan data dalam stack
void clear()
{
      stackbaru.top = -1;
      printf("\nSekarang stack kosong");
}

//fungsi utama
int main()
{
      stackbaru.top = -1;
      //pendeklarasian variabel
      char menu;
      char ulang;
      //perulangan dengan do-while
      do
      {
            system("cls");
            printf("\t PROGRAM STACK\n");
            printf("\t===============\n");
            printf("Menu : ");
            puts("\n1. Pop stack");
            puts("2. Push stack");
            puts("3. Cetak");
            puts("4. Bersihkan stack");
```

```
            puts("5. Exit");

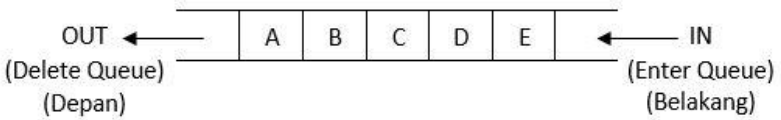
            cout<<"Menu pilihan Anda : ";
            cin>>menu;

            if(menu == '1')
            {
                  pop(); //panggil fungsi pop()
                  ulang = 'y';
                  getch();
            }
            else if(menu == '2')
            {
                  cout<<"\nTambah Data";
                  cout<<"\n-----------";
                  cout<<"\nData yang akan
disimpan di stack : ";
                  cin>>dta;
                 push(dta); /*panggil fungsi
push(dta)--dta sesuai dengan data ynag diinput*/
                  ulang = 'y';
            }
            else if(menu == '3')
            {
                  print(); /*panggil fungsi untuk
mencetak data dalam stack*/
                  cout<<"\n\nUlang ? (y/t)";
                  cin>>ulang;
            }
            else if(menu == '4')
            {
                  clear(); //panggil fungsi untuk
membersihkan stack
                  cout<<"\n\nUlang ? (y/t)";
                  cin>>ulang;
            }
            else if(menu == '5')
            {
                  exit(0); //keluar dari program
            }
```

```
        }while(ulang == 'Y' || ulang == 'y');
/*akan selalu diulang ketika ulang == 'y' ||
ulang'Y'*/
}
```

D. KONSEP DASAR QUEUE

Queue atau Antrian merupakan kumpulan elemen dengan penyisipan dan penghapusan elemen yang dilakukan dari sisi/gerbang yang berbeda. Penyisipan dilakukan dari gerbang belakang dan penghapusan dilakukan dari gerbang depan, sehingga Queue mempunyai sifat FIFO (*First In First Out*), yaitu elemen yang pertama masuk akan keluar pertama juga.



Gambar 3.4 Queue dengan 5 Elemen

Gambar 1 menunjukan 5 elemen (A,B,C,D,E) dimasukkan secara berturut-turut dari belakang. Maka Elemen A yang merupakan elemen pertama masuk sehingga menempati posisi paling depan, Elemen E yang merupakan elemen terakhir masuk sehingga elemen E menempati posisi paling belakang.

F. LINIER QUEUE DENGAN ARRAY

1.1. Deklarasi Queue

```
#define MaxQ N
#define true 1
#define false 0
struct Queue
    {
        TypeData Isi[MaxQ+1];
```

```
    TypeData Depan;
    TypeData Belakang;
};
```

1.2. Operasi pada Linear Queue

Sama halnya dengan Stack, operasi suatu Queue diantaranya penyisipan elemen dan penghapusan elemen serta operasi untuk mengecek apakah Queue kosong atau penuh.

a. Operasi Inisialisasi

Operasi ini digunakan untuk menginisialisasikan suatu Queue, yaitu dengan memberi nilai depan=1 dan nilai belakang=0. Dengan kata lain digunakan untuk membuat suatu Queue yang baru.

```
void INITS (Queue &Q)
{
        Q.Depan = 1;
        Q.Belakang = 0;
}
```

b. Operasi Queue Kosong

Digunakan untuk mengecek apakah suatu Queue dalam keadaan kosong atau tidak. Karena pada saat penghapusan elemen Queue, harus dijamin bahwa Queue tidak boleh kosong. Dikatakan suatu Queue kosong adalah jika penunjuk belakang=0.

c. Operasi Queue Penuh

Operasi ini digunakan untuk mengecek apakah suatu Queue dalam keadaan penuh atau tidak, karena saat penyisipan elemen Queue, harus dijamin bahwa Queue tidak penuh. Suatu Queue dikatakan penuh jika penunjuk belakang=MaxQ. Operasi ini dibuat dalam suatu fungsi, sehingga mengembalikan nilai true jika Queue penuh,

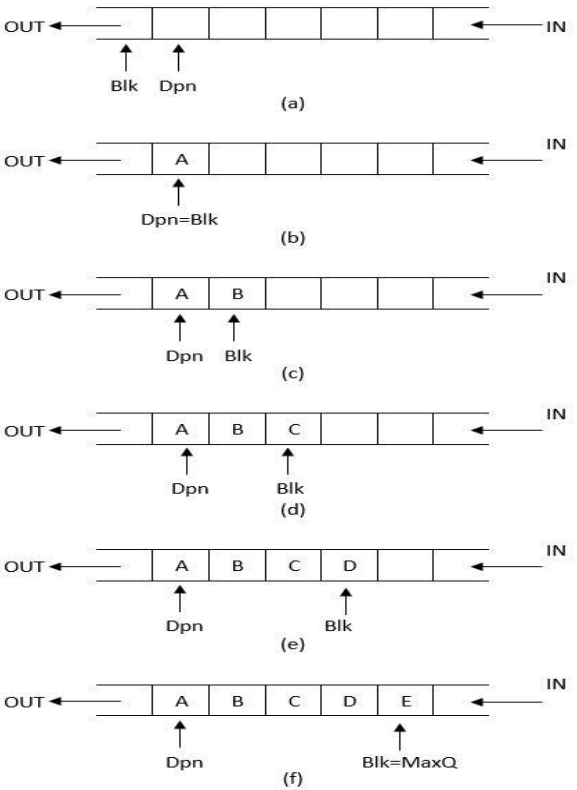
tetapi fungsi mengembalikan nilai false jika Queue tidak penuh.

d. Operasi Mengosongkan Queue

Digunakan untuk membuat Queue dalam keadaan kosong, berbeda dengan operasi inisialisasi Queue, karena dalam operasi inisialisasi dilakukan untuk Queue yang belum pernah ada. Sedangkan pada operasi ini Queue sebelumnya sudah ada dan karena sesuatu hal misalnya Queue akan dikosongkan.

e. Operasi Penyisipan Elemen Queue

Operasi ini untuk menyisipkan elemen ke dalam Queue, selama Queue tidak penuh. Setiap penyisipan elemen dilakukan maka penunjuk belakang akan bertambah satu. Penunjuk tidak boleh bergerak (naik ataupun turun). Untuk jelasnya perhatikan Gambar 2. sebagai ilustrasi penyisipan elemen Queue.



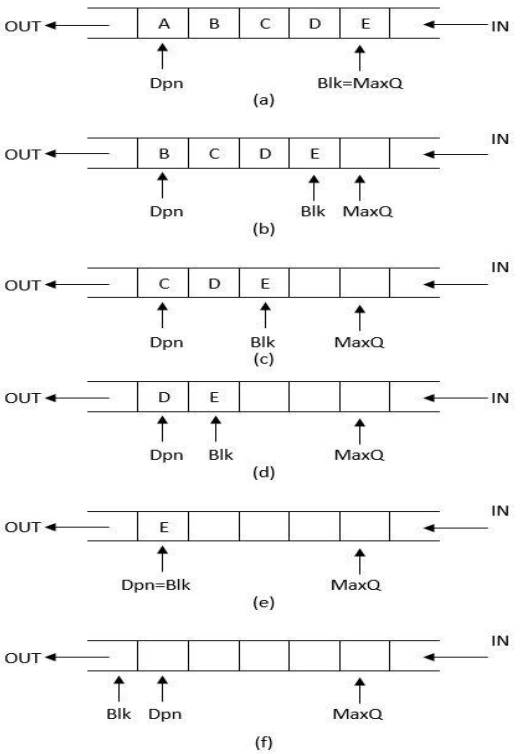
Gambar 3.5. Operasi Penyisipan Elemen Queue

Gambar (a) Queue dalam keadaan kosong, Gambar (b) elemen A disisipkan ke Queue sehingga penunjuk Blk (Belakang) naik satu. Demikian seterusnya hingga elemen Q disisipkan dan penunjuk Blk sama dengan MaxQ.

f. Operasi Penghapusan Elemen Queue

Operasi ini digunakan untuk menghapus elemen dari Queue. Elemen dalam suatu Queue dapat dihapus selama Queue tidak dalam keadaan kosong. Setiap penghapusan elemen dilakukan maka semua elemen berikutnya harus bergeser satu posisi ke depan kemudian penunjuk belakang

akan berkurang satu. Penunjuk depan tidak boleh bergerak (naik ataupun turun), diilustrasikan pada Gambar 3.



Gambar 3.6 Penghapusan Elemen Queue

g. Operasi Pencetakan Isi Queue

Operasi ini digunakan untuk mencetak elemen-elemen isi dari suatu Queue. Berikut fungsi untuk mencetak elemen Queue.

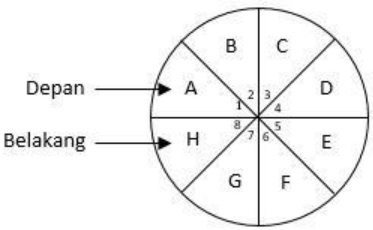
```
void CETAK(Queue Q)
{
        int;
        cout<<endl<<"Isi Queue : ";
```

```
    if(Q.Belakang >= Q.Depan)
    {
        for(i=1;i<=Q.Belakang;i++)
        {
            cout<<Q.Isi[i];
        }
    }
    else
        cout<<"Queue Kosong....";
}
```

F. CIRCULAR QUEUE DENGAN ARRAY

Kelemahan dari Delete Queue adalah setiap melakukan penghapusan harus diikuti dengan pergeseran dari setiap elemen yang ada dalam Queue satu posisi ke depan. Untuk mengatasi kelemahan ini dapat dilakukan dengan Queue Melingkar (*Circular Queue*) (Ahmad Talha Siddiqui, 2024).

Circular Queue adalah suatu Queue yang dibuat seakan-akan merupakan sebuah lingkaran. Misalkan kita mempunyai suatu Queue dengan maksimal 8 elemen yang dibuat secara melingkar seperti pada Gambar 4.



Gambar 3.7 Circular Queue dengan 8 Elemen

1. Deklarasi Circular Queue

   Deklarasi Linier Queue sama dengan deklarasi Circular Queue yaitu Depan=1 dan Belakang=0.

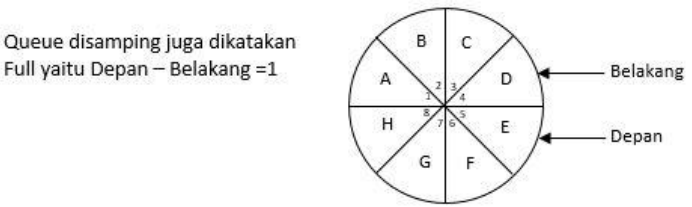
2. Operasi pada Circular Queue

   a. Operasi Inisialisasi

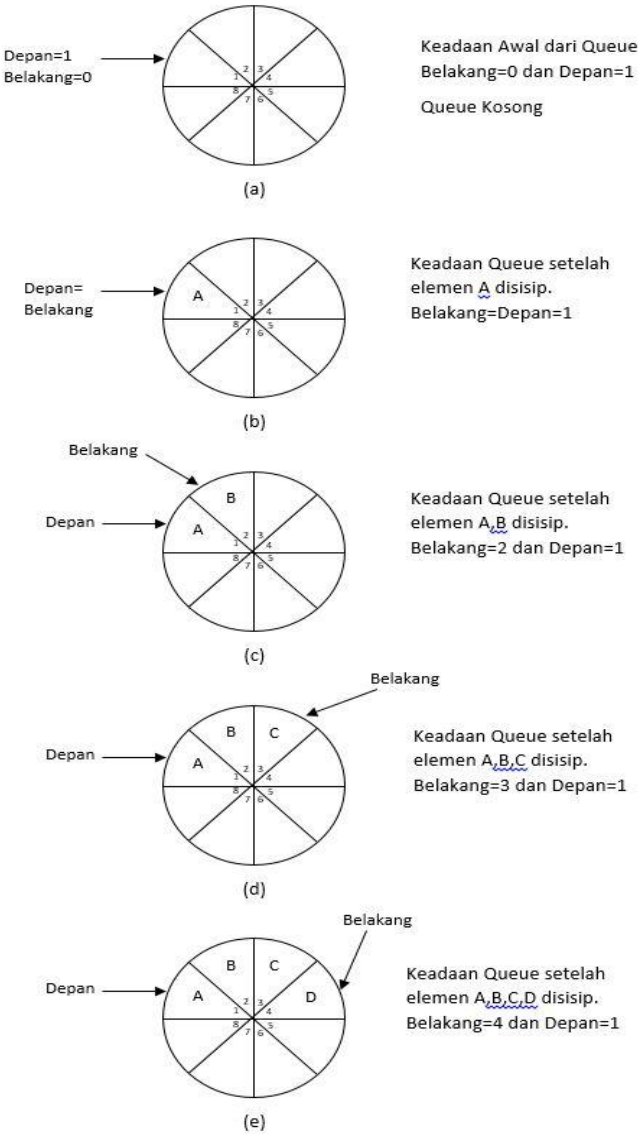
      Operasi ini digunakan untuk menginisialisasikan atau membuat suatu Queue baru, yaitu dengan memberi nilai Depan=1 dan nilai Belakang=0.

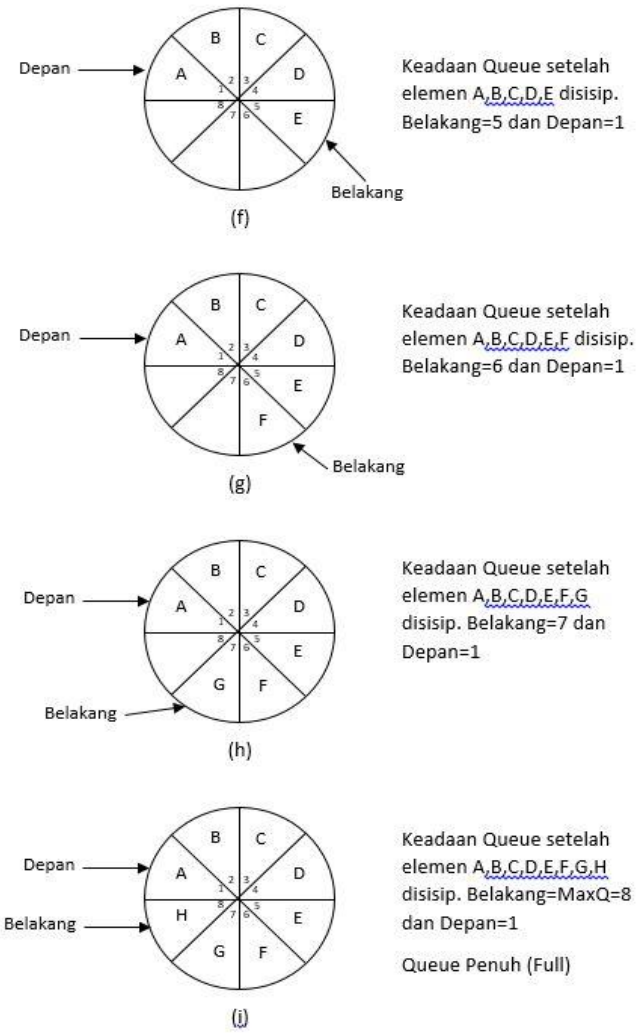
   b. Operasi Penyisipan Elemen Queue

      Operasi ini untuk menyisipkan elemen ke dalam Queue. Setiap kali elemen dimasukkan ke Queue maka penunjuk belakang naik 1 (Belakang+1) jika belakang lebih dari maksimum elemen array. Tetapi jika belakang sama dengan maksimum elemen array maka belakang diset menjadi 1 (Belakang=1). Elemen dapat dimasukkan ke dalam Queue selama Queue belum penuh. Suatu Queue dikatakan penuh jika Depan=1 dan Belakang=Maksimum atau Depan-Belakang=1. Gambar 5. merupakan ilustrasi penyisipan elemen ke dalam Queue dengan maksimum 8 elemen. Dari Gambar 5. bahwa penyisipan elemen, penunjuk belakang selalu bertambah satu jika belakang masih lebih kecil dari maksimum. Queue penuh jika Depan=1 dan Belakang=Maksimum. Suatu Queue juga dikatakan penuh jika Depan – Belakang = 1. Perhatikan Gambar 6.



Gambar 3.8 Queue Penuh


Depan=1
Belakang=0
Keadaan Awal dari Queue
Belakang=0 dan Depan=1
Queue Kosong
(a)
Depan=
Belakang
A
Keadaan Queue setelah
elemen A disisip.
Belakang=Depan=1
(b)
Belakang
Depan
B
A
Keadaan Queue setelah
elemen A,B disisip.
Belakang=2 dan Depan=1
(c)
Belakang
Depan
B
C
A
Keadaan Queue setelah
elemen A,B,C disisip.
Belakang=3 dan Depan=1
(d)
Belakang
Depan
B
C
A
D
Keadaan Queue setelah
elemen A,B,C,D disisip.
Belakang=4 dan Depan=1
(e)



Gambar 3.9 Operasi Penyisipan Elemen Pada Circular Queue

c. Operasi Penghapusan Elemen Queue

Operasi ini digunakan untuk mengambil atau menghapus elemen dari suatu Queue. Operasi penghapusan elemen pada Circular yang dihapus adalah selalu yang ada di depan sehingga penunjuk depan akan naik satu (Depan+1) jika

depan lebih kecil dari maksimum elemen array. Tetapi depan diset menjadi satu (Depan=1) jika depan sama dengan maksimum elemen array. Setiap kali penghapusan elemen maka depan akan naik satu atau diset menjadi 1. Operasi penghapusan dapat dilakukan selama Queue belum kosong.

Dari gambar di atas dapat dilihat bahwa suatu Queue dikatakan:

- Kosong (Empty) adalah jika Depan = 1 dan Belakang = 0 (Depan – Belakang = 1).
- Penuh (Full) adalah jika:
  - Belakang = MaxQ dan Depan=1
  - Depan – Belakang = 1
- Jika Depan = 3 dan Belakang = 6 maka isi Queue adalah {C, D, E, F}.
- Jika Depan = 6 dan Belakang = 3 maka isi Queue adalah {F, G, H, A, B, C}.

G. RANGKUMAN

Queue adalah salah satu struktur data yang digunakan dalam pemrograman untuk menyimpan sekumpulan elemen dengan prinsip “First-In-First-Out” (FIFO), artinya elemen yang pertama masuk ke dalam antrian akan menjadi elemen yang pertama pula keluar dari antrian. Queue memiliki beberapa fungsi penting dalam pemrograman diantaranya :

a. Mengatur Urutan: Menjaga urutan tugas, yang pertama masuk adalah yang pertama keluar. Seperti antrian nyata.
b. Penjadwalan: Membantu mengatur pekerjaan berdasarkan prioritas atau urutan, berguna saat banyak tugas.
c. Buffering: Sementara menyimpan data cepat sebelum diproses lebih lanjut.
d. Algoritma: Berguna dalam algoritma seperti BFS yang menjelajahi langkah demi langkah.

e. Simulasi: Digunakan dalam model antrian di dunia nyata, contohnya pesawat atau antrian belanja.
f. Pengelolaan Memori: Mengontrol penggunaan sumber daya seperti memori.
g. Aplikasi Chat: Membantu mengatur pesan sebelum ditampilkan.
h. Manajemen Proses: Mengatur proses yang akan dieksekusi oleh komputer.
i. Sistem Antrian: Dipakai di bank, rumah sakit, dll., mengatur giliran pelanggan

H. TES FORMATIF

1. Bagaimanakan konsep karakteristik dari sebuah queue ?
   a. FIFO
   b. LIFO
   c. FIFO dan LIFO
   d. tidak ada jawaban benar

2. Jenis queue yang sering diimplementasikan kecuali
   a. Linier queue dengan array
   b. Circular queue dengan array
   c. Jawaban A dan B benar
   d. Jawaban A dan B salah

I. LATIHAN

Buatlah aplikasi antrian di sebuah bank dengan jumlah slot antrian 3 dan masing-masing jumalh antrian dapat diinput secara dinamis !

KEGIATAN BELAJAR 4

STRUKTUR DATA *TREE* DAN BINARY *TREE*

DESKRIPSI PEMBELAJARAN

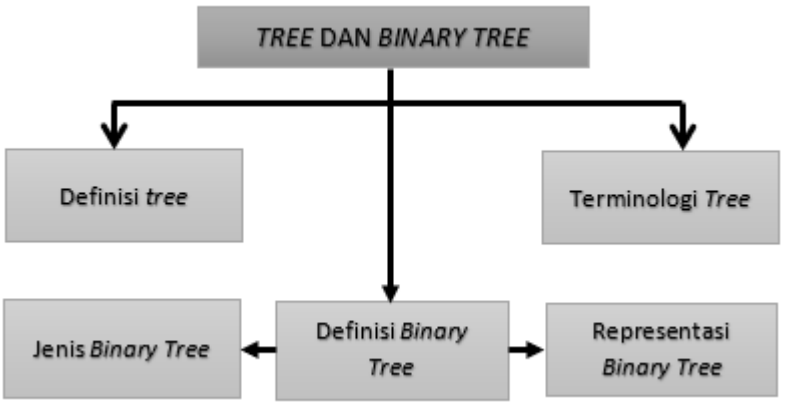
Pada bab ini mahasiswa mempelajari pengantar *tree* dan binary *tree*. Diharapkan mahasiswa memiliki wawasan dan pemahaman yang cukup untuk dapat mengimplementasikan *tree* dan *binary tree* dalam struktur data.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu menjelaskan tentang *tree* dan *binary tree*.
2. Mampu mampu mengidentifikasi terminologi pada sebuah *tree*
3. Mampu mampu membedakan *binary tree* berdasarkan jenisnya.
4. Mampu mengkonversi masalah ke dalam bentuk *binary tree*

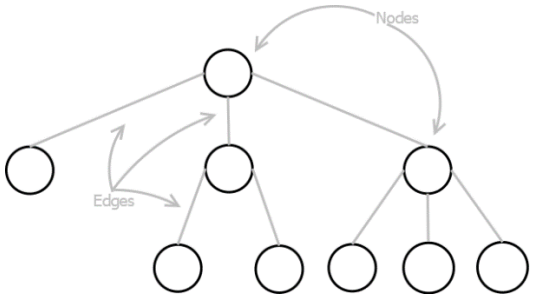
PETA KONSEP PEMBELAJARAN



A. PENGANTAR *TREE*

Pada kegiatan belajar ini akan dibahas tentang hal-hal yang berkaitan dengan *tree* dalam struktur data. *Tree* dalam struktur data adalah representasi *tree* pada kehidupan sehari-hari. Jika dalam kehidupan sehari-hari *tree* atau pohon memiliki daun, ranting, batang dan akar maka *tree* dalam struktur data juga akan memiliki beberapa bagian dari *tree* dalam kehidupan sehari-hari seperti daun dan ranting.

Ilustrasi *tree* dapat dilihat pada gambar 4.1 di bawah ini. Lingkaran putih adalah *node* dalam struktur data yang dalam kehidupan sehari-hari dianggap sebagai daun. Sementara antara *node* yang satu dengan *node* lainnya dihubungkan oleh *edge* yang dalam kehidupan sehari-hari dianggap sebagai ranting.



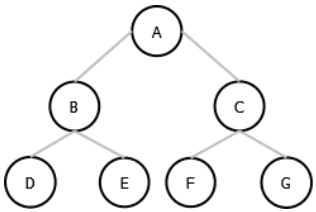
Gambar 4. 1 Ilustrasi tree
Sumber: (Lafore, 2002)

1. Definisi *Tree*

Tree (pohon) merupakan salah satu jenis struktur data yang terdiri dari simpul-simpul (*nodes*) yang saling terhubung (*edges*) satu sama lain secara hierarkis. *Nodes* dalam implementasinya dapat merepresentasikan data, sementara *edges* adalah jalan atau lintasan yang menghubungkan antara *nodes*. Pada dasarnya *tree* dapat digunakan untuk merepresentasikan data yang memiliki hierarkis seperti struktur perusahaan, struktur

keluarga, struktur direktori pada sistem file komputer atau permasalahan lain yang melibatkan pembagian dan percabangan.

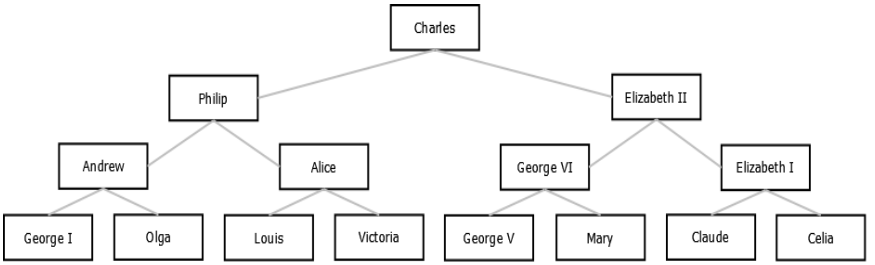
Berikut ini adalah contoh *tree* yang memiliki 7 *node*s dan 6 *edges*.



Gambar 4. 2 Contoh tree sederhana dalam struktur data

Pada contoh 4.2 di atas dapat diketahui bahwa A adalah akar (*root*). Akar A, memiliki dua anak (*child*) yaitu B dan C. B memiliki dua daun (*leaf*) yaitu D dan E. Begitupun dengan F dan G merupakan daun dari C. Dalam kehidupan nyata, objek terakhir dari pohon adalah daun. Oleh karena itu D,E dan F, G disebut sebagai daun pada gambar 4.2 di atas karena tidak lagi memiliki anak seperti B dan C.

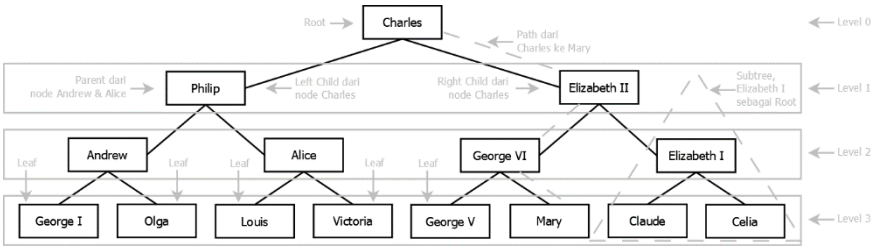
Tree juga dapat merepresentasikan struktur keluarga. Berikut adalah struktur keluarga pangeran Charles yang direpresentasikan dalam bentuk *tree*. Anda juga bisa merepresentasikan keluarga mulai dari kakek hingga sekarang dalam bentuk *tree*.



Gambar 4. 3 Contoh tree dalam keluarga pangeran Charles
Sumber: (Daisma, n.d.)
website: https://daismabali.medium.com/

2. Terminologi *Tree*

Diawal telah diketahui bahwa dalam *tree* terdapat *nodes* dan *edges*. Pada pembahasan gambar 4.2 juga sudah menyinggung istilah *child* dan *leaf*. Untuk memudahkan pemahaman tentang *tree*, berikut dijelaskan beberapa terminologi *tree* dalam struktur data dengan menjadikan gambar 4.4 berikut sebagai representasi terminologi *tree*.



Gambar 4. 4 Representasi terminologi tree keluarga pangeran Charles

1. *Path*

   Path adalah urutan lintasan atau hubungan yang dapat dilalui jika ingin melakukan *tracking* dari *node* yang satu ke *node* yang lain. Pada gambar 4.4, dari Charles ke Mary yang digambarkan dengan garis putus-putus adalah sebuah path dalam *tree*.

2. *Root*

   Dalam *tree* hanya akan terdapat satu *root*. Root ini berada paling puncak atau level 0 (nol). Oleh karena itu, pada gambar 4.4 *node* yang berstatus *tree* adalah Charles.

3. *Parent*

   Parent pada *tree* adalah *node* yang memiliki satu atau lebih *edge* di bagian bawahnya. Pada gambar 4.4 dapat diketahui bahwa *Charles* adalah *parent* dari *node* Philip dan Elizabeth II. *Node* Philip juga adalah *parent* untuk Andrew dan Alice. Begitupun dengan Elizabeth II adalah *parent* bagi George VI dan Elizabeth I.

4. *Child*

   Child adalah *node* dalam *tree* yang hanya memiliki satu *edges* pada bagian atasnya. Jadi *node* tertentu dapat dianggap sebagai *parent* ataupun *child* berdasarkan sudut pandangnya. Contoh, pada gambar 4.4 dapat dilihat bahwa *node* Philip adalah *Left child* dari *node* Charles, tapi *node* Philip adalah *parent* untuk *node* Andrew dan Alice.

5. *Leaf*

   Leaf adalah *node* yang paling akhir dalam *tree*. *Leaf* tidak memiliki *edge* lagi pada bagian bawahnya. Dalam gambar 4.4 *node* yang berstatus sebagai *leaf* adalah George I, Olga, Louis, Victoria, George V, Mary, Claude, dan Celia

6. *Subtree*

   Subtree adalah *node-node* pada *tree* yang memiliki *child*. Pada keadaan ini *node* tersebut dapat dianggap sebagai root dalam *subtree*. Contoh, pada gambar 4.4 salah satu *node* yang memiliki *child* adalah Elizabeth I. Jadi *node* Elizabeth I, Claude dan Celia adalah sebuah *subtree* yang biasanya digambarkan dengan garis putus-putus yang membentuk segitiga.

7. *Visiting*

   Visiting adalah *node-node* yang telah dikunjungi atau telah dicek pada saat pencarian *node* tertentu dalam *tree*. Misalnya akan dicari *node* Alice pada gambar 4.4 dengan metode BFS

(*Breadth First Search*)atau DFS (*Depth First Search*) . *Node-node* yang telah dikunjungi sampai menemukan *node* Alice disebut sebagai *visiting*.

8. *Traversing*
   Traversing atau penelusuran adalah kegiatan mengunjungi seluruh *node* dalam urutan tertentu, misalnya inorder traversal, preorder traversal dan postorder traversal.
9. Level
   Level pada sebuah *tree* adalah banyaknya generasi dari awal *root* sampai pada *leaf*. Berdasarkan gambar 4.4 maka *tree* di atas memiliki 3 level yaitu level 0 adalah *root* yaitu *node* Charles. Level 1 adalah *child* dari *node* Charles yaitu Philip dan Elizabeth II. Level 2 adalah adalah *child* dari *node* Philip dan Elizabeth II yaitu Andrew, Alice, George VI, dan Elizabeth I. Level 3 adalah seluruh *leaf*.

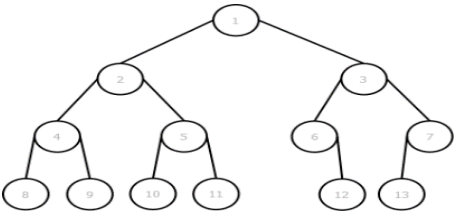
B. BINARY TREE

Pada materi sebelumnya telah dibahas tentang *tree* dan terminologinya. Oleh karena itu diasumsikan istilah *node*s dan *edges* dalam definisi telah dipahami. Begitupun dengan beberapa terminologi seperti *root, parent, child, leaf* dan *level* telah diketahui. Pada materi *binary tree* akan dibahas tentang apa itu *binary tree*, bagaimana representasi penerapannya dalam struktur data.

2. Definisi *Binary tree*

Salah satu struktur data *tree* yang umum digunakan adalah *Binary tree*. *Binary tree* atau pohon biner merupakan struktur data hirarkis yang terdiri dari *node*s dan *edges* dimana setiap *node* hanya boleh memiliki (maksimal) 2 *child left child* dan *right child*. Menurut (Goodrich et al., 2014) *binary tree* merupakan sebuah *ordered tree* yang memiliki 3 sifat utama yaitu (1) setiap *node* hanya boleh memiliki maksimal 2 *child*, (2) setiap *child* memiliki nama masing-masing yaitu *left child* dan *right child*, dan

(3) *node left child* selalu dikunjungi lebih awal daripada*node right child*.



Gambar 4. 5 Contoh binary tree
Sumber: (Binary tree Data Structure, n.d.)
https://www.geeksforgeeks.org/binary-tree-data-structure/

Perhatikan gambar 4.5. Pada level 0, *Node* 1 memiliki 2 *child* yaitu *left child* 2 dan *right child* 3. Pada level 1, *Node* 2 memiliki2 *child* yaitu *left child* 4 dan *right child* 5. Begitupun dengan *node* 3 memiliki 2 *child* yaitu *left child* 6 dan *right child* 7. Pada level 2 terdapat 4 *node* yaitu 4, 5, 6 dan 7. *Node* 4 dan 5 keduanya memiliki 2 *child* sementara pada *node* 6 dan 7 hanya memilikimasing-masing 1 *child*. Berdasarkan sifat binary *child*, contoh pada gambar 4.5 termasuk binary *child* karena setiap *node* tidak ada yang memiliki *child* lebih dari 2.

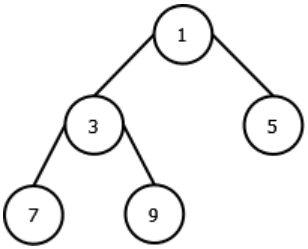
Bagaimana dengan gambar 4.1, apakah termasuk *Binary tree*?. Berdasarkan sifat *binary tree* yang menurut (Goodrich et al., 2014) maka gambar 4.1 bukan sebuah *binary tree* karena *node* pada level 0 memiliki *child* lebih dari 2. Begitupun salah salah satu *node* pada level 1, memiliki *child* lebih dari 2.

3. Jenis *Binary tree*

Ada beberapa jenis *binary tree* yang dibedakan atas struktur dan sifat khusus sesuai dengan kebutuhan dalam implementasinya. Beberapa jenis *binary tree* yang umum adalah *full binary tree, complete binary tree, perfect binary tree, balanced binary tree, degenerated binary tree, dan skewed binary tree.*
Full binary tree memiliki dua ciri yaitu:

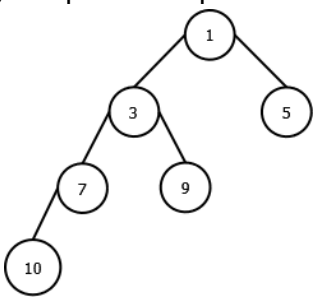
1. Setiap *node*s memiliki tepat 0 atau 2 *child*
2. Semua *leaf* berada pada level yang sama

Untuk lebih jelasnya dapat dilihat pada contoh berikut.



Gambar 4. 6 Full binary tree

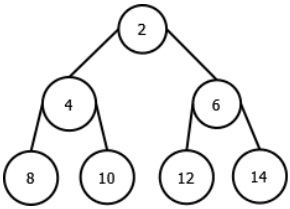
Sedangkan *complete binary tree* umumnya digunakan pada implementasi *heap array*. Adapun ciri utama pada *complete binary tree* yaitu semua level terisi penuh kecuali level terakhir yang merupakan *leaf* dimana *leaf* selalu terisi dari kiri ke kanan. Untuk lebih jelasnya dapat dilihat pada contoh berikut:



Gambar 4. 7 Complete binary tree

Perfect binary tree memiliki dua ciri yaitu:
1. Semua level terisi penuh dengan *node*s
2. Semua *leaf* berada pada level yang sama

Untuk lebih jelasnya dapat dilihat pada contoh berikut.

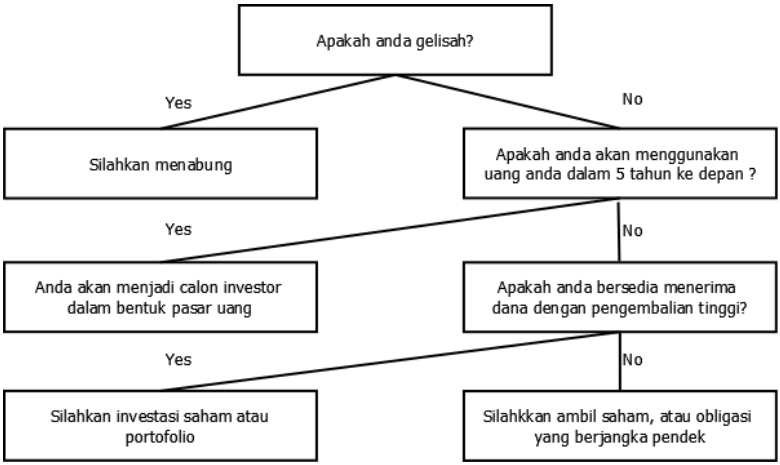


Gambar 4. 8 Perfect binary tree

4. Representasi *Binary tree*

Binary tree banyak diterapkan dalam dunia komputer. Umumnya *binary tree* melekat pada beberapa algoritma yang digunakan untuk menyelesaikan masalah seperti pencarian, pengurutan, kompresi data, ekspresi matematika, pengambilan keputusan, dan analisis sintaksis dalam kompilasi (Lee & Hubbard, 2014).

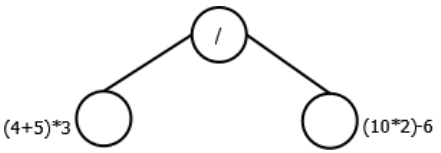
Salah satu representasi dari *binary tree* yang umum ditemukan adalah pada pohon keputusan atau decision *tree*. Decision *tree* merupakan salah satu bentuk *binary tree* karena setiap *node* yang mewakili pertanyaan tertentu hanya memiliki dua jawaban yaitu yes atau no. Artinya tidak ada *node*s dalam decision *tree* yang akan menghasilkan *child* lebih dari dua sehingga memenuhisyarat dari *binary tree*. Untuk lebih jelasnya perhatikan contoh decision *tree* berikut dalam memberikan rekomendasi pada calon investor.



Gambar 4. 9 Representasi binary tree pada decision tree
Sumber: (Goodrich et al., 2014)

Binary tree juga dapat direpresentasikan dalam aritmatika. Pada *binary tree* terdapat *node* internal yang selalu memiliki *child* kecuali *node leaf*. *Node* internal ini dapat diisi dengan operator, sedangkan *node leaf* dapat dapat diisi dengan variabel atau konstanta. Misalkan terdapat sebuah aritmatika yaitu $((((4 + 5) * 3)/((10 * 2) - 6))$. Bagaimana cara merepresentasikanaritmatika tersebut dalam *binary tree*?.

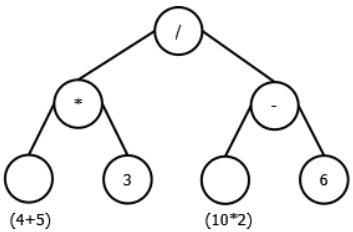
Langkah pertama adalah menentukan operator yang akan menjadi root pada *binary tree*. Umumnya operator yang berada di tengah adalah root. Diantara semua operator yang ada maka operator / (bagi) akan menjadi root pada level 0. Kemudian root / memiliki *left child* dan *right child* yang dideskripsikan gambar berikut.



Gambar 4. 10 Representasi binary tree pada aritmatika

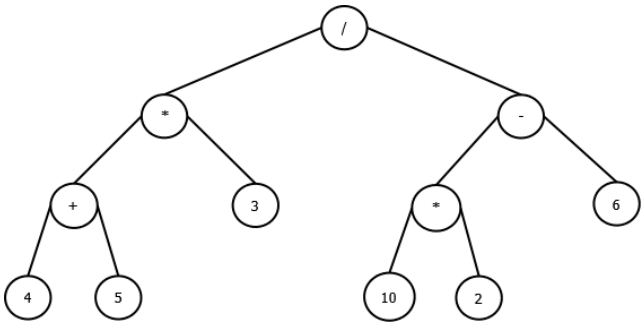
Persamaan $(4 + 5) * 3$ memiliki dua operator. Untuk itu harus ditentukan operator mana yang dapat mengisi *node* pada *left child* dari root /. Karena dipisahkan oleh kurung kurawal maka operator * adalah *left child* dari root bagi. Begitupun dengan $(10 * 2) - 6)$, masih memiliki dua operator dan operator – adalah *right child* dari root /.

Pada level 1, operator * dan – adalah *node* siblings yang juga berstatus parent. Oleh karena itu masing-masing *node*s juga memiliki *child* yaitu *left child* dan *right child* seperti pada gambar berikut.



Gambar 4. 11 Representasi binary tree pada aritmatika

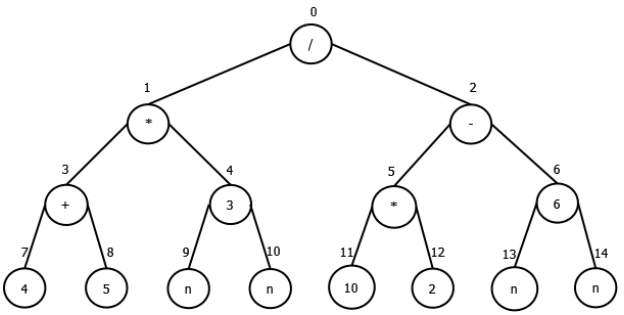
Pada level 2, *right child* dari *node*s * dan – sudah menjadi *leaf*. Artinya tidak ada lagi *child* pada *node* tersebut. Sementara pada *left child* dari operator * dan – masih dapat berstatus parent yang juga memiliki *child* yaitu *left child* dan *right child* seperti pada gambar dibawah ini.



Gambar 4. 12 Representasi binary tree pada aritmatika

Pada level 3, semua *node* sudah tidak memiliki *child* atau sudah tidak ada operator yang dapat dipisahkan. Oleh karena itu keseluruhan *node* pada level 3 adalah *leaf* dan representasi aritmatika $((((4+5)*3)/((10*2)-6))$ ke dalam bentuk *binary tree* telah selesai dilakukan.

Binary tree juga dapat merepresentasikan sebuah array. Pada umumnya, representasi ini mengambil sifat dari struktur *binary tree* yaitu setiap simpul hanya memiliki maksimal dua *child* yaitu *left child* dan *right child*. Untuk lebih jelasnya, representasi *binary tree* dalam array akan diimplementasikan pada gambar 4.12 di atas yang proses implementasinya ditunjukan oleh gambar 4.13 di bawah ini.



Gambar 4. 13 Penentuan indeks array pada binary tree

Hal-hal yang harus diperhatikan saat menentukan indeks array pada *binary tree* diantaranya yaitu menentukan root pada *tree* kemudian diberi indeks 0. Kemudian indeks berikutnya diberikan sesuai level yang dimulai dari kiri-kanan. Jika terdapat *node* yang tidak memiliki *child* maka indeks tetap diberikan dan *child* diberi label null atau n.

Berdasarkan gambar 4.13 di atas dapat diketahui bahwa terdapat 14 indeks yang diawali dari 0 sampai 14. Artinya terdapat 15 *node* pada gambar di atas, akan tetapi hanya 11 *node* yang aktif dan 4 *node*nya tidak aktif. Hal ini disebabkan oleh beberapa *node* yang tidak memiliki *child* akan tetapi tetap

diberikan indeks. Oleh karena itu representasi array pada gambar 4.12 ditunjukan oleh gambar 4.14 berikut.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
/	*	-	+	3	*	6	4	5			10	2		

Gambar 4. 14 Representasi array pada binary tree

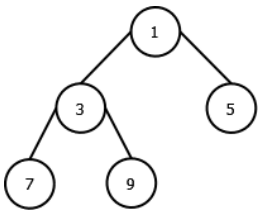
C. RANGKUMAN

Tree atau pohon merupakan struktur data hierarkis yang terdiri atas *node*s yang dihubungkan oleh *edges*. *Node* dapat merepresentasikan data apa saja sementara *edges* merupakan lintasan yang menghubungkan antara *node*s. Dalam *tree* setiap *node* dapat memiliki *child* kecuali *node* yang berstatus sebagai *leaf*. Ada beberapa terminologi umum dalam *tree* diantaranya adalah *path, root, parent, child, leaf, subtree,visiting, traversing, dan level*. Sedangkan *binary tree* atau pohon biner merupakan jenis khusus dari *tree* yang memiliki karakteristik yaitu setiap *node*s hanya boleh memiliki *child* maksimal sebanyak dua yaitu *left child* dan *right child*. Dalam menggunakan *binary tree*, *child* akan selalu memiliki nilai yang lebih kecil daripadaparent, sementara *left child* akan selalu lebih kecil daripada*right child*. *Binary tree* memiliki beberapa jenis yaitu *full binary tree, complete binary tree, perfect binary tree, balanced binary tree, degenerated binary tree, dan skewed binary tree.* Perbedaan jenis dalam *binary tree* ini didasarkan pada implementasinya dalam menyelesaikan masalah. Selain itu juga, *binary tree* dapat merepresentasikan suatu decision *tree*, aritmatika dan array. Pada dasarnya *tree* dan *binary tree* merupakan struktur data non linear yang sangat penting dalam ilmu komputer yang umumnya melekat pada algoritma tertentu yang menjanjikan efisiensi.

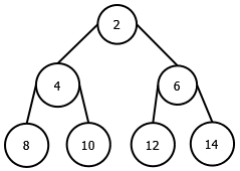
D. TES FORMATIF

Pilihlah salah satu jawaban yang paling benar menurut anda.

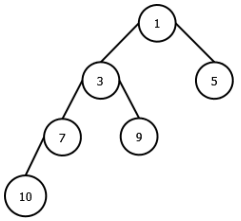
1. Sebuah *node* yang tidak memiliki *child* lagi disebut ...
   a. *Root*
   b. *Edge*
   c. *Leaf*
   d. *Path*
2. Sebuah *tree* selalu memiliki *node* yang berada ada level nol. *Node* tersebut disebut sebagai...
   a. *Root*
   b. *Subroot*
   c. *Leaf*
   d. *Path*
3. Berdasarkan gambar di samping, *node* yang berstatus sebagai sibling yang berada pada level 1 adalah ...
   a. 1
   b. 3 dan 5
   c. 3 dan 7
   d. 7 dan 9



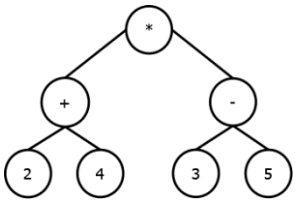
4. Berdasarkan gambar di samping, ada berapa banyak *subtree* yang tersedia...
   a. 0
   b. 1
   c. 2
   d. 3



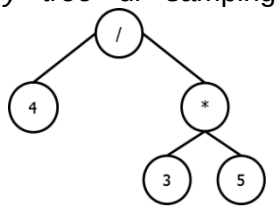
5. Berdasarkan gambar di samping, *node* yang berstatus sebagai *leaf* adalah ...
   a. 3 dan 5
   b. 7 dan 9
   c. 9 dan 5
   d. 10, 9, dan 5



6. Setiap *node* dapat memiliki. Berapa banyak maksimal *child* dari sebuah *node* agar dapat disebut sebagai *binary tree*...
   a. 1
   b. 2
   c. 3
   d. 4
7. Ada beberapa jenis dari *binary tree*. Berdasarkan gambar pada soal nomor 8, maka jenis binary yang tepat adalah ...
   a. *Full binary tree*
   b. *Complete binary tree*
   c. *Perfect binary tree*
   d. *Skewed binary tree*
8. Berdasarkan gambar disamping, bentuk aritmatika yang tetap adalah ...



   a. $(2 + 4) - (3 * 5)$
   b. $(2 + 4) * (3 - 5)$
   c. $(2 + 4) - (3 * 5)$
   d. $(3 + 4) * (2 - 5)$
9. Jumlah *node* yang aktif pada gambar soal nomor 10 adalah ...
   a. 5
   b. 6
   c. 7
   d. 8
10. Representasi array berdasarkan *binary tree* di samping adalah...



a.	/	4	*	3	5		
b.	/	4			*	3	5
c.			/	4	*	3	5
d.	/	4	*			3	5

E. LATIHAN

1. *Tree* dapat digunakan untuk merepresentasikan data yang memiliki hierarkis seperti struktur keluarga. Buatlah *tree* yang mendeskripsikan struktur keluargamu dimana kakekmu berkedudukan sebagai root.
2. Berdasarkan soal nomor 1, tentukan terminologi apa saja yang dapat dilabeli pada *tree* yang telah dibuat
3. Jelaskan perbedaan antara full *binary tree* dan complete *binary tree*. Berikan contoh masing-masing.
4. *Tree* dapat merepresentasikan sebuah aritmatika. Ubah aritmatika berikut yaitu $\big(3 \times (4 + 5)\big)/\big(6 - (10 \times 2)\big)$ke dalam bentuk *binary tree*.
5. *Tree* dapat merepresentasikan sebuah array. Soal nomor 4 menghasilkan sebuah *binary tree*. Ubah *binary tree* tersebut ke dalam bentuk array. Tentukan berapa banyak *node* yang aktif dan *node* yang tidak aktif.

KEGIATAN BELAJAR 5

PENGANTAR GRAF: REPRESENTASI GRAF DAN PENELUSURAN GRAF (DEPTH FIRST BREATH FIRST SEARCH)

DESKRIPSI PEMBELAJARAN

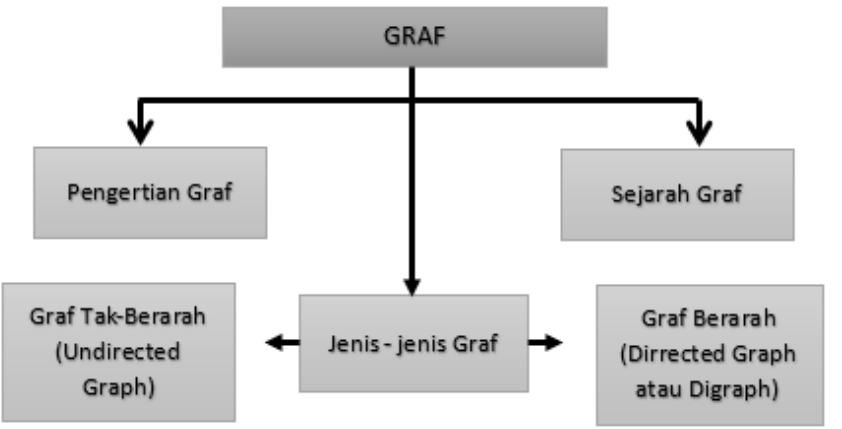
Mahasiswa mempelajari sejarah dan pengenalan jenis – jenis graf dan konsep dasar teoritis graf. Mahasiswa diharapkan memiliki wawasan dan pemahaman untuk modal dasar mempelajari dan menerapkan graf lebih lanjut.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu mengetahui Sejarah dan definisi graf.
2. Mampu menjelaskan fungsi dan manfaat pada graf
3. Mampu menjelaskan jenis-jenis graf.

PETA KONSEP PEMBELAJARAN



A. PENGERTIAN DAN SEJARAH

Teori Graf atau teori grafik dalam matematika dan ilmu komputer merupakan cabang kajian yang mempelajari sifatsifat graf atau grafik. Graf merupakan sekumpulan objek terstruktur dimana beberapa pasangan objek mempunyai hubungan atau keterkaitan.

Teori Graf ini pertamakali diperkenalkan oleh seorang matematikawan Swiss, Leonhard Euler, dalam sebuah artikel ilmiah yang berjudul “Seven Brdiges of Konigsberg” yang membahas ada atau tidaknya struktur yang saat ini dikenal sebagai sirkuit Euler pada graf keterhubungan daratan kota Konigsberg dan pulau kecil ditengah sungai Pregel yang dihubungkan oleh tujuh buah jembatan.

Graf pertama kali dikenalkan oleh Euler pada tahun 1736 sebagai solusi dari jembatan konisbreg. Graf merupakan salah satu bentuk pengembangan dari himpunan tegas. Pewarnaan graf merupakan salah satu bagian dari graf yang paling banyak diaplikasikan pada kehidupan nyata. (Rahayuningsih 2018)

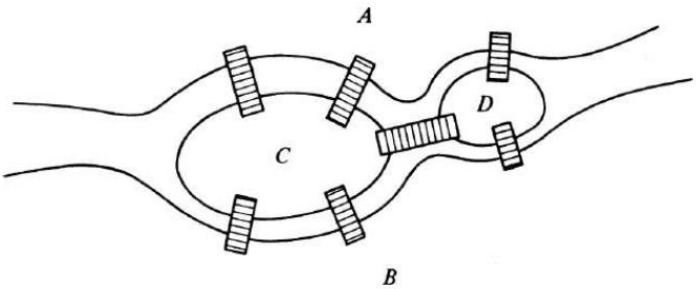
Tahun 1847, G.R. Kirchoff (1824 – 1887) berhasil mengembangkan teori pohon (Theory of trees) yang digunakan dalam persoalan jaringan listrik. Sepuluh tahun kemudian, A. Coyley (1821 – 1895) juga menggunakan konsep pohon untuk menjelaskan permasalahan kimia yaitu hidrokarbon. (Prakasta and Munandar 2023)

Karena kesederhanaannya yang melekat, teori graf memiliki rentang yang sangat luas

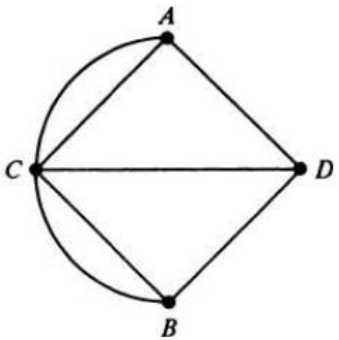
Aplikasi dalam Teknik, dalam Ilmu Fisika, Sosial, dan Biologi, dalam linguistik, dan di banyak bidang lainnya. Grafik dapat digunakan untuk mewakili hampir setiap situasi fisik yang melibatkan objek diskrit dan hubungan di antara mereka.

Berikut ini adalah empat contoh dari antara ratusan aplikasi tersebut.

Masalah Jembatan Königsberg: Masalah jembatan Königsberg mungkin adalah contoh paling terkenal dalam teori grafik. Itu adalah masalah lama sampai dipecahkan oleh Leonhard Euler (1707-1783) pada tahun 1736, dengan menggunakan grafik. Euler menulis makalah pertama dalam teori graf dan dengan demikian menjadi pencetus teori grafik serta topologi lainnya. Masalahnya digambarkan dalam Gambar 1-4 dua pulau, C dan D, dibentuk oleh Sungai Pregel di Königsberg (kemudian ibukota Prusia Timur tetapi sekarang berganti nama menjadi Kaliningrad dan di Soviet Barat Rusia) terhubung satu sama lain dan ke bank A dan B dengan tujuh jembatan (Deo 2016)



Gambar 5.1: Königsberg bridge problem
Sumber: (Deo 2016)



Gambar 5.2: Graph of Königsberg bridge problem
Sumber: (Deo 2016)

B. DEFINISI GRAF

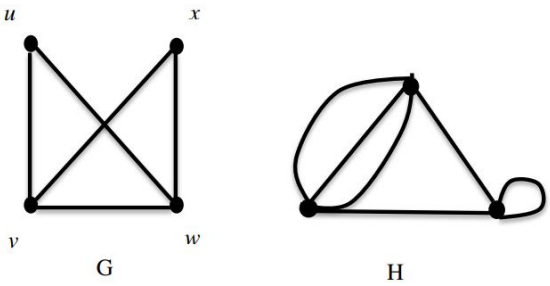
Suatu graph terdiri dari suatu himpunan tak kosong yang masing masing unsurnya disebut titik (*vertex*) dan suatu himpunan pasangan tak berurutan dari titik- titik tersebut yang disebut sisi (*edge*).

Di sini G melambangkan suatu graph. Himpunan titik di graph G dinyatakan dengan V(G) dan himpunan sisi di graph G dinyatakan dengan E(G). Jika banyak titik dan banyak sisi di G terhingga, maka G disebut graph terhingga.

Dua sisi atau lebih yang menghubungkan satu pasang titik disebut sisi rangkap (*multiple edges*). Suatu sisi yang titik ujungnya sama disebut loop. Graph tanpa sisi rangkap dan tanpa loop disebut graph sederhana (*simple graph*).

Jika u dan v titik–titik di G dan e = *uv* suatu sisi di G, maka dikatakan:
e menghubungkan u dan v,
u dan v terhubung langsung (*adjacent*),
u terkait (*incident*) dengan e,

e terkait (*incident*) dengan u,
u dan v di sebut titik ujung dari e
Contoh



Gambar 5.3: Contoh Graph G dan H
Sumber: (Deo 2016)

Pada gambar 1.3 , graph G adalah sederhana, dengan

V(G) = {u,v,w,x} E (G) = {uv, uw, vw, vx, wx}

|V(G)| = 4|E(G)| = 5
graph H tidak sederhana karena memuat loop dan sisi rangkap (Prakasta and Munandar 2023)

Sebuah graf linier (atau secara sederhana disebut graf) *G* = (*V,E*) adalah suatu sistem yang terdiri atas suatu himpunan objek V = {v1, *v2, …*} yang disebut himpunan titik dan sebuah koleksi *E* = {*ei, e2, …*} yang merupakan koleksi sisi sedemikian hingga tiap sisi *ek* dikaitkan dengan suatu pasangan tak-terurut (v*1*, vj). Titik v1, *vj* yang berkaitan dengan *ek* disebut titik-titik ujung sisi *ek.* (Buhaerah, Busrah, and Sanjaya 2019)

Cara merepresentasikan sebuah graf yang paling umum adalah dengan diagram. Dalam diagram tersebut, titik-titik dinyatakan sebagai noktah dan tiap sisi dinyatakan sebagai kurva sederhana yang menghubungkan tiap dua titik. (Deo 2016)

C. JENIS JENIS GRAF

Berdasarkan orientasi arah pada sisi, maka secara umum graf dibedakan menjadi dua jenis

a. Graf Tak-Berarah (*Undirected Graph*)

Graf yang sisinya tidak mempunyai orientasi arah disebut graf tak terarah. Pada graf tak berarah, urutan pasangan simpul yang dihubungkan oleh sisi tidak diperhatikan. Jadi, (*u,v*)=(*v,u*) adalah sisi yang sama.

Grafik yang dipelajari dalam buku ini sejauh ini adalah grafik yang tidak diarahkan. Tidak arah ditugaskan ke tepi dalam grafik. Sebuah tepi ek antara simpul vi dan v

J dapat dianggap pergi dari vertex VI ke vertex VJ atau dari VJ ke VI. Di bagian ini kita akan mempertimbangkan grafik terarah – grafik di mana tepi memiliki arah.

Banyak situasi fisik membutuhkan grafik yang diarahkan. Peta jalan kota dengan jalan satu arah, jaringan aliran dengan katup di pipa, dan jaringan listrik, misalnya, diwakili oleh grafik yang diarahkan. Grafik terarah digunakan dalam representasi abstrak dari program komputer, di mana simpul berdiri untuk instruksi program dan tepi menentukan urutan eksekusi. Grafik terarah adalah alat yang sangat berharga dalam studi mesin sekuensial. Diarahkan Grafik dalam bentuk grafik aliran sinyal digunakan untuk analisis sistem dalam kontrol teori.

Sebagian besar konsep dan terminologi grafik yang tidak diarahkan juga berlaku ke grafik yang diarahkan. Misalnya Planaritas grafik tidak tergantung pada apakah grafik diarahkan atau tidak diarahkan, dan oleh karena itu baik grafik terarah maupun tidak terarah. Hal yang sama berlaku untuk sebagian besar topik lainnya tertutup sejauh ini. Akan untuk mencurahkan lagi untuk penelitian ini grafik terarah, sebagian besar berulang, dengan perubahan kecil, apa yang sudah terjadi Mengatakan.

Oleh karena itu, dalam bagian ini, kami terutama akan mengeluarkan sifat-sifat tersebut Grafik terarah yang tidak dibagikan oleh grafik yang tidak diarahkan.

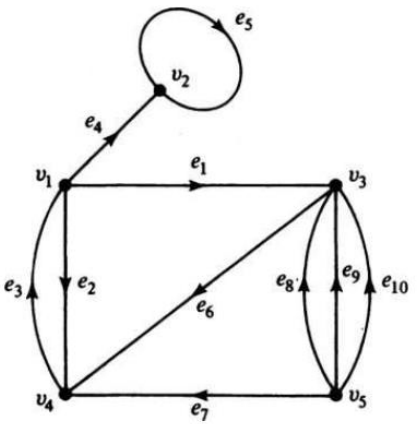
Untuk grafik yang tidak terarah, tetangga (bukan penerus) dari setiap vertex terdaftar. Oleh karena itu, setiap tepi muncul dua kali − redundansi yang jelas.

Untuk membandingkan efisiensi penyimpanannya dengan matriks adjacency, biarkan d_{av} menjadi derajat rata-rata (derajat keluar dalam kasus digraf) dari simpul-simpul di grafik. Dengan asumsi bahwa satu kata komputer diperlukan untuk label setiap simpul, Persyaratan penyimpanan total untuk grafik vertex n adalah kata-kata n $(1 + d_{av})$. Dengan demikian daftar penerus lebih efisien daripada matriks adjacency anda pasti telah mengamati bahwa metode representasi grafik di atas tidak sepenuhnya berbeda. Bahkan, mereka selalu terkait dalam hal yang mereka sampaikan informasi yang sama. Variasi tambahan dalam representasi ini dapat dibuat agar sesuai dengan persyaratan yang ada. Misalnya, grafik tertimbang dapat berupa matrils yang diwakili oleh n oleh n matriks berat (juga disebut matriks biaya atau jarak matriks), yang seperti matriks adjacency kecuali bahwa alih-alih 1 bobot tepi muncul sebagai entri dalam matriks. Namun, itu harus disimpan di Ingatlah bahwa dalam banyak masalah, efisiensi algoritma mungkin tergantung pada bentuk di mana grafik disajikan. Dengan demikian pilihan data yang tepat Struktur itu penting. (Deo 2016)

b. Graf Berarah (*Dirrected Graph atau Digraph*)

Graf yang setiap sisinya diberikan orientasi arah disebut sebagai graf berarah. Pada graf berarah, (u,v) dan (v,u) menyatakan sisi berarah yang berbeda, dengan kata lain $(u,v) \neq (v,u)$. Untuk sisi berarah (u,v), simpul u dinamakan simpul asal dan simpul v dinamakan simpul terminal.

Grafik terarah (atau digraf singkatnya) G terdiri dari satu set simpul V = {v1, v2, . . .}, satu set tepi E = {e1, e2, . . .}, dan pemetaan Ψ yang memetakan setiap tepi ke beberapa pasangan simpul yang dipesan (vi, vj). Seperti dalam kasus tidak terarah grafik, simpul diwakili oleh titik dan tepi oleh segmen garis antara dan vj dengan panah diarahkan dari vi ke vj. Misalnya, Gambar 1.4 menunjukkan digraf dengan lima simpul dan sepuluh tepi. Digraf juga disebut sebagai grafik berorientasi.



Gambar 5.4: Directed graph with 5 vertices and 10 edges
Sumber: (Deo 2016)

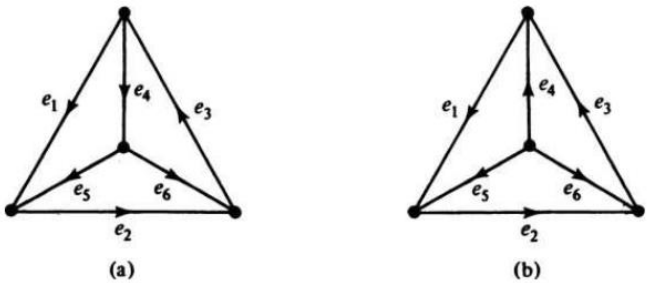
Dalam digraf, tepi tidak hanya insiden pada simpul, tetapi juga insiden di luar simpul dan insiden menjadi simpul. Vertex v_i, yang ujungnya ek adalah insiden keluar dari, disebut simpul awal e_k. Vertex v_j, yang ek adalah insiden menjadi, adalah disebut simpul terminal ek. Pada Gambar 1.4, v_5 adalah simpul awal dan v_4 adalah simpul terminal tepi E_7. Tepi di mana simpul awal dan terminal berada.

Hal yang sama membentuk self-loop, seperti E_5. (Beberapa penulis mencadangkan istilah busur untuk tepi berorientasi atau

terarah. Kami menggunakan istilah tepi untuk berarti tidak terarah atau tepi yang diarahkan. Setiap kali ada kemungkinan kebingungan, kami akan secara eksplisit state directed atau undirected edge.

Jumlah insiden tepi dari vertex v_i disebut out-degree (atau out-valence atau outward demidegree) dari v_i dan ditulis $d^+(v_i)$. Jumlah Insiden tepi menjadi VI disebut in-degree (atau invalence atau inward demidegree) dari v_1 dan ditulis sebagai $d^-(v_i)$. Pada Gambar 1.4, misalnya,

$d^+(v_1) = 3$, $d^-(v_1) = 1$,
$d^+(v_2) = 1$, $d^-(v_2) = 2$,
$d^+(v_5) = 4$, $d^-(v_5) = 0$.



Gambar 5.5 Two nonisomorphic digraphs
Sumber: (Deo 2016)

Digraf Isomorfik: Grafik isomorfik didefinisikan sedemikian rupa sehingga mereka memilikiperilaku identik dalam hal properti grafik. Dengan kata lain, jika label mereka

Dihapus, dua grafik isomorfik tidak dapat dibedakan. Untuk dua digraf menjadiisomorfik tidak hanya harus grafik tidak terarah yang sesuai menjadi isomorfik, tetapi arah dari tepi yang sesuai juga harus setuju. Misalnya, Gbr. 9-2 menunjukkan dua digraf

yang tidak isomorfik, meskipun mereka adalah orientasi dari grafik tidak terarah yang sama.

Gambar 5.5 segera menunjukkan masalah. Berapa jumlah yang berbeda (yaitu, nonisomorfik) orientasi dari grafik yang tidak diarahkan yang diberikan? Masalahnya adalah diselesaikan oleh F. Harary dan E. M. Palmer pada tahun 1966.

Digraf Sederhana: Digraf yang tidak memiliki tepi self-loop atau paralel disebutdigraf sederhana (Gambar 1.5 dan 1.6, misalnya).

Digraf Asimetris: Digraf yang memiliki paling banyak satu tepi terarah antara sepasang simpul, tetapi diizinkan untuk memiliki loop sendiri, disebut asimetris atau antisimetris.

Digraf Simetris: Digraf di mana untuk setiap tepi (a, b) (yaitu, dari vertex a ke b) ada juga tepi (b, a).

Digraf yang sederhana dan simetris disebut simetris sederhana digraf. Demikian pula, digraf yang sederhana dan asimetris itu sederhana Asimetris. Alasan untuk istilah simetris dan asimetris adalah jelas dalam konteks hubungan biner.

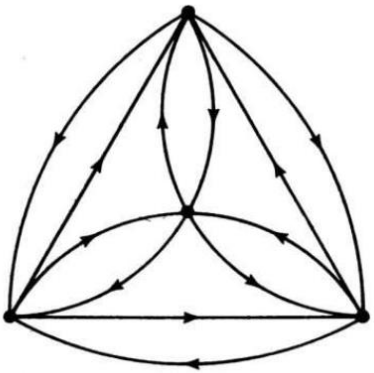
Digraf Lengkap: Grafik lengkap yang tidak diarahkan didefinisikan sebagai grafik sederhana grafik di mana setiap simpul bergabung dengan setiap simpul lainnya persis dengan satu sisi.

Untuk digraf kami memiliki dua jenis grafik lengkap. Simetris lengkap digraph adalah digraf sederhana di mana ada tepat satu tepi diarahkan dari setiap simpul ke setiap simpul lainnya (Gbr. 1.6), dan digraf asimetris lengkap adalah digraf asimetris di mana ada tepat satu tepi antara setiap pasangan simpul (Gbr. 1.5).

Digraf asimetris lengkap dari n simpul berisi tepi n (n – l) / 2, tetapi a digraf simetris lengkap dari n simpul berisi tepi n (n – 1).

Lengkap digraph asimetris juga disebut turnamen atau turnamen lengkap.

Sebuah digraf dikatakan seimbang jika untuk setiap simpul vi derajat sama dengan derajat luar. (Deo 2016)

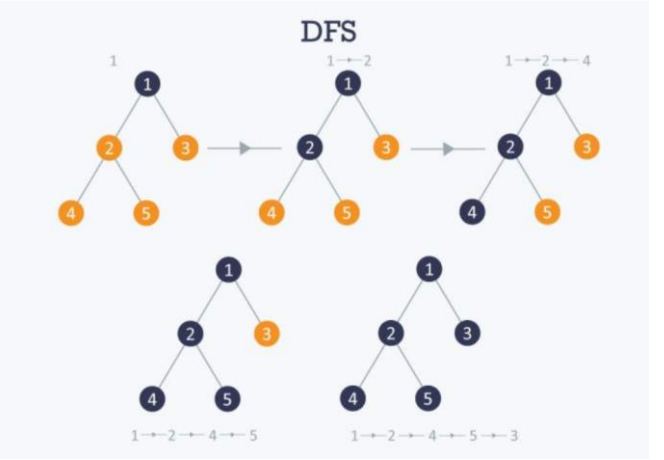


Gambar 5.6 Complete symmetric digraph of four vertices
Sumber: (Deo 2016)

D. DEPTH-FIRST SEARCH (DFS)

Depth-first search (DFS) adalah salah satu algoritma penelusuran graf. Dalam setiap iterasi pencarian, DFS akan mengunjungi terlebih dahulu simpul yang bertetangga dengan simpul yang sedang diperiksa dan simpul tersebut belum pernah dikunjungi. Setiap simpul yang sudah dikunjungi akan ditandai agar DFS tidak memroses simpul yang sama dua kali. Proses tersebut dilakukan terus menerus sampai DFS mencapai simpul yang tidak memiliki tetangga yang belum pernah dikunjungi. Maka dari itu, DFS kemudian kembali ke simpul sebelumnya dan mencari lagi tetangga yang belum pernah dikunjungi. Proses ini dilakukan terus menerus sampai DFS kembali lagi ke simpul awal, dan pencarian pun selesai. Jika masih ada simpul yang belum dikunjungi, DFS akan

mulai mencari lagi dari simpul tersebut, dengan mengikuti langkahlangkah diatas.



Gambar 5.7. Ilustrasi penelusuran graf dengan algoritma DFS (Sumber : (Setya 2018)

Dalam beberapa kasus, kita dapat memodifikasi DFS agar tidak mencari ke seluruh simpul dalam graf. Kita bisa merancang algoritma DFS yang hanya mencari satu simpul saja dari apa yang kita ingin cari dalam graf, sehingga ketika menemukan simpul yang dicari, algoritma berhenti. Namun, tentunya cara penelusuran graf yang digunakan algoritma harus seperti definisi DFS, yaitu dengan mencari ke simpul tetangga terlebih dahulu.

Karena pohon juga merupakan graf, maka kita bisa menerapkan algoritma DFS untuk mencari simpul pada pohon. Maka, pencarian akan dimulai dari akar sebuah pohon, lalu diteruskan ke salah satu anaknya terlebih dahulu. Hal itu diulang terus menerus sampai menemukan daun. Ketika menemukan daun, kembalilah ke simpul orangtuanya dan mencari ke anak lain yang belum ditelusuri. Hal tersebut diulang terus menerus sampai algoritma kembali ke akar dan semua simpul telah dikunjungi.

Satu hal yang perlu diperhatikan saat menelusuri pohon dengan DFS adalah kedalaman pohon tersebut. Terkadang, suatu pohon bisa memiliki upapohon yang memiliki kedalaman sangat besar, padahal apa yang dicari tidak ada pada upapohon tersebut. Selain itu, perlu diperhatikan pula jika kedalaman pohon bisa tidak terbatas. Hal tersebut akan membuat algoritma DFS akan berjalan lambat, bahkan tidak bisa berhenti jika kedalaman pohon tidak terbatas. Maka dari itu, algoritma DFS cocok untuk menelusuri pohon yang memiliki kedalaman terbatas. (Setya 2018)

E. BREADTH FIRST SEARCH (BFS)

Algoritma Breadth First Search (BFS) merupakan algoritma penelusuran graf. Breadth First Search meupakan algoritma sederhana untuk melakukan penelusuran pada graf, algoritma ini merupakan algoritma yang menjadi pola dasar dan fundamental bagi banyak algoritma graf. Algoritma BFS menelusuri batas ambang antara node yang sudah dikunjungi dan node yang belum dikunjungi. Algoritma ini mengunjungi node sedemikian sehingga setiap node yang berjarak dari node awal akan dikunjungi sebelum mengunjungi node yang berjarak s+1 dari node awal(layer orde). Penelusuan graf pada algoritma Breadth First Search dilakukan dengan menelusuri setiap node dari awal. Struktur data antrian(queue) digunakkan untuk melakukan proses ekspansi Breadth First Search.

Struktur data antrian digunakkan untuk menampung setiap node atau kemungkinan node yang valid untuk dilalui. Sifat dari queue adalah First In First Out. Oleh karena itu setiap node yang dipilih dan di-expand pertama kali oleh algoritma ini akan pertama kali juga diproses dan ditelusuri. Sifat dari queue tersebut mempertahankan sifat dari algoritma Breadth First Search itu sendiri, dimana algoritma ini menelusuri secara layer order. Setiap node yang telah

ditelusuri oleh algoritma breadth first search tidak akan dikunjungi lagi.

Dewasa ini Algoritma BFS merupakan algoritma untuk penentuan jalur terpendek antara titik awal dengan titik akhir tanpa melewati hambatan. Algoritma BFS menggunakkan pendekatan heuristik yang menelusuri seluruh verteks yang direpresentasikan dengan node, dengan cara memperikirakan rute tebaik yang dapat dilalui dari node tersebut. (Alkindi et al. 2018)

F. RANGKUMAN

Berdasarkan uraian di atas di mulai dari teori graf sampai dengan algoritma graf Suatu graph terdiri dari suatu himpunan tak kosong yang masing masing unsurnya disebut titik (*vertex*) dan suatu himpunan pasangan tak berurutan dari titik- titik tersebut yang disebut sisi (*edge*).

Berdasarkan orientasi arah pada sisi, maka secara umum graf dibedakan menjadi dua jenis :
a. Graf Tak-Berarah (Undirected Graph)
b. Graf Berarah (Dirrected Graph atau Digraph)

Depth-first search (DFS) adalah salah satu algoritma penelusuran graf. Dalam setiap iterasi pencarian, DFS akan mengunjungi terlebih dahulu simpul yang bertetangga dengan simpul yang sedang diperiksa dan simpul tersebut belum pernah dikunjungi.

Algoritma Breadth First Search (BFS) merupakan algoritma penelusuran graf. Breadth First Search meupakan algoritma sederhana untuk melakukan penelusuran pada graf, algoritma ini merupakan algoritma yang menjadi pola dasar dan fundamental bagi banyak algoritma graf. Algoritma BFS menelusuri batas ambang antara node yang sudah dikunjungi dan node yang belum dikunjungi.

G. TES FORMATIF

1. Pernyataan yang benar berikut ini adalah
   a. Graf campuran memiliki sisi berarah dan tidak berarah
   b. Diperbolehkan sisi rangkap pada graf sederhana
   c. Graf berarah sederhana mempunyai gelang

2. Dalam graf sederhana mempunyai sifat
   a. Tak berarah
   b. Berarah
   c. Tak berarah dan berarah
   d. Diperbolehkan gelang

H. LATIHAN

Sebutkan dan jelaskan jenis jenis graf.
Berikan contoh penerapan pada kehidupan sehari hari.

KEGIATAN BELAJAR 6

MINIMUM SPANNING TREE (MST)

DESKRIPSI PEMBELAJARAN

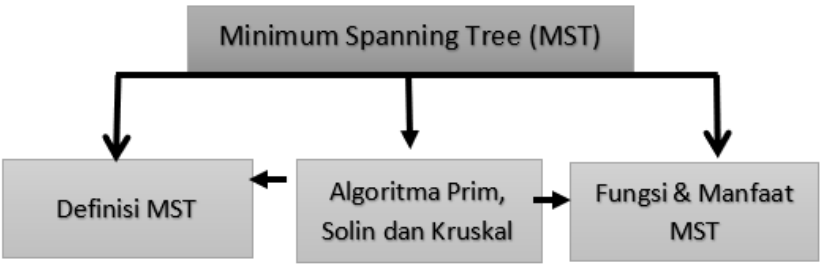
Pada bab ini mahasiswa mempelajari pengenalan dan konsep dasar teoritis tentang Minimal Spanning Tree /MST (pohon rentang minimal). Diharapkan mahasiswa memiliki wawasan dan pemahaman tentang Minimal Spaning Tree lebih lanjut.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu menjelaskan definisi tentang minimum spanning tree/MST (pohon rentang minimal).
2. Mempu menjelaskan fungsi dan manfaat minimal spanning tree.
3. Mampu menguraikan pohon merentang minimum (MST) sebagai pohon merentang yang mempunyai bobot minimum di antara semua pohon merentang yang mungkin
4. Menggunakan Algorima Prim, Solin dan Kruskal

PETA KONSEP PEMBELAJARAN

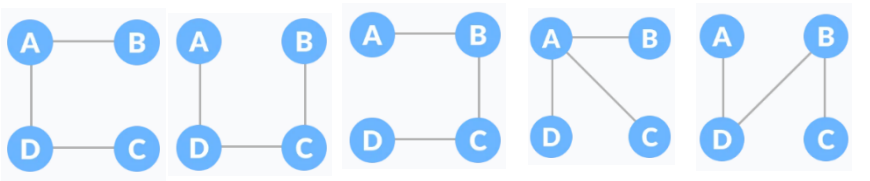


A. PENGERTIAN MINIMUM SPANNIG TREE (MST)

Sebelum kita mempelajari pohon merentang minimal (minimum spanning tree), kita perlu memahami dua graf yaitu graf tak berarah dan graf terhubung. Graf tak berarah adalah graf yang sisi-sisinya tidak mengarah ke segala arah (yaitu sisi-sisinya bersifat dua arah). Graf terhubung adalah graf yang selalu terdapat jalur dari suatu simpul ke simpul lainnya.

Pohon merentang minimal (minimal spanning tree) adalah sub-graf dari graf terhubung tak berarah yang memuat semua simpul dari graf tersebut dengan jumlah sisi yang seminimal mungkin. Jika sebuah simpul terlewatkan, maka simpul tersebut bukan pohon merentang. Pohon merentang minimum (minimum spanning tree) adalah pohon merentang yang jumlah bobot rusuknya seminimal mungkin.

Contoh pohon rentang



Pohon merentang minimum merupakan salah satu permasalahan yang paling banyak terjadi, masalah yang dipelajari baik di bidang algoritma grafik dan juga merupakan sub masalah dari masalah sulit Non deterministic-Polinomial (NP) seperti masalah perjalanan petugas pengantar surat (Khan, Aesha, and Sarker 2018), selain itu aplikasi dibidang taksonomi pada jaringan komputer (Bharath-Kumar and Jaffe 1983), pencitraan pada medis (An and Chavez 2000), jaringan pasokan air dan transportasi jaringan (Akpan and Iwok 2017) dan lain lain.

B. FUNGSI MINIMUM SPANNING TREE DALAM TEORI GRAF

Minimum Spanning Tree (MST) memiliki beberapa fungsi penting dalam teori graf:

1. **Koneksi Semua Node (Vertex): MST** memastikan bahwa semua node (simpul atau vertex) dalam graf terhubung satu sama lain. Ini memastikan bahwa tidak ada simpul yang terisolasi atau terputus dari graf.
2. **Menghubungkan dengan Bobot Minimum:** MST menghubungkan semua node dalam graf dengan bobot minimum total. Bobot atau panjang kabel/jarak yang dibutuhkan untuk menghubungkan node-node ini adalah minimum mungkin. Oleh karena itu, MST menyediakan solusi yang efisien untuk beberapa masalah jarak atau biaya minimum.
3. **Menghindari Siklus:** MST adalah subgraf dari graf asli yang tidak memiliki siklus. Ini berarti tidak ada lingkaran tertutup dalam MST. Siklus tidak diperlukan dalam pohon (tree), dan MST memastikan bahwa koneksi antara node dilakukan tanpa membentuk siklus.
4. **Pemeliharaan Jaringan Terhubung:** Dalam konteks jaringan komunikasi atau distribusi, MST membantu memelihara jaringan terhubung dengan meminimalkan total biaya koneksi antara node. Ini juga dapat diterapkan pada permasalahan kabel atau jalur komunikasi.(Hougaard and Tvede 2015)
5. **Aplikasi pada Jaringan Komputer:** MST dapat diterapkan pada jaringan komputer untuk mengoptimalkan penggunaan jalur koneksi dan meminimalkan latensi komunikasi. Ini dapat membantu dalam merancang infrastruktur jaringan yang efisien.(Briscoe et al. 2016)
6. **Rancang Bangun Jaringan Listrik atau Telekomunikasi:** Dalam konteks perancangan infrastruktur, MST dapat digunakan untuk merancang jalur listrik atau jalur telekomunikasi dengan biaya minimum.(Romanuke 2023)

7. **Routing dalam Jaringan Komunikasi:** MST dapat diterapkan pada protokol routing untuk mengoptimalkan jalur komunikasi antara node-node dalam jaringan.(Anon 2020)

Penerapan MST dapat memberikan solusi efisien dalam berbagai konteks dan permasalahan optimasi di berbagai bidang

C. MANFAAT DAN FUNGSI PENERAPAN SPANNING TREE DALAM BERBAGAI BIDANG

Minimum Spanning Tree (MST) memiliki manfaat dan fungsi yang dapat mengoptimalkan berbagai aspek kehidupan sehari-hari. Berikut adalah beberapa manfaat dan fungsi penerapan MST dalam berbagai bidang:

1. Jaringan Komunikasi:
   - Manfaat: MST dapat digunakan untuk merancang jaringan telekomunikasi atau jaringan komputer dengan efisiensi tinggi, mengoptimalkan jalur koneksi dan meminimalkan biaya.
   - Fungsi: Menentukan jalur komunikasi yang optimal untuk menghubungkan perangkat-perangkat elektronik di rumah atau kantor.
2. Transportasi dan Logistik:
   - Manfaat: MST membantu merancang rute pengiriman barang yang efisien, mengurangi biaya bahan bakar, waktu perjalanan, dan emisi karbon.
   - Fungsi: Menentukan rute pengiriman optimal untuk mengantarkan barang dari produsen ke konsumen.
3. Perencanaan Perkotaan:
   - Manfaat: MST dapat digunakan dalam perencanaan perkotaan untuk merancang jaringan jalan yang efisien, mengurangi kemacetan, dan meningkatkan mobilitas.
   - Fungsi: Menentukan jalur jalan yang optimal untuk menghubungkan berbagai area dalam suatu kota.

4. Manajemen Energi:
    - Manfaat: MST membantu dalam perencanaan distribusi energi listrik, meminimalkan kerugian daya dan meningkatkan efisiensi penggunaan energi.
    - Fungsi: Menentukan jalur distribusi listrik yang optimal dari sumber ke konsumen.
5. Perencanaan Jaringan Air dan Gas:
    - Manfaat: MST dapat digunakan untuk merancang jaringan distribusi air atau gas dengan efisiensi tinggi.
    - Fungsi: Menentukan jalur distribusi yang optimal untuk memastikan pasokan air atau gas yang efisien.
6. Rute Wisata:
    - Manfaat: MST dapat digunakan untuk merancang rute perjalanan wisata yang optimal, mengunjungi tempat-tempat menarik dengan efisien.
    - Fungsi: Menentukan jalur perjalanan wisata yang meminimalkan waktu perjalanan dan biaya.
7. Jaringan Sensor dan Internet of Things (IoT):
    - Manfaat: MST membantu merancang jaringan sensor atau IoT dengan penggunaan daya yang efisien.
    - Fungsi: Menentukan lokasi sensor atau perangkat IoT serta jalur komunikasi yang optimal.
8. Manajemen Sumber Daya Cloud Computing:
    - Manfaat: MST dapat digunakan untuk merancang rute koneksi antar server dalam lingkungan cloud computing.
    - Fungsi: Menentukan jalur komunikasi yang optimal untuk mengoptimalkan penggunaan sumber daya dalam sistem cloud.

Dengan penerapan MST, kehidupan sehari-hari dapat diatur dengan lebih efisien, menghasilkan penggunaan sumber daya yang lebih baik, dan mengurangi biaya operasional.

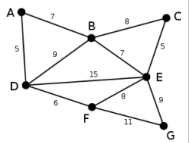
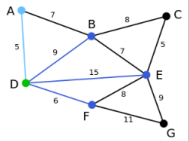
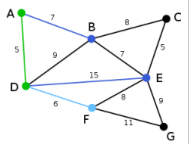
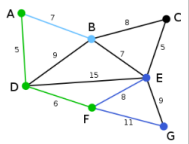
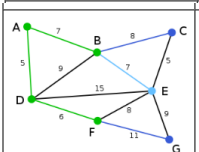
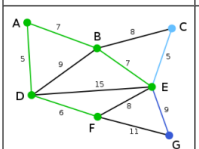
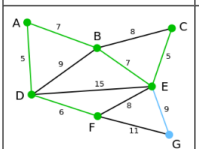
D. ALGORITMA PRIM

Algoritma Prim adalah suatu algoritma di dalam teori graf yang bertujuan menemukan Minimum Spanning Tree untuk menghubungkan graf berbobot. algoritma ini menemukan subset dari sebuah tepi yang membentuk sebuah Tree yang meliputi setiap titik nya. Dimana total beban dari setiap tepi di Tree diminimalkan. Jika Graf tidak terhubung, maka ini hanya akan menemukan sebuah Minimum Spanning Tree menjadi satu jalur untuk komponen yang terhubung.(Syahputra et al. 2016)

Langkah-langkah menentukan Algoritma Prim

Langkah 1 : Tentukan titik sembarang sebagai titik awal MST.
Langkah 2 : Ikuti langkah 3 sampai 5 hingga ada simpul yang tidak termasuk dalam MST (disebut simpul pinggiran).
Langkah 3 **:** Temukan sisi-sisi yang menghubungkan setiap simpul pohon dengan simpul pinggiran.
Langkah 4 : Temukan nilai minimum di antara sisi-sisi ini.
Langkah 5 **:** Tambahkan tepi yang dipilih ke MST jika tidak membentuk siklus apa pun.
Langkah 6 **:** Kembalikan MST dan keluar

Contoh Algoritma Prim

	Ini adalah graf berbobot awal. Graf ini bukan pohon karena ada circuit. Nama yang lebih tepat untuk diagram ini adalah graf atau jaringan. Angka-angka dekat garis penghubung adalah bobotnya. Belum ada garis yang ditandai, dan node **D** dipilih secara sembarang sebagai titik awal.
	Node kedua yang dipilih adalah yang terdekat ke **D**: **A** jauhnya 5, **B** 9, **E** 15, dan **F** 6. Dari keempatnya, 5 adalah yang terkecil, jadi kita tandai node **A** dan cabang **DA**.
	Node berikutnya yang dipilih adalah yang terdekat dari **D** *atau* **A**. **B** jauhnya 9 dari **D** dan 7 dari **A**, **E** jauhnya 15, dan **F** 6. 6 adalah yang terkecil, jadi kita tandai node **F** dan cabang **DF**.
	Algoritme ini berlanjut seperti di atas. Node **B**, yang jauhnya 7 dari **A**, ditandai. Di sini, cabang **DB** ditandai merah, karena baik node **B** dan node **D** telah ditandai hijau, sehingga DB tidak dapat digunakan.
	Dalam hal ini, kita dapat memilih antara **C**, **E**, dan **G**. **C** jauhnya 8 dari **B**, **E** 7 dari **B**, dan **G** 11 dari **F**. **E** yang terdekat, jadi kita tandai node **E** dan cabang **EB**. Dua cabang lain ditandai merah, karena kedua node yang terhubung telah digunakan.
	Di sini, node yang tersedia adalah **C** dan **G**. **C** jauhnya 5 dari **E**, dan **G** 9 dari **E**. **C** dipilih, jadi ditandai bersama dengan cabang **EC**. Cabang **BC** juga ditandai merah.
	Node **G** adalah satu-satunya yang tersisa. Jauhnya 11 dari **F**, dan 9 dari **E**. **E** lebih dekat, jadi kita tandai cabang **EG**. Sekarang semua node telah terhubung, dan pohon rentang minimum ditunjukkan dengan warna hijau, bobotnya 39.

Gambar 6.1: Langkah-langkah Algoritma Prim

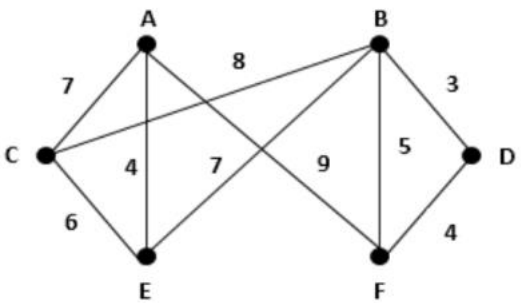
E. ALGORITMA SOLIN

Algoritma Solin menentukan pohon merentang minimum (Minimum Spanning Tree) dengan cara melakukan penghapusan sisi-sisi (Ruas) yang tidak menyebabkan graf menjadi tidak terhubung atau membentuk sirkuit. Penghapusan tersebut dimulai dari sisi atau ruas yang memiliki bobot terbesar hingga terkecil.Penghapusan dilakukan setelah sebelumnya sisi-sisi(Ruas) pada graf diurutkan berdasarkan bobotnya dari besar ke kecil.(Kurniawan, Wamiliana, and Fauzi 2016)

Langkah-langkah mencari pohon perentang minimum dengan Algoritma Solin sebagai berikut:

1. Urutkan ruas (edges) dari Graf G menurut bobotnya, dari besar ke kecil.
2. Lakukan penghapusan ruas berdasarkan urutan yang sudah dilakukan, dengan ketentuan bahwa penghapusan ruas tersebut tidak menyebabkan graf menjaditidak terhubung atau membentuk sirkuit.

Contoh Algoritma Solin



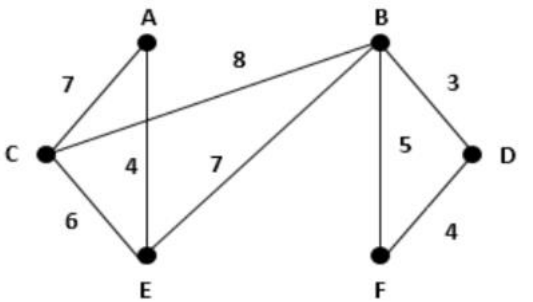
Gambar 6.2: Graf Berbobot G

1. Urutkan ruas – ruas dari graf berbobot G dengan bobot terbesar sampai bobot yang terkecil, diperoleh

AF	BC	AC	BE	CE	BF	AE	DF	BD
9	8	7	7	6	5	4	4	3

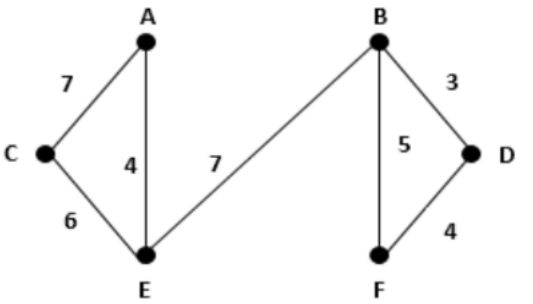
2. Lakukan penghapusan ruas berdasarkan urutan yang sudah dilakukan, denganketentuan bahwa penghapusan ruas tersebut tidak menyebabkan graf menjadi tidak terhubung atau membentuk sirkuit.

   Hapus ruas AF, karena tidak memutus graf G, diperoleh



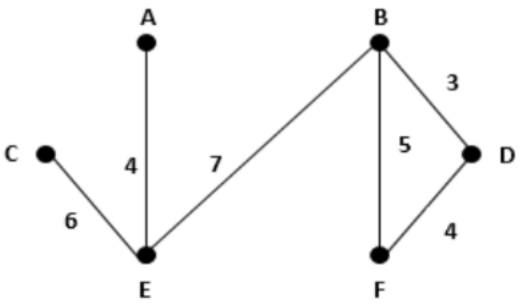
Gambar 6.3: Penghapusan ruas AF

   Hapus ruas BC, karena tidak memutus graf G, diperoleh



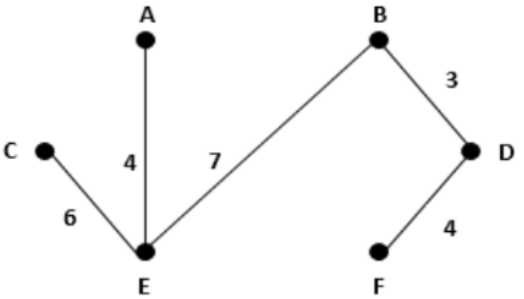
Gambar 6.4: Penghapusan ruas BC

Hapus ruas AC, karena tidak memutus graf G, diperoleh



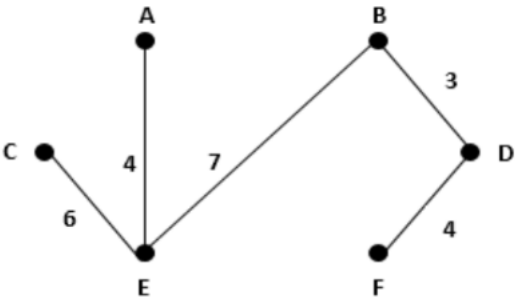
Gambar 6.5: Penghapusan ruas AC

- Hapus ruas BE (tidak boleh) karena memutus graf G
- Hapus ruas CE (tidak boleh) karena memutus graf G
- Hapus ruas BF (boleh) karena tidak memutus graf G dan diperoleh



Gambar 6.6: Penghapusan ruas BF

- Berikutnya ruas AE, DF, dan BD tidak dapat dihapus karena memutus graf G. Jadi minimum spanning tree dari graf berbobot G adalah sebagai berikut :



Gambar 6.7: Hasil Minimum Spanning Tree dengan Algoritma Solin

- Jadi jumlah bobot minimal spanning treedari graf G adalah 6 + 4 + 7 + 3 + 4 = 24

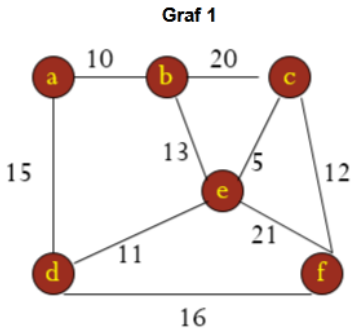
F. ALGORITMA KRUSKAL

Algoritma Solin menentukan pohon merentang minimum (Minimum Spanning Tree) dengan cara melakukan penghapusan sisi-sisi (Ruas) yang tidak menyebabkan graf menjadi tidak terhubung atau membentuk sirkuit. Penghapusan tersebut dimulai dari sisi atau ruas yang memiliki bobot terkecil hingga terbesar. (Anon 2014)

Langkah-langkah Algoritma Kruskal dalam mencari MST:

1. Mengurutkan semua sisi dalam graf berdasarkan bobotnya.
2. Membuat himpunan subset dari setiap simpul awal.
3. Memilih sisi dengan bobot terkecil dari sisi-sisi yang telah diurutkan.
4. Memeriksa apakah penambahan sisi tersebut membentuk siklus.
5. Jika tidak membentuk siklus, tambahkan sisi tersebut ke dalam himpunan solusi.

6. Ulangi langkah 3-5 hingga semua simpul terhubung dan MST terbentuk.



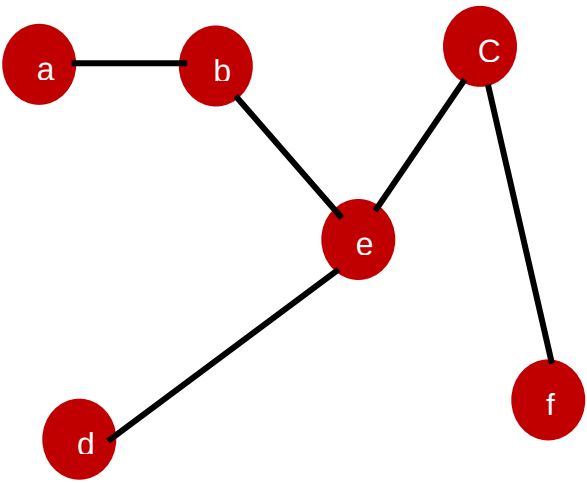
Gambar 6.8: Graf 1 Berbobot

1. Urutkan ruas – ruas dari graf berbobot G dengan bobot terkecil sampai bobot yang terkecil, diperoleh

EC	AB	DE	CF	BE	AD	DF	BC	EF
5	10	11	12	13	15	16	20	21

2. Lakukan penghapusan ruas berdasarkan urutan yang sudah dilakukan, denganketentuan bahwa penghapusan ruas tersebut tidak menyebabkan graf menjadi tidak terhubung atau membentuk sirkuit.

3. Didapatkan hasil EC= 5, AB = 10, DE =11, CF=12, BE=13
   Nilai Bobot 5 + 10 + 11 + 12 + 13 = 51



Gambar 6.9: Hasil Graf 1 Algoritma Kruskal

G. RANGKUMAN

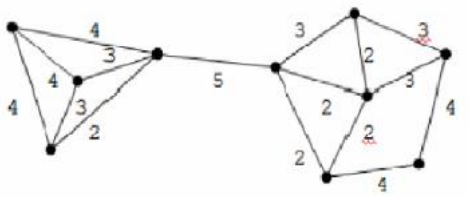
Berdasarkan Uraian diatas Minimum Spanning Tree adalah sub-graf dari graf terhubung tak berarah yang memuat semua simpul dari graf tersebut dengan jumlah sisi yang seminimal. Algoritma yang digunakan memecahkan masalah minimal spanning tree terdiri dari 3 jenis algoritma Algoritma yaitu Algoritma Prim, Solin dan Kruskal. Ciri dari Minimum Spanning Tree adalah Graf Terhubung dan Tidak Memiliki Sirkuit.

Penerapan Minimal Spanning Tree dapat memecahkan masalah yang meliputi berbagai bidang dalam kehidupan sehari-hari, yakni dalam bidang Tranportasi, Jaringan Komunikasi dan Logistik, Perencanaan tata kota, Rute wisata dala lain-lain.

Algoritma Prim dan Kruskal dapat menghasilkan nilai bobot minimum yang dapat menjadi Solusi dalam perencanaan biaya dalam suatu proyek dan penataan suatu kota.

H. TES FORMATIF

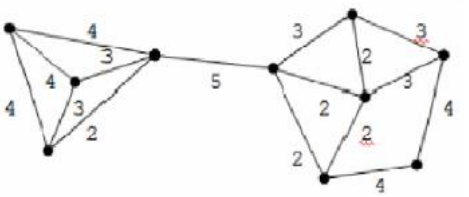
- Dengan Algoritma Solin Tentukan Nilai Bobot Graf G1



Graf G1

a) 25
b) 27
c) 15
d) 29
e) 17

- Dengan Algoritma Kruskal Tentukan Nilai Bobot Graf G1



Graf G1

a) 25
b) 27
c) 15
d) 29

e) 17

I. LATIHAN

Selesaikan Graf 2 menggunakan Algoritma Prim, Solin dan Kruskal

KEGIATAN BELAJAR 7

GEOMETRI ALGORITHM

DESKRIPSI PEMBELAJARAN

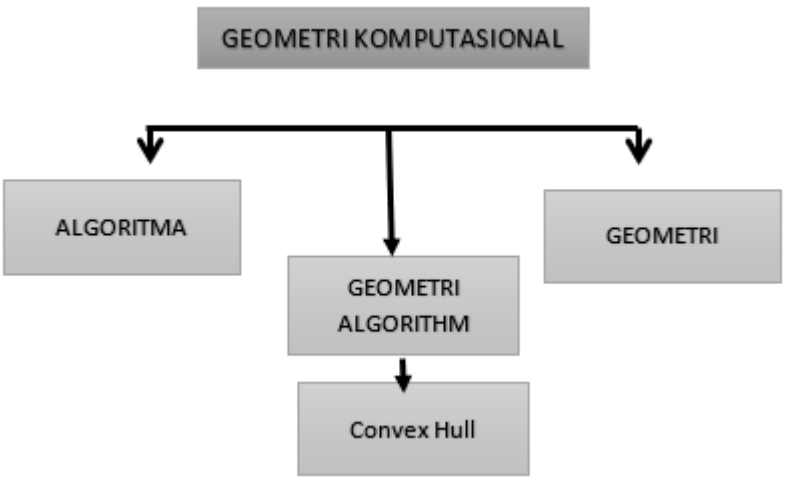
Pada bab ini mahasiswa mempelajari tentang pengembangan dan analisis algoritma untuk menyelesaikan masalah-masalah geometri. Diharapkan mahasiswa dapat melihat bagaimana konsep-konsep algoritma geometri dapat diterapkan dalam dunia nyata, termasuk aplikasi dalam ilmu komputer grafis, pengolahan citra, simulasi fisika, dan bidang lainnya.

KOMPETENSI PEMBELAJARAN

Setelah mengikuti perkuliahan ini diharapkan mahasiswa dan mahasiswi memiliki pengetahuan dan kemampuan :

1. Mampu memahami konsep Geometri Komputasional
2. Mampu merancang dan Menganalisis Algoritma Geometri
3. Mampu memahami dan menerapkan Konsep Convex Hull

PETA KONSEP PEMBELAJARAN



A. PENGENALAN GEOMETRI KOMPUTASIONAL

Geometri komputasional adalah cabang ilmu komputer yang berkaitan dengan pemrosesan dan analisis objek geometri menggunakan algoritma dan metode komputasional. Bidang ini mencakup berbagai aplikasi, mulai dari perancangan grafika komputer hingga pemodelan dan analisis objek geometris dalam dunia nyata.

Beberapa masalah geometri murni muncul dari studi tentang algoritma geometri komputasi, dan masalah seperti itu juga dianggap sebagai bagian dari geometri komputasi. Geometri komputasi merupakan salah satu bidang komputasi tertua dalam sejak zaman kuno, meskipun geometri komputasi modern yang saat ini masih dalam perkembangan.

Analisi algoritma adalah pusat dari geometri komputasi, yang memiliki peran signifikan dalam mengolah kumpulan data yang jumlahnya puluhan atau bahkan ratusan juta. Untuk himpunan seperti itu, perbedaan antara $0 (n ^ { 2 })$ dan $0 (n \log n)$ bias saja diartikan sebagai perbedaan antara hari dan detik dalam komputasi..

Cabang utama geometri komputasi adalah:

- Geometri komputasi kombinatorial , juga disebut geometri algoritmik , yang menangani objek geometris sebagai entitas diskrit . Sebuah buku landasan dalam subjek oleh Preparata dan Shamos tanggal penggunaan pertama dari istilah "geometri komputasi" dalam pengertian ini pada tahun 1975.
- Geometri komputasi numerik , juga disebut geometri mesin , desain geometris berbantuan komputer (CAGD), atau pemodelan geometris , yang terutama berhubungan dengan representasi objek dunia nyata dalam bentuk yang sesuai untuk komputasi komputer dalam sistem CAD / CAM. Cabang ini dapat dilihat sebagai pengembangan lebih lanjut dari geometri deskriptif dan sering dianggap sebagai cabang grafik

komputer atau CAD. Istilah "geometri komputasi" dalam arti ini telah digunakan sejak tahun 1971.

B. PENGERTIAN DAN KONSEP ALGORITMA GEOMETRI

Algoritma merupakan serangkaian langkah atau aturan yang terdefinisi degan jelas untuk menyelesaikan suatu tugas atau masalah tertentu. Algoritma digunakan dalam berbagai konteks, termasuk matematika, ilmu computer, pemrograman, dan bidang lainnya. Algoritma biasanya digunakan sebagai panduan untuk menyelesaikan masalah atau mencapai tujuan tertentu.

Algoritma berguna untuk membantu pemrogram (programmer) dalam memecahkan masalah. Mereka mengembangkan sebuah algoritma untuk menghasilkan serangkaian solusi yang dibutuhkan dalam pemecahan masalah. Algoritma ini akan diimplementasikan ke dalam sebuah program.

Sedangkan geometri merupakan suatu ilmu didalam system matematika yang didalamnya mempelajari garis, ruang, dan volume yang bersifat abstrak dan berkaitan satu sama lain, mempunyai garis dan titik sehingga menjadi sebuah symbol seperti bentuk persegi, segitiga, lingkaran dan lain-lain.



Gambar 7.1: ilustrasi Eropa dan Arab berlatih Geometri Abad 15

Untuk mempresentasikan konsep algoritma geometri, yaitu Algoritma geometri merupakan cabang dari ilmu yang dikhususkan untuk mempelajari algoritma yang dapat dinyatakan dalam suatu geometri.

Ilmu ini membahas tentang bagaimana membentuk polygon dari sekumpulan titk, pendeteksian suatu titik terhadap suatu polygon apakah titik itu ada dalam polygon atau tidak, proses pencarian suatu titik dalam range tertentu, yang nantinya akan diaplikasikan dalam proses pencarian sejumlah data, cara mencari titik-titik terluar perpotongan garis dan mencari titik terdekat

Tujuannya adalah bahwa algoritma geometri dapat digunakan untuk membantu manusia dalam menyelesaikan permasalahan yang berhubungan dengan bentuk-bentuk geometri, juga berhubungan dengan titik dan garis sebagai dasar dari bangun geometri.

1. **Computer Graphics**
   Kegunaan algoritma geometri yaitu untuk menggambarkan, membuat, dan memvisualisasikan dunia nyata yang kompleks ke dunia virtual. Seperti yang sering kita lihat di film-film, video games, atau simulasi virtual.
2. **Shape Reconstruction**
   Yaitu merekonstruksi suatu objek 2D ke dalam model computer 3D. biasanya digunakan untuk pencitraan alat medis, mikroskop, geologi, dan lain-lain.
3. **Computer Vision**
   Yaitu untuk pemulihan struktur 3D dari gambar 2D, terutama yang menggunakan informasi stereoscopic dan gerak. Pengaplikasian ini juga termasuk segmentasi citra untuk mengekstrak sebuah informasi tentang objek dalam sebuah adegan.

Beberapa hal yang bias diaplikasikan dari ilmu Algoritma Geometrik:

4. **Geographical Informations Systems**
   Pengaplikasian dalam bidang ini meliputi pemodelan dan perkiraan medan yang rumit, serta analisis dari klasifikasi lahan, perencanaan pembangunan, dan lainna. Untuk system transportasi juga digunakan untuk perencanaan rute, pemantauan kepadatan lalu lintas, dan lain-lain.
5. **Mesh Generation**
   Digunakan dalam pemodelan elemen hingga struktur teknik dan cairan (Hydro dan Aero) misalnya simulasi terowongan angina, peramalan cuaca, dan lain-lain. Mesh Generation adalah aspek geometri yang terkait dengan algoritma untuk membangun pemodelan baik pada objek 2D maupun 3D.
6. **Robotics**
   Algoritma geometri diperlukan untuk memecahkan masalah perencanaan gerak robot.

C. REPRESENTASI TITIK, GARIS, POLIGON

Representasi titik, garis, dan polygon adalah bagian penting dari pemrograman grafis komputasi geometri, dan banyakn aplikasi lainnya. Berikut adalah beberapa cara umum untuk mempresentasikan titik, garis, dan polygon:

1. Representasi Titik
   a. Koordinat Kartesian 2D : merepresentasikan titik dengan sepasang koordinat (x, y) dalam system koordinat kartesian.
   b. Koordinat Kartesian 3D : merepresentasikan titik dengan tiga koordinat (x, y, z) dalam system koordinat kartesian 3D.
   c. Matriks Homogen : merepresentasikan titik sebagai matriks homogeny, terutama digunakan dalan transformasi geometris.

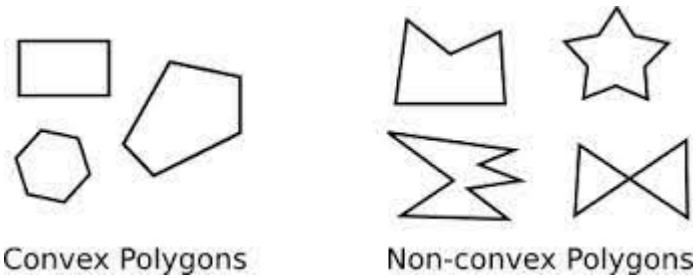
2. Representasi Garis
    a. Rumus Linear : merepresentasikan garis menggunakan persamaan linear, yaitu y = mx + b (untuk 2D) atau ax + by + c = 0 (untuk umum).
    b. Parameterisasi : merepresentasikan garis sebagai fungsi parameter, seperti P(t) = P0 + t * (P1 – P0), di mana P0 dan P1 adlah titik-titik ujung garis.
    c. Vektor Arah dan Titik Awal : merepresentasikan garis menggunakan vektor arah dan titik awal.

3. Representasi Poligon
    a. Urutan Titik : merepresentasikan polygon dengan menyusun urutan titik-titik yang membentuk batas polygon.
    b. Daftar Simpul (Vertex) : merepresentasikan polygon dengan menyimpan daftar simpul (vertex) yang membentuk polygon.
    c. Matriks : merepresentasikan polygon sebagai matriks yang menyimpan koordinat simpul dan koneksi antara simpul-simpul tersebut.
    d. Rantai simpleks (Simlicial Chain) : Mempresentasikan polygon sebagai rantai simpleks, yang terdiri dari simul, tepi, dan wajah.
    e. Half-Edge Data Structure : Struktur data khusus yang digunakan untuk mempresentasikan polygon dengan menyimpan informasi setiap sisi (half-edge), simpul, dan wajah.

D. CONVEX HULL

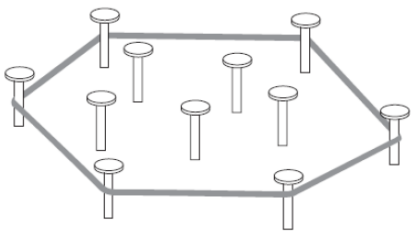
Salah satu hal penting dalam komputasi geometri adalah menentukan *convex hull* dari kumpulan titik. Himpunan titik pada bidang planar disebut *convex* jika untuk sembarang dua titik pada bidang tersebut (missal p dan q), seluruh segmen garis yang berakhir di p dan q berada pada himpunan tersebut.

Contoh pada gambar 1 adalah polygon yang *convex,* sedangkan gambar 7.2 menunjukan contoh yang *non-convex*.



Gambar 7.2: Gambar Polygon Convex dan Non- Convex

Convex hull dari himpunan titik S adalh himpunan *convex* terkecil (*convex polygon)* yang mengandung S



Gambar 7.3: Gambar Himpunan convex polygon

Untuk dua titik, maka convex hull berupa garis yang menghubungkan 2 titik tersebut.



Untuk tiga titik yang terletak pada satu garis, maka *convex hull* adalah sebuah garis yang menghubungkan dua titik terjauh.

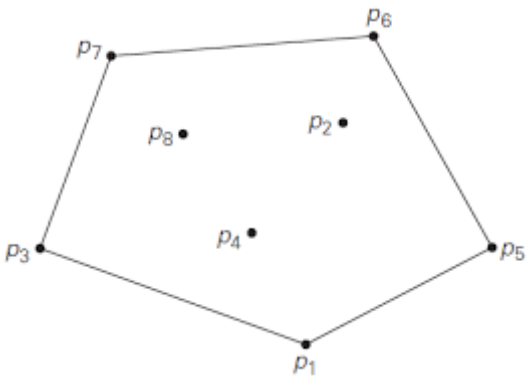


Sedangkan *convex hull* untuk tiga titik yang tidak terletak pada satu garis adalah sebuah segitiga yang menghubungkan ketiga titik tersebut.



Untuk titik yang lebih banyak dan tidak terletak pada satu garis, maka convex hull berupa polygon *convex* dengan sisi berupa garis yang menghubungkan beberapa titik pada S.

Contoh *convex hull* untuk delapan titik dapat dilihat sebagai berikut



Gambar 7.4: Gambar Convex Hull Delapan Titik

Penerapan *convex hull* memiliki berbagai manfaat diberbagai bidang, termasuk komputasi geometri, grafika computer, pengolahan gambar dan algoritma optimasi. Berikut adalah beberpa pemanfaatan *convex hull :*

1. Komputasi Geometri
   a. Algoritma Perbandingan Poin : *convex hull* digunakan dalam masalah perbandingan geometri, seperti menentukan titik-titik yang membentuk batas luar dari suatu polygon.

   b. Pencarian Titik Terjauh : dalam beberapa kasus, *convex hull* digunakan untuk mencari titik yang terjauh dari suatu himpunan titik.

2. Grafika Komputer
   Clipping (Pengubahan Tampilan) : *convex hull* dapat digunakan dalam algoritma pengubahan tampilan untuk menentukan area yang terlihat atau tidak terlihat dari suatu objek terhadap pandangan kamera atau tampilan layar.

3. Pengolahan Gambar
   Segmentasi gambar : *convex hull* dapat digunakan dalam segmentasi gambar untuk mengelompokkan pixel-pixel ke dalam region yang sesuai dengan *convex hull.*

4. Pemodelan Geometris
   Desain Objek 3D : dalam desain objek 3D, *convex hull* dapat digunakan untuk mengelompokkan titik-titik yang membentuk batas luar objek.

5. Optimasi
   Optimasi Transportasi dan Logistik : dalam masalah rute atau distribusi, *convex hull* dapat membantu dalam mengidentifikasi jalur optimal atau wilayah yang mencakup seluruh rute distribusi.

6. Keamanan
   Deteksi Intrusi : *convex hull* dapat digunakan dalam system keamanan untuk mendeteksi area atau zona yang memerlukan pemantauan khusus atau keamanan tambahan.
7. Ekonomi dan Keuangan

Manajemen Portofolio Keuangan : *convex hull* dapat digunakan dalam analisis portofolio keuangan untuk mengidentifikasi kombinasi optimal dari asset dengan risiko minimal.

E. CONVEX HULL DENGAN DIVIDE AND CONQUER

Tujuan: menemukan kumpulan titik 'terluar' yang membentuk *convex hull*

Ide dasar: menggunakan algoritma *Quicksort*
S: himpunan titik sebanyak *n*, dengan *n*>1, yaitu titik $p_1(x_1, y_1)$ hingga $p_n(x_n, y_n)$ pada bidang kartesian dua dimensi.

Kumpulan titik diurutkan berdasarkan nilai absis yang menaik, dan jika ada nilai absis yang sama, maka diurutkan dengan nilai ordinat yang menaik

p_1 dan p_n adalah dua titik ekstrim yang akan membentuk *convex hull* untuk kumpulan titik tersebut.

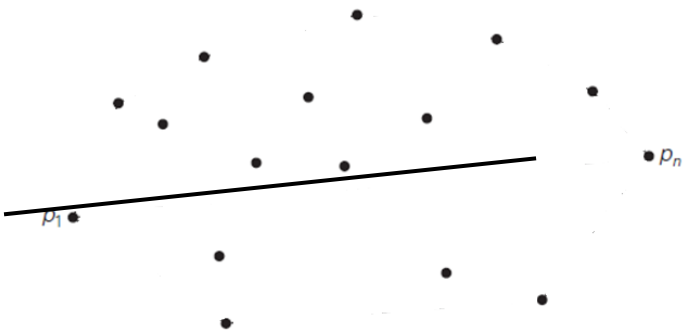
Ilustrasi:



F. IDE DIVIDE AND CONQUER

1. Garis yang menghubungkan p_1 dan p_n ($p1pn$) membagi *S* menjadi dua bagian yaitu S1 (kumpulan titik di sebelah kiri atau atas garis $p1pn$) dan S2 (kumpulan titik di sebelah kanan atau bawah garis $p1pn$).

   Ilustrasi :



   Untuk memeriksa apakah sebuah titik berada di sebelah kiri (atau atas) suatu garis yang dibentuk dua titik, gunakan penentuan determinan:

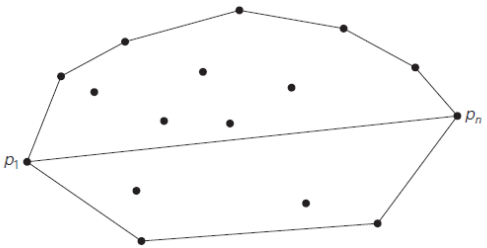
   Titik (x3,y3) berada di sebelah kiri dari garis ((x1,y1),(x2,y2)) jika hasil determinan positif

$$\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} = x_1y_2 + x_3y_1 + x_2y_3 - x_3y_2 - x_2y_1 - x_1y_3$$

2. Semua titik pada S yang berada pada garis $p1pn$ (selain titik $p1$ dan pn) tidak mungkin membentuk *convex hull*, sehingga bisa diabaikan dari pemeriksaan.

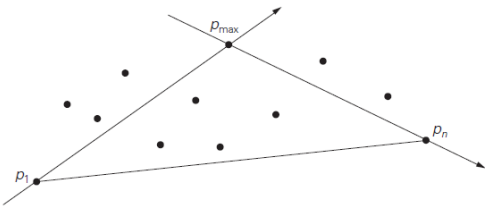
3. Kumpulan titik pada S1 bisa membentuk *convex hull* bagian atas, dan kumpulan titik pada S2 bisa membentuk *convex hull* bagian bawah → terapkan D & C.

   Ilustrasi:



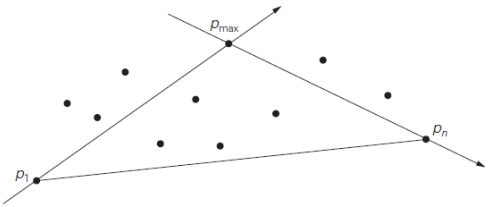
4. Untuk sebuah bagian (misal S1), terdapat dua kemungkinan:
   - Jika tidak ada titik lain selain S1, maka titik $p1$ dan pn menjadi pembentuk *convex hull* bagian S1
   - Jika S1 tidak kosong, pilih sebuah titik yang memiliki jarak terjauh dari garis $p1pn$ (misal p_{max}). Jika terdapat beberapa titik dengan jarak yang sama, pilih sebuah titik yang memaksimalkan sudut p_{max} p_1 p_n

   Semua titik yang berada di dalam daerah segitiga p_{max} p_1 p_n diabaikan untuk pemeriksaan lebih lanjut



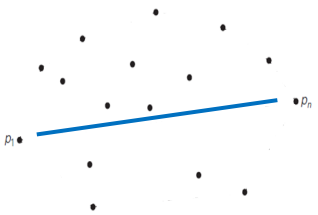
5. Tentukan kumpulan titik yang berada di sebelah kiri garis $p1pmax$ (menjadi bagian $S_{1,1}$), dan di sebelah kanan garis $p1pmax$(menjadi bagian $S_{1,2}$)

Titik-titik di dalam segitiga tidak diproses, karena tidak mungkin membentuk convex hull

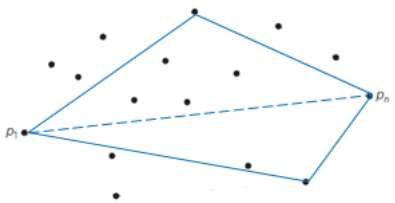


6. Lakukan hal yang sama (butir 4 dan 5) untuk bagian S2, hingga bagian ‘kiri’ dan ‘kanan’ kosong

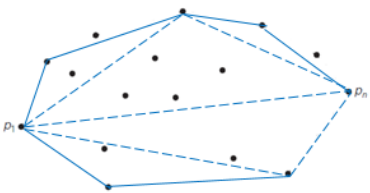
7. Kembalikan pasangan titik yang dihasilkan



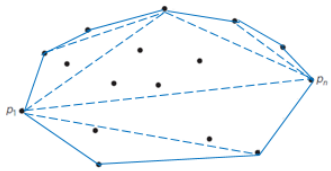
(I)



(II)

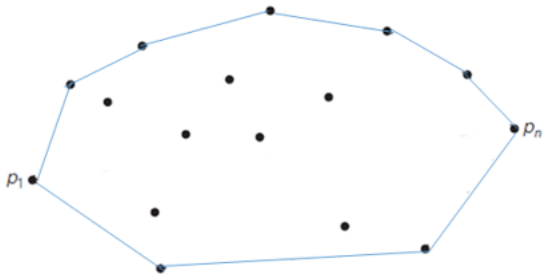


(III)



(IV)

Hasil akhir:



G. RANGKUMAN

Berdasarkan uraian di atas dimulai dari perkembangan ilmu geometri komputasi.yangmemproses dan menganalisis objek geometri menggunakan algoritma dan metode komputasional. Bidang ini mencakup berbagai aplikasi, mulai dari perencanaan grafika komputer hingga pemodelan dan analisis objek geometris dalam dunia nyata, Beberapa masalah geometri murni muncul dari studi tentang algoritma geometri komputasi, dan masalah seperti itu juga dianggap sebagai bagian dari geometri komputasi.

Geometri komputasi merupakan salah satu bidang komputasi tertua dalam sejak zaman kuno, meskipun geometri komputasi modern yang saat ini masih dalam perkembangan. Untuk mempresentasikan konsep algoritma geometri, yaitu Algoritma geometri merupakan cabang dari ilmu yang dikhususkan untuk mempelajari algoritma yang dapat dinyatakan dalam suatu geometri. Tujuannya adalah bahwa algoritma geometri dapat digunakan untuk membantu manusia dalam menyelesaikan permasalahan yang berhubungan dengan bentuk-bentuk geometri. Untuk mempresentasikan algoritma geometri, maka kita harus mempeljari tentang bagaimana mempresentasikan titik, garis, dan polygon. Salah satu hal penting dalam komputasi geometri adalah menentukan *convex hull* dari kumpulan titik. Himpunan titik pada

bidang planar disebut *convex* jika untuk sembarang dua titik pada bidang tersebut (missal p dan q), seluruh segmen garis yang berakhir di p dan q berada pada himpunan tersebut. Penerapan *convex hull* memiliki berbagai manfaat diberbagai bidang, termasuk komputasi geometri, grafika computer, pengolahan gambar dan algoritma optimasi. Antara lain pemanfaat *convex hull* adalah dalam bidang komputasi geometri, grafika komputer, pengolahan gambar, pemodelan geometris, optimasi, serta bervariasi tergantung pada konteks dan aplikasi spesifiknya.

H. TES FORMATIF

1. Apa yang dimaksud dengan *Convex Hull* ?
   a. Himpunan terkecil titik dalam ruang *Euclidean* yang mencakup beberapa titik.
   b. Himpunan titik yang membentuk polygon *convex.*
   c. Area yang dihasilkan oleh beberapa titik dalam suatu lingkaran.
   d. Titik tengah dari himpunan titik.
   e. Salah semua

2. Apakah *convex hull* dapat diterapkan pada himpunan titik dalam ruang tiga dimensi (3D) ?
   a. Ya, bisa
   b. Tidak, hanya bias dalam ruang dua dimensi (2D)
   c. Hanya bias jika titik-titik membentuk kubus*.*
   d. Hanya bias jika titik-titik membentuk bidang datar
   e. Salah semua.

I. LATIHAN

Jelaskan mengapa analisis jarak dan area penting dalam algoritma geometri. Bagaimana algoritma Shoelace digunakan untuk menghitung luas polygon? Berikan contoh situasi di mana analisis jarak atau area dapat diterapkan.

DAFTAR PUSTAKA

Abdussakir, 2009. Pembelajaran Geometri Sesuai Teori Van Hiele. Jurnal. Desember.

Adib, M. (2021, Februari 4). Memahami Stack, Struktur Data yang Mudahkan Programmer Jalankan Tugasnya. Diambil kembali dari Glints: https://glints.com/id/lowongan/stack-adalah/

Ahmad Talha Siddiqui, S. A. (2024). Data Structure Using C theory and Program. New Delhi, India: CRS Publisher.

Aini, Nurul 2012. Geometri 2. Malang: Intimedia

Akpan, N. P. &., and I. A. Iwok. 2017. A Minimum Spanning Tree Approach of Solving a Transportation Problem Optimal. Vol. 5.

Alkindi, Ulza, Nur Akhmad, Yogo Kartiko, and Triono Putro. 2018. "Implementasi Algoritma Breadth First Search Pada Pacman Untuk Mengatur Pergerakan Karakter." Jurnal Riset Komputer (JURIKOM) 5(6):599–604.

An, L., and S. Chavez. 2000. 'A Fast Implementation of the Minimum Spanning Tree Method for Phase Unwrapping'. IEEE Transactions on Medical Imaging 19(8). doi: 10.1109/42.876306.

Anon. 2014. 'ALGORITMA PRIM DAN KRUSKAL DALAM MENCARI MINIMUM SPANNING TREE PADA BAHASA PEMROGRAMAN C'. JURNAL SISTEM INFORMASI UNIVERSITAS SURYADARMA 8(2). doi: 10.35968/jsi.v8i2.711.

Anon. 2020. ‘Enhancing Routing for Wireless Ad-Hoc Networks to Improve Quality of Service in Mobile Learning’. International Journal of Advanced Trends in Computer Science and Engineering 9(5). doi: 10.30534/ijatcse/2020/199952020.

Anshary, M. A., Hidayar, E. W., & Amalia, T. 2020. Protoype Program Hand Gesture Recognize Using Convex Hull Method and Convexity Defect on Android. Jurnal Online Informatika, 205-211.

Bharath-Kumar, Kadaba, and Jeffrey M. Jaffe. 1983. ‘Routing to Multiple Destinations in Computer Networks’. IEEE Transactions on Communications 31(3). doi: 10.1109/TCOM.1983.1095818.

Binary tree Data Structure. (n.d.). Retrieved January 4, 2024, from https://www.geeksforgeeks.org/binary-tree-data-structure/

Briscoe, Bob, Anna Brunstrom, Andreas Petlund, David Hayes, David Ros, Ing Jyh Tsang, Stein Gjessing, Gorry Fairhurst, Carsten Griwodz, and Michael Welzl. 2016. ‘Reducing Internet Latency: A Survey of Techniques and Their Merits’. IEEE Communications Surveys and Tutorials 18(3).

Bud, J. R. 2020. Python 3 Object-Oriented Programming: Menerapkan Prinsip dan Pola Desain untuk Proyek Python yang Efektif (3rd ed.). Packt Publishing.

Buhaerah, Zulfiqar Busrah, and Herlan Sanjaya. 2019. Teori Graf Dan Aplikasinya.

Cecep Kurnia Sastradipraja, M. B. (2022). Buku Ajar Algoritma dan Struktur Data. Politeknik Fiksi Ganesha: Kaizen Media Publishing.

Daisma. (n.d.). Memahami Konsep Tree dalam Struktur Data Lengkap dengan Source Code Programnya. Retrieved January 4, 2024, from https://daismabali.medium.com/memahami-konsep-tree-dalam-struktur-data-lengkap-dengan-source-code-programnya-acbd0a8733d6

Deo, Narsingh. 2016. Graph Theory with Applications to Engineering & Computer Science.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms, Sixth Edition. In Scientific American (Vol. 251, Issue 3). https://doi.org/10.1038/scientificamerican0984-60

Hartanto, A. 2018. Belajar Cepat: Pemrograman Berorientasi Objek dengan Python. Elex Media Komputindo.

Hermawati, F.A. 2004. Pengolahan Citra Digital. Andi: Yogyakarta

Hougaard, Jens Leth, and Mich Tvede. 2015. 'Minimum Cost Connection Networks: Truth-Telling and Implementation'. Journal of Economic Theory 157. doi: 10.1016/j.jet.2014.12.009.

Khan, Afsana, Afrida Anzum Aesha, and Juthi Sarker. 2018. 'A New Algorithmic Approach to Finding Minimum Spanning Tree'. in 4th International Conference on Electrical Engineering and Information and Communication Technology, iCEEiCT 2018.

Kurniawan, D., W. Wamiliana, and C. S. N. Fauzi. 2016. 'Perbandingan Kompleksitas Algoritma Prim, Algoritma Kruskal, Dan Algoritma Sollin Untuk Menyelesaikan Masalah Minimum Spanning Tree'. Jurnal Komputasi.

Lafore, R. (2002). Data Structures and Algorithms in Java (2nd Edition). In Science. http://www.amazon.com/Data-Structures-Algorithms-Java-2nd/dp/0672324539

Lee, K. D., & Hubbard, S. (2014). Data structures and algorithms with Python. In Springer. http://shop.oreilly.com/product/0636920029557.do

Lippman, S. B., Lajoie, J., & Moo, B. E. 2019. C++ Primer (5th ed.). Addison-Wesley Professional.

Meyer, B. 1997. Object-Oriented Software Construction (2nd ed.). Prentice Hall.

Munir, Rinaldi & Maulidadevi, Nur Ulfa 2021. Algoritma Divide and Conquer. Bandung : Program Studi Teknik Informatika Sekolah Teknik Elektro dan Informatika ITB

Munir, Rinaldi. 2004. Algoritma dan Struktur Data dengan Bahasa Pemrograman C. Bandung: Informatika.

Penulis idwebhost. (2022, Januari 27). Tutorial: Pengertian Struktur Data dan Tipenya dalam Pemrograman! Diambil kembali dari idwebhost.com: https://idwebhost.com/blog/struktur-data-dan-tipenya/

Permana, H. 2020. Pemrograman Berorientasi Objek dengan PHP. Penerbit Buku Pendidikan Deepublish.

Prahasta E., 2002, Konsep-konsep Dasar Sistem Informasi Geografis, Informatika Bandung

Prahasta E., 2004, Sistem Informasi Geografis : Tutorial Arcview, Informatika Bandung.

Prakasta, Muhammad Lutfi, and Arif Munandar. 2023. “Pewarnaan Fraksional Fuzzy Pada Graf Fuzzy Beserta Aplikasinya Dalam Penjadwalan Ujian.” Jurnal Fourier 12(1):1–9. doi: 10.14421/fourier.2023.121.1-9.

Prasetyo, E. 2018. Pemrograman Berorientasi Objek dengan Visual Basic. Gava Media.

Prodi Teknik Informatika. (2016). Dokumen Praktikum: Struktur Data. Dipetik Desember 17, 2023, dari Universitas Pembangunan Jaya: https://ocw.upj.ac.id/files/Handout-TIF104-Modul-Praltikum-Struktur-Data.pdf

Rahayuningsih, Sri. 2018. Teori Graph Dan Penerapannya.

Romanuke, Vadim. 2023. ‘BUILDING MINIMUM SPANNING TREES BY LIMITED NUMBER OF NODES OVER TRIANGULATED SET OF INITIAL NODES’. Information and Telecommunication Sciences (1). doi: 10.20535/2411-2976.12023.41-50.

S, R. A. (2021). Struktur Data. Bandung.

Sarwosri. 2019. Pemrograman Berorientasi Objek dengan C++. Deepublish.

Schapire, R. E., & Freund, Y. (2012). Boosting Foundations and Algorithms. London, England: The MIT Press.

Setiawan, A. 2021. Pengantar Pemrograman Berorientasi Objek dengan Kotlin. Penerbit Informatika.

Setiawan, S., Manurung, R., & Azurat, A. (2023, September 07). Implementasi ADT: Linked - List. Dipetik Desember 15, 2023, dari aren.cs.ui.ac.id/sda/:

http://aren.cs.ui.ac.id/sda/resources/sda2010/07_linkedlist.pdf

Setya, Antonio. 2018. “Aplikasi Strategi Greedy Dan Depth First Search Dalam Pemampatan Data Menggunakan Algoritma Huffman.” (13516002).

Sigit. (2021, November 28). Perbedaan Array List dan Linked List di Bahasa Pemrograman. Dipetik Desember 07, 2023, dari PemburuKode: https://pemburukode.com/perbedaan-array-list-dan-linked-list-di-bahasa-pemrograman/

Sutanta, E. 2018. Pemrograman Berorientasi Objek dengan Java. Andi.

Suyanto. (2021). Struktur Data. Palembang: Universitas Bina Dharma.

Syahputra, Trinanda, Dedi Setiawan, Program Studi, Sistem Informasi, and Royal Kisaran. 2016. ‘IMPLEMENTASI ALGORITMA PRIM DENGAN TEORI GRAPH PADA WPF GRAPH’. Jurnal Teknologi Dan Sistem Informasi.

Trivusi. (2022, September 16). Struktur Data Linked List: Pengertian, Karakteristik, dan Jenis-jenisnya. Diambil kembali dari trivusi.web.id: https://www.trivusi.web.id/2022/07/struktur-data-linked-list.html?m=1

Ulum, M. B. (2018). Modul Kuliah Struktur Data Linked List. Dipetik Desember 14, 2023, dari lms-paralel.esaunggul.ac.id: https://lms-paralel.esaunggul.ac.id/pluginfile.php?file=%2F86227%2Fmod_resource%2Fcontent%2F1%2FModul+Struktur+Data-Linked+List.pdf

Wicaksono, A. 2019. Menguasai Pemrograman Berorientasi Objek dengan C#. Penerbit Andi.

Yuliyanti, S. (2023). Modul Kuliah Algoritma dan Struktur Data. Tasikmalaya: Universitas Siliwangi.

TENTANG PENULIS



Moh. Erkamim, S.Kom., M.Kom.

Seorang penulis, peneliti dan dosen tetap di Prodi Sistem Informasi Kota Cerdas, Universitas Tunas Pembangunan Surakarta. Penulis mengajar di mata kuliah pengantar sistem informasi, arsitektur komputer, komunikasi data dan jaringan, Tata Kelola teknologi informasi, pemrograman web dan freamwork, struktur data, etika profesi IT. Tidak hanya mengajar, penulis menjabat struktural sebagai Ketua Prodi D4 Sistem Informasi Kota Cerdas (SIKC) Universitas Tunas Pembangunan (UTP) Surakarta Tahun 2022-2026. Penulis Alumni S2 Teknik Informatika Universitas AMIKOM Yogyakarta, S1 Teknik Informatika Universitas Sahid Surakarta, D3 Teknik Informatika Universitas Sebelas Maret (UNS), selain akademisi juga merupakan praktisi software development di PT. Power Techno Solution. Ketertarikan dan tridharma penulis berfokus pada Tata Kelola Teknologi Informasi, Pemrograman Web/Freamwork, Komunikasi Data dan Jaringan, Sistem Pendukung Keputusan, Data Mining, Kecerdasan Buatan. Saat ini penulis menulis buku, dan aktif meneliti di bidangnya.



Iim Abdurrohim, S.T., M.T.

Seorang penulis dan dosen tetap Program Studi Teknik Informatika Fakultas Ilmu Komputer dan Sistem Informasi Universitas Kebangsaan Republik Indonesia – Bandung – Jawa Barat. Lahir di Kota Serang, 13 Oktober 1970 Provinsi Banten. Penulis merupakan ayah dari Khairani Ghassan Dhabitah, S.Ds., dengan seorang istri Letkol Sus Tri Hastuti, S.T., M.I.Pol. Menamatkan pendidikan tinggi dimulai dari program Diploma (D3) di PATI Politeknik ITB prodi Informatika, melanjutkan pendidikan program Sarjana (S1) prodi Teknik Informatika di STTIndonesia dan STMIK "AMIKBandung" kemudian menyelesaikan program Pasca Sarjana (S2) di Universitas Langlang Buana, Kota Bandung prodi Magister Teknik Informatika konsentrasi di bidang *Security Software Enggineering*. Pengalaman bekerja menjadi Direktur Utama di PT Bumi Raya Konsultan, sebelumnya sebagai Supervisor di PT LAPI Divusi perusahaan yang bergerak dibidang jasa Konsultansi Teknologi Informasi, sebelumnya sebagai *software enggineer* di Satgas Departemen Informatika Institut Teknologi Bandung sedangkan mengawali kariernya sebagai *programmer* dilingkungan Lembaga Afiliasi Penelitian dan Industri Institut Teknologi Bandung. Buku yang telah ditulis dan terbit berjudul: *Literasi Digital: Pengetahuan & Transformasi Terkini Teknologi Digital Era Industri 4.0 dan Society 5.0 pada Bagian 9.*



Siti Yuliyanti, S.T, M.Kom

seorang Penulis dan Dosen Prodi Teknik Informatika Fakultas Teknik Universitas Siliwangi Tasikmalaya. Lahir di Cimahi, 17 Juli 1988 Jawa Barat. Penulis merupakan anak kedua dari tiga bersaudara dari pasangan bapak Amid (Alm) dan Ibu Saryati. Penulis menyelesaikan pendidikan Program Sarjana (S1) di STMIK Bandung Program Studi Teknik Informatika dan menyelesaikan Program Pasca Sarjana (S2) di Institut Pertanian Bogor (IPB) Program Studi Ilmu Komputer dan saat ini konsentrasi penelitian di bidang Text Mining, Data Science, Data Mining, Machine Learning, juga aktif mengajar dengan mengampu matakuliah Pemrograman Dasar, Algoritma & Struktur data, Aljabar Linier & Matriks dan Data Mining.



Rahmat Karim, S.Si.,M.Si.

Seorang penulis dan dosen tetap Prodi Ilmu Komputer, Fakultas Teknologi Informasi, Universitas Universitas Sembilanbelas November Kolaka. Lahir di desa Lolibu, 26 Juli 1990, Kab. Buton Tengah, Sulawesi Tenggara. Penulis merupakan anak ke-dua dari tiga bersaudara. Pendidikan program Sarjana (S1) Universitas Halu Oleo Prodi Matematika dan menyelesaikan program Pasca Sarjana (S2) di Institut Teknologi Bandung “ITB” prodi Sains Komputasi.



Alkautsar Rahman, S.T., M.Kom.

Pria kelahiran Kota Baja Cilegon merantau ke Jawa Barat untuk meneruskan pendidikan S1 di Universitas Pasundan, setelah berhasil menyelesaikan pendidikan di bidang Informatika, penulis kembali ke kota kelahirannya melanjutkan karir di sebuah anak perusahaan Krakatau Steel, kembali penulis ingin melanjutkan study S2 di Universitas Komputer Indonesia dan memulai karirnya kembali menjadi dosen tetap di Universitas Kebangsaan Republik Indonesia Bandung.



Tb. M. Adrie Admira, S.T., M.Kom.

Seorang penulis dan dosen tetap Prodi Sistem Informasi Sekolah Tinggi Manajemen Informatika dan Komputer Jakarta STI&K, lahir 12 September 1981 di Jakarta Pusat. Penulis merupakan anak ke-tiga dari tiga bersaudara dari pasangan Bapak Drs. Tb. Tony Aziz (Alm) dan Ibu Dra. R.A. Riskiah, Apt. (Almh) Tinggal di Kota Tangerang, Banten. Pendidikan program Sarjana (S1) Universitas Gunadarma Jurusan Teknik Informatika Fakultas Teknologi Industri dan menyelesaikan program Pasca Sarjana (S2) Magister Teknologi Informasi di Sekolah Tinggi Manajemen Informatika dan Komputer Jakarta STI&K Tahun 2021. Buku yang telah ditulis dan terbit berjudul: *Digitalisasi Governance, selain menulis juga sebagai reviewer dan editor dibeberapa jurnal nasional dan aktif diberbagai Organisasi Keagamaan dan Kemasyarakatan.*



Achmad Ridwan, S.Kom., M.Kom.

Seorang penulis dan dosen tetap Prodi Sistem Informasi Fakultas Sains, Teknologi dan Matematika Universitas Muhammadiyah Kudus. Lahir di desa Jekulo, 28 Juni 1984 Jawa tengah.. Pendidikan program Serjana (S1) Universitas Muria Kudus Prodi Sistem Informasi dan menyelesaikan program Pasca Sarjana (S2) di Universitas Dian Nuswantoro "Udinus" Semarang Program Studi Teknik Informatika konsentrasi di bidang Sistem Informasi. Menjabat sebagai Ketua Program Studi Sistem Informasi Universitas Muhammadiyah Kudus. Buku yang pernah ditulis dan terbit diantaranya : Kompeten di dunia kerja dengan Ms.Word 2016. Daftar Paten yang pernah dibuat diantaranya : Karya cipta EC00202036349 10 Juni 2020 Learning Management System UMKU, Karya Cipta EC00202108184 25 Januari 2021 Aplikasi Antrian Wisuda Secara Drive Thru . Daftar Publikasi Artikel Jurnal Diantaranya : Developing Mathematical Exercise Software For Visually Impaired Students, The Comparison Of Accuracy Between Naïve Bayes Clasifier And C4. 5 Algorithm In Classifying Toddler Nutrition Status Based On Anthropometry Index, Penerapan Algoritma Naïve Bayes Untuk Klasifikasi Penyakit Diabetes Mellitus, Penerapan Teknik Bagging Untuk Meningkatkan Akurasi Klasifikasi Pada Algoritma C4. 5 Dalam Menentukan Blogger Profesional, Text Mining Sentimen Analisis Pengguna Aplikasi Marketplace Tokopedia Berdasar Rating dan Komentar Pada Google Play Store, Application of the C4. 5 Algorithm for Early Cervical Cancer Classification, Classification of Heart Failure using the Naïve Bayes Algorithm dll.

Penerbit :
PT. Sonpedia Publishing Indonesia

Buku Gudang Ilmu, Membaca Solusi Kebodohan, Menulis Cara Terbaik Mengikat Ilmu. Everyday New Books



Redaksi :
Jl. Kenali Jaya No 166
Kota Jambi 36129
Tel +6282177858344
Email: sonpediapublishing@gmail.com
Website: www.buku.sonpedia.com